기획 워크벤치

사용 설명서

게임 시스템·BM 기획자를 위한 밸런스 검증 도구 — 기능별 사용 방법과 계산 원리

강화 · 가챠 · 성장 표 · 스탯·전투 · 데이터 테이블 · 전투 시뮬레이터 · 패턴 프레임 · 브리핑

https://junhy96.github.io/gamedesign_workbench/

문서 기준: 2026년 10월 · 화면은 각 도구의 기본 템플릿 상태

차례

1. 기획 워크벤치 소개 6

1.1 이 문서를 읽는 방법 6

1.2 도구 한눈에 보기 6

1.3 실행과 데이터 7

1.4 화면 공통 구성 — 헤더 8

1.5 화면 공통 구성 — 도구 머리와 공통 버튼 8

1.6 스냅샷과 프로젝트 파일 10

1.7 공식 계산과 시뮬레이션 — 이 도구의 기본 원칙 10

1.8 공통 기능 — 툴팁·시드·입력 검증·결과 분포·프리셋 비교·기획서 11

2. 강화 확률·비용 설계 19

2.1 무엇을 하는 도구인가 19

2.2 화면 구성 19

2.3 빠르게 시작하기 20

2.4 설정 — 강화 구조 21

2.5 단계별 확률과 비용 표 23

2.6 총비용 분포와 예산 26

2.7 강화 경로 예시 27

2.8 달성 분위별 수치 28

2.9 재화별 기대 소모와 계산 검증 29

2.10 밸런스 도구 32

2.11 다른 탭과 연결 39

3. 가챠·수집 설계 40

3.1 무엇을 하는 도구인가 40

3.2 화면 구성 41

3.3 빠르게 시작하기 43

3.4 ① 등급과 확률 43

3.5 소환 레벨 스위치와 10연 보장 44

3.6 ② 목표 — 무엇을 몇 장 모으나 48

3.7 달성 확률·예산과 분위 49

3.8 왜 이만큼 드나 — 비용 원인 분해 50

3.9 계산 검증과 수렴 50

3.10 확률 공시표 54
3.11 판단 사례 — 마일리지가 꼬리를 자르는 방식 55
4. 스탯·전투 밸런스 56
4.1 무엇을 하는 도구인가 56
4.2 화면 구성 57
4.3 빠르게 시작하기 58
4.4 전투 방식 58
4.5 규칙 수식 60
4.6 엔티티와 스탯 62
4.7 아이템과 빌드 64
4.8 레벨 곡선과 승률 행렬 65
4.9 1:1 상세 67
4.10 배틀 시뮬레이터 (FIGHT · M.C) 69
4.11 레벨별 승률 (구간 리포트) 75
4.12 CP와 실제 승률 76
4.13 밸런스 도구 77
4.14 가져오기와 내보내기 79
5. 데이터 테이블 80
5.1 무엇을 하는 도구인가 80
5.2 화면 구성 81
5.3 빠르게 시작하기 82
5.4 일곱 개 테이블과 ID 참조 82
5.5 최종 스탯 계산 순서 84
5.6 검증 — 빨간 칸과 노란 칸 85
5.7 CSV와 JSON 86
5.8 속성 상성 87
5.9 저장과 공유 87
6. 전투 시뮬레이터 88
6.1 무엇을 하는 도구인가 88
6.2 화면 구성 89
6.3 빠르게 시작하기 90
6.4 규칙 설정 91
6.5 데미지 공식과 계산 순서 93
6.6 효과 · 그로기 · 페이즈 · 시너지 94

6.7 파티 구성과 클리어 분포 96
6.8 기여도 분석 97
6.9 교체 비교 99
6.10 조합 비교 100
6.11 전투 로그 101
6.12 기획서 — 전투 규칙 102

7. 패턴 프레임 검증기 105

7.1 무엇을 하는 도구인가 105
7.2 화면 구성 106
7.3 빠르게 시작하기 106
7.4 프레임 타임라인 107
7.5 보스 연계 표와 판정 108
7.6 계산 원리 109
7.7 PPT 내보내기 110

8. 성장 표 템플릿 (엑셀) 111

8.1 무엇을 하는 도구인가 111
8.2 화면 구성 112
8.3 빠르게 시작하기 113
8.4 ① 레벨 곡선 113
8.5 ② 공급원 — 한 행 = 콘텐츠 하나 115
8.6 ③ 세그먼트와 마일스톤 116
8.7 미리보기와 검증 결과 118
8.8 내보낸 엑셀의 구조 120
8.9 엑셀에서 콘텐츠 늘리기 122
8.10 계산 검증 123

9. 브리핑 124

9.1 화면 구성 124
9.2 탭별 지표와 목표 125
9.3 회의 직전 5분 사용법 126

10. 부가 도구 — 다이어그램과 PDF 127

10.1 자연어 다이어그램 127
10.2 PDF 도구 128

11. 도구를 엮어 쓰는 방법 130

11.1 강화 BM 가격 정하기 130
11.2 가챠 공시 준비 130
11.3 성장 곡선 초안을 팀 엑셀로 130
11.4 전투 데이터를 엑셀에서 고쳐 보스 난이도 맞추기 131
11.5 팀과 공유하기 131
11.6 자주 묻는 질문 132

부록 — 알고리즘·수학 사전 134

A.1 여러 도구가 함께 쓰는 방법 134
A.2 강화 확률·비용 설계 (2장) 134
A.3 가챠·수집 설계 (3장) 134
A.4 스탯·전투 밸런스 (4장) 134
A.5 데이터 테이블 (5장) 134
A.6 전투 시뮬레이터 (6장) 135
A.7 패턴 프레임 검증기 (7장) 135
A.8 성장 표 템플릿 (8장) 135

1. 기획 워크벤치 소개

기획 워크벤치는 게임 시스템·BM 기획자가 엑셀로 풀기 어려운 밸런스 계산을 브라우저에서 바로 확인하는 도구 모음입니다. 기획서의 숫자(확률·비용·성장률·스탯)를 넣으면 기대값과 분포, 문제가 생기는 구간과 그 원인을 함께 보여 줍니다.

1.1 이 문서를 읽는 방법

각 도구는 같은 순서로 설명합니다.

- **무엇을 하는 도구인가** — 어떤 질문에 답하는지, 언제 쓰는지
- **화면 구성** — 실제 화면 그림에 빨간 번호 상자로 만져야 할 곳을 표시하고, 그림 아래 표에서 번호별로 설명
- **빠르게 시작하기** — 템플릿에서 출발해 내 수치로 바꾸기까지의 순서
- **기능별 사용 방법** — 기능마다 하는 일, 입력 방법, 결과를 읽는 방법
- **쓰인 알고리즘·수학** — 보라색 상자. 그 기능의 어느 부분에 어떤 방법이 쓰였는지와 그 방법이 무엇인지

여러 도구에서 함께 쓰이는 방법(몬테카를로 시뮬레이션, 마르코프 체인, 동적 계획법 등)은 마지막 장 "부록 — 알고리즘·수학 사전"에 정의와 수식, 직관, 한계까지 자세히 정리했습니다. 본문의 보라색 상자는 "이 화면에서는 이렇게 쓰였다"에 집중하고, 원리가 궁금하면 부록을 보면 됩니다.

그림 속 숫자는 모두 각 도구의 **기본 템플릿**을 그대로 돌린 결과입니다. 템플릿 수치는 설계 예시이며 특정 게임의 실제 수치가 아닙니다.

1.2 도구 한눈에 보기

분류	도구	답하는 질문	핵심 계산
BM·성장	강화 확률 설계	+15까지 평균 얼마, 운 나쁜 1%는 얼마? 보조 아이템 가격이 적정한가?	흡수 마르코프 체인, 마르코프 결정 과정, 몬테카를로
BM·성장	가챠·수집 설계	이 유닛을 이만큼 모으려면 실제로 몇 번	포아송화 적분, 상태 분포

분류	도구	답하는 질문	핵심 계산
		뽑나? 천장·마일리지 효과는?	동적 계획법, 기하분포 시뮬레이션
BM·성장	성장 표 템플릿 (엑셀)	코어 유저가 14일차에 Lv30이 되려면 레벨 곡선은? 라이트 유저는 며칠 동안 레벨이 멈추나?	일차별 전진 계산, 이분 탐색, 수식 엑셀 생성
전투 밸런스	스탯·전투	CP가 높은데 지는 캐릭터가 있나? 상성을 50%로 맞추려면?	공격 횟수 분포 동적 계획법, Wilson 구간, 스피어만 상관
전투 밸런스	데이터 테이블	엑셀 전투 시트의 ID 참조가 깨진 곳은? 캐릭터의 최종 스탯은?	참조 무결성 검사, CSV 변환
전투 밸런스	전투 시뮬레이터	5인 파티가 이 보스를 몇 %로, 몇 턴에 잡나? 누가 빠지면 못 잡나?	이산 사건 시뮬레이션, 몬테카를로, 반대 가정
전투 밸런스	패턴 프레임 검증	이 타격을 보고 피할 수 있나, 외워야 하나?	회피 윈도우 계산, 최적 입력 동적 계획법
공유	브리핑	지난안 대비 무엇이 바뀌었고 목표를 통과했나?	스냅샷 비교, 목표 판정
부가 도구	다이어그램 · PDF	문장으로 흐름도 만들기 / PDF 편집	Mermaid, pdf-lib

1.3 실행과 데이터

- **설치 없음** — 웹 주소(https://junhy96.github.io/gamedesign_workbench/)를 열면 바로 씁니다. 로그인도 없습니다.
- **계산은 브라우저 안에서만** — 강화·가챠·성장 표·스탯 전투·데이터 테이블·전투 시뮬레이터·프레임·브리핑에 넣은 숫자는 서버로 가지 않습니다. 성장 표의 엑셀 파일도 브라우저 안에서 만듭니다. 다이어그램 부가 도구만, 사용자가 직접 넣은 API 키로 Claude API를 부릅니다.
- **자동 저장** — 각 탭의 설정은 이 브라우저의 저장소(localStorage)에 자동으로 저장됩니다. 다른 브라우저·다른 PC로 옮기려면 공유 링크나 JSON·프로젝트 파일을 씁니다.
- **공유 링크** — 설정 전체를 주소의 `#` 뒤에 담습니다. 주소의 `#` 뒷부분은 서버로 전송되지 않으므로 링크를 받은 사람의 브라우저에서만 풀립니다.

- **로컬 실행** — 저장소를 받은 경우 `npm run dev`로 http://localhost:5173 에서 엽니다 (ES 모듈이라 파일 더블클릭으로는 열리지 않음).

1.4 화면 공통 구성 — 헤더

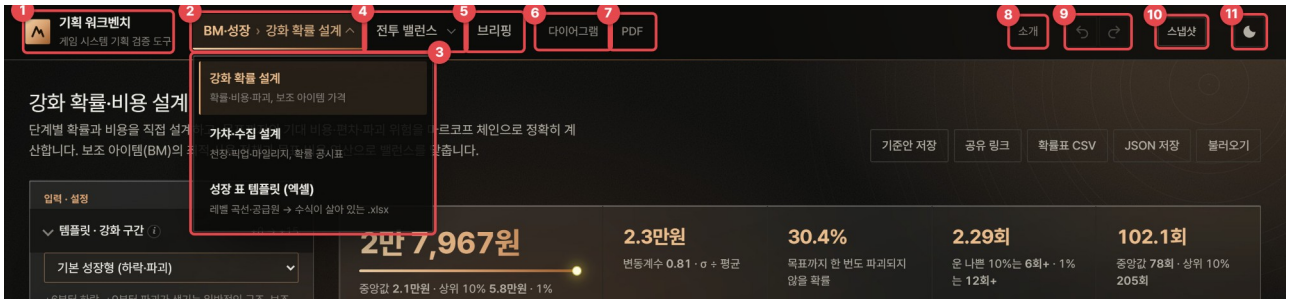


그림 1. 헤더와 분류 메뉴 (BM·성장 메뉴를 연 상태)

1	**로고** — 누르면 처음 화면(강화 확률 설계 탭의 맨 위)으로 돌아갑니다.
2	**BM·성장 분류** — 지금 보고 있는 도구가 이 분류에 있으면 탄색 바탕과 아래쪽 그라데이션 선으로 강조되고, 오른쪽에 도구 이름이 함께 나옵니다.
3	**도구 목록** — 분류에 마우스를 올리면 열립니다. 터치 화면은 분류를 눌러 열고, 키보드는 분류에 초점을 둔 채 ↓ 키로 엽니다. 도구 이름 아래에 한 줄 설명이 있습니다.
4	**전투 밸런스 분류** — 스탯·전투, 전투 시뮬레이터, 패턴 프레임 검증, 데이터 테이블.
5	**브리핑** — 모든 탭의 결론을 한 장으로 모으는 탭 (9장).
6	**다이어그램** — 문장으로 흐름도·상태 머신을 만드는 부가 도구 (10장).
7	**PDF** — 브라우저 안에서만 처리하는 PDF 편집 도구 (별도 페이지).
8	**소개** — 도구별 계산 방법·검증·사례를 정리한 소개 페이지.
9	**되돌리기 / 다시 하기** — 지금 탭의 변경을 한 단계씩 되돌립니다. 단축키 Ctrl+Z / Ctrl+Shift+Z. 탭마다 최대 50단계이고, 입력칸 안에서는 브라우저 기본 되돌리기가 먼저 동작합니다.
10	**스냅샷** — 모든 탭 설정을 한 번에 저장·비교·되돌리기 (1.6절).
11	**테마** — 라이트·다크 전환. 처음에는 다크로 열리고, 한 번 고르면 그 선택을 기억합니다.

1.5 화면 공통 구성 — 도구 머리와 공통 버튼

모든 도구는 같은 배치를 씁니다. 왼쪽 세로 칸이 **입력 · 설정**, 오른쪽 위가 **핵심 지표(KPI) 카드**, 그 아래로 표와 그래프, 맨 아래에 **검증 결과**와 **밸런스 도구**가 이어집니다. 카드 왼쪽 위의 작은 표시는 카드의 역할입니다: 입력(직접 고치는 곳), 결과(계산 결과), 검증(경고와 공식·시뮬레이션 대조), 도구(역산·최적화처럼 버튼을 눌러 돌리는 기능).



그림 2. 도구 머리의 공통 버튼 (강화 탭 예시 — 탭마다 있는 버튼이 조금씩 다름)

1	**기준안 저장** (강화·스탯·전투 탭) — 지금 결과를 "기준안"으로 저장해 두고, 이후 수치를 바꾸면 기준안 대비 변화를 KPI 카드에 함께 보여 줍니다. 비교를 끝내려면 **기준안 해제**.
2	**공유 링크** — 설정 전체가 담긴 주소를 클립보드에 복사합니다. 받은 사람이 열면 같은 설정이 됩니다.
3	**CSV 내보내기** — 탭에 따라 확률표·결과·레벨별 스탯 등 표를 CSV로 저장합니다. 엑셀에서 한글이 깨지면 [데이터 → 텍스트/CSV]에서 UTF-8로 열면 됩니다 (가챠 확률 공시표 CSV는 엑셀에서 바로 열리도록 저장).
4	**JSON 저장** — 설정을 파일로 저장합니다. 버전 관리나 메일 첨부에 씁니다.
5	**불러오기** — JSON 파일을 읽어 설정을 바꿉니다. 바꾸기 전 상태는 되돌리기(↶)로 돌아갈 수 있습니다.

> **팁 항목명 옆 (i)에 마우스를 올려 보세요**
>
> 표 머리글·카드 제목·입력칸 이름 옆의 작은 (i)에 마우스를 올리면 설명 말풍선이 뜹니다(1.8절). 예를 들어 강화 표의 "민감도" 머리글의 (i)에 마우스를 올리면 "이 단계 성공률을 1%p 올리면 총 기대 비용이 몇 % 바뀌는지"라는 설명이 나옵니다.

1.6 스냅샷과 프로젝트 파일

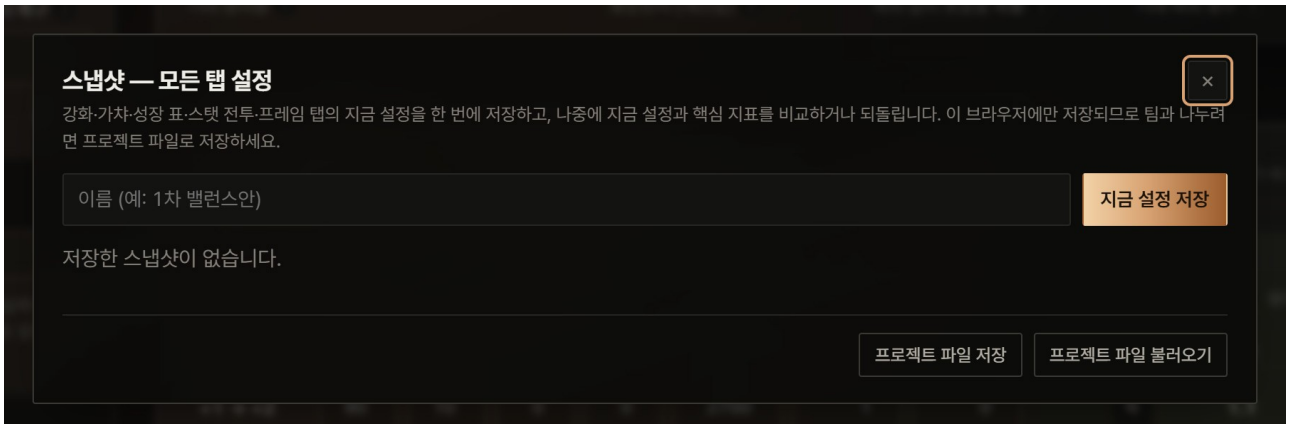


그림 3. 스냅샷 창 (헤더의 "스냅샷" 버튼)

스냅샷은 **모든 탭의 설정을 한 번에** 저장하는 기능입니다. 탭 하나만 저장하는 "기준안"과 달리, 강화·가챠·성장 표·스탯·전투·프레임 다섯 개 도구의 설정과 데이터 테이블(5장)을 묶어서 이름을 붙여 둡니다. 전투 시뮬레이터의 규칙·구성은 스냅샷에 들어가지 않으므로 탭 안의 프리셋(1.8절)으로 저장합니다.

1. 수치를 고치기 전에 헤더의 **스냅샷**을 누르고 이름(예: "1차 회의안")을 넣은 뒤 **지금 설정 저장**을 누릅니다.
2. 여러 탭의 수치를 고칩니다.
3. 스냅샷 창에서 저장해 둔 스냅샷의 **지금과 비교**를 누르면, 탭별 핵심 지표가 스냅샷과 지금 사이에 얼마나 바뀌었는지 표로 나옵니다.
4. 고친 것이 마음에 들지 않으면 **되돌리기**로 그 스냅샷의 설정 전체를 복원합니다.
5. **프로젝트 파일 저장**은 모든 탭 설정과 스냅샷을 파일 하나로 내보냅니다. 팀원이 **프로젝트 파일 불러오기**로 불러오면 같은 상태가 됩니다.

브리핑 탭(9장)은 스냅샷을 "기준"으로 골라 바뀐 지표를 한 장으로 보여 줍니다. 회의 전에 스냅샷을 저장해 두는 습관을 들이면 "지난번과 뭐가 달라졌나"를 바로 답할 수 있습니다.

1.7 공식 계산과 시뮬레이션 — 이 도구의 기본 원칙

워크벤치는 같은 질문을 두 가지 방법으로 계산하고 서로 맞춰 봅니다.

- **공식 계산** — 확률 모델을 수식(연립방정식, 적분, 동적 계획법)으로 풀어 오차 없는 값을 구합니다. 같은 설정이면 언제나 같은 값이 나오고, 0.005% 같은 아주 작은 확률도 빠르게 계산됩니다.
- **시뮬레이션(몬테카를로)** — 게임 규칙대로 가상의 유저를 수천~수만 번 실제로 돌려 봅니다. 공식으로 풀기 어려운 설정(데미지 변동, 특성, 교환이 섞인 가챠)도 계산할 수 있고, 평균뿐 아니라 "운 나쁜 1%"처럼 분포 전체를 보여 줍니다.

두 방법이 모두 가능한 곳에서는 **계산 검증** 카드에 나란히 표시하고, 시뮬레이션의 95% 범위 안에 공식 계산값이 들어오는지 확인합니다. 시뮬레이션은 **시드를 고정한 난수**를 써서 같은 설정이면 항상 같은 결과가 나옵니다(새로 고쳐도 숫자가 흔들리지 않음). 시드와 시행 횟수는 각 탭의 시뮬레이션 설정에서 바꿀 수 있습니다(1.8절).

> 알고리즘 **몬테카를로 시뮬레이션 — 이 문서 전체에서 쓰는 뜻**
>
> **무엇인가**: 확률이 섞인 과정을 난수로 여러 번 실제로 돌려 보고, 결과를 모아 평균·분포를 추정하는 방법입니다. 이름은 카지노로 유명한 모나코의 몬테카를로에서 왔습니다.
>
> **어디에 쓰였나**: 강화(2만 회), 가챠(목표당 3,000회 기본), 스탯·전투(데미지 변동·특성이 있는 1:1, 검증용 4,000회), 전투 시뮬레이터(설정당 300판).
>
> **얼마나 믿을 수 있나**: 평균의 오차는 표준오차 σ/√n 정도로, 횟수를 4배 늘려야 오차가 절반이 됩니다. 각 도구의 "시뮬레이션 수렴" 그래프가 횟수에 따라 평균과 95% 범위가 어떻게 좁아지는지 보여 줍니다. 자세한 내용은 부록 A.1.1.

1.8 공통 기능 — 툴팁·시드·입력 검증·결과 분포·프리셋 비교·기획서

강화·가챠·전투 시뮬레이터(6장)에는 같은 모양의 기능이 공통으로 들어 있습니다(툴팁·입력 검증은 모든 탭, 시드는 스탯·전투에도). 한 번 익히면 모든 탭에서 같은 방법으로 씁니다.

용어 툴팁 (i)

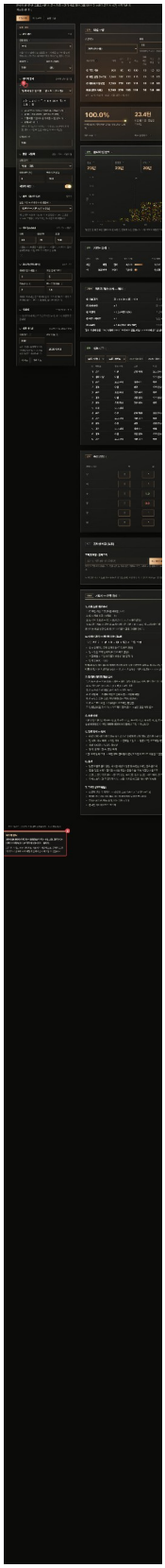


그림 4. 용어 툴팁 — 전투 시뮬레이터의 "공식" (i)에 마우스를 올린 상태

1	**(i)** — 항목명 옆의 작은 아이콘. 마우스를 올리거나 Tab 키로 초점을 주면 설명이 뜨고, Esc로 닫습니다.
2	**설명 상자** — 굵은 제목, 뜻, 쓰는 법(회색 글) 순서입니다.

설명은 용어 사전 하나(178개 항목)에서 관리해 모든 탭이 같은 문장을 씁니다. 표 머리글·카드 제목·입력칸 이름에 자동으로 붙으므로, 처음 보는 항목은 (i)부터 확인하세요.

시행 횟수와 랜덤 시드

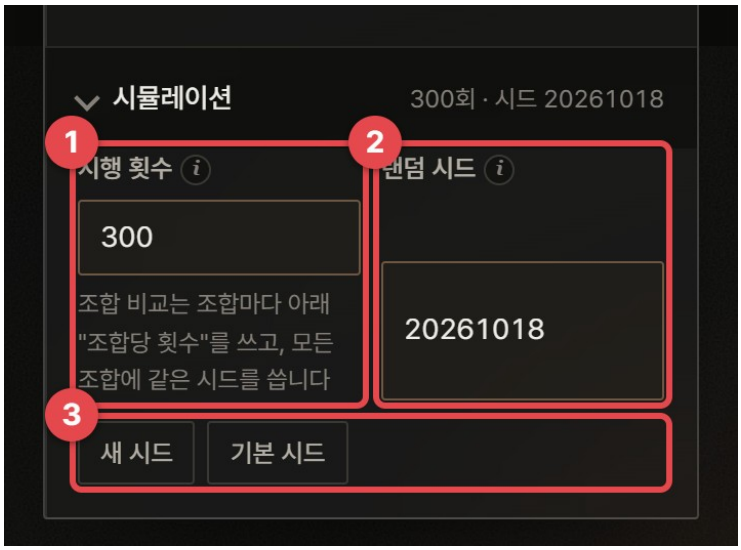


그림 5. 시뮬레이션 설정 (전투 시뮬레이터)

1	**시행 횟수** — 시뮬레이션을 몇 번 돌릴지. 강화 2만 회, 가챠 목표당 3,000회, 스탯·전투 4,000회, 전투 시뮬레이터 300판이 기본입니다.
2	**랜덤 시드** — 난수의 씨앗값. 같은 시드면 같은 결과가 나옵니다.
3	**새 시드 · 기본 시드** — 새 시드는 무작위 시드로 다시 돌리고, 기본 시드는 처음 값으로 되돌립니다.

시드를 바꿔도 평균은 거의 그대로이고, 분위·승률이 표본 오차만큼 움직입니다. 시드에 따라 결과가 크게 흔들리면 횟수가 모자란 것이니 시행 횟수를 늘리세요. 시드를 기본값으로 두면 이 기능이 생기기 전과 같은 결과가 나오므로, 예전 설정과 스냅샷의 숫자가 바뀌지 않습니다. 조합 비교·교체 비교·프리셋 비교는 모든 경우에 같은 시드를 써서 차이가 운이 아니라 설정 때문이 되게 합니다(공통 난수, 부록 A.1.4).

입력 검증



그림 6. 입력 검증 — 가챠 등급 표의 확률 칸에 120을 넣은 경우

1	**빨간 칸** — 범위를 벗어난 칸. 마우스를 올리면 "노멀 레벨 1 확률: 100 이하여야 합니다 (확률은 100% 이하)"처럼 이유가 나옵니다.

숫자 칸에 확률 0% 미만·100% 초과, 음수(0 이상이어야 하는 칸), 숫자가 아닌 값을 넣으면 그 칸이 바로 빨갛게 바뀝니다. 그림처럼 합계 칸도 135%로 빨갛게 표시됩니다. 확률 합계나 ID 참조처럼 여러 칸에 걸친 검사는 각 탭의 합계 칸·오류 목록과 데이터 테이블 검증(5.6절)이 맡습니다.

결과 분포 (P10 · P50 · P90 · P99)

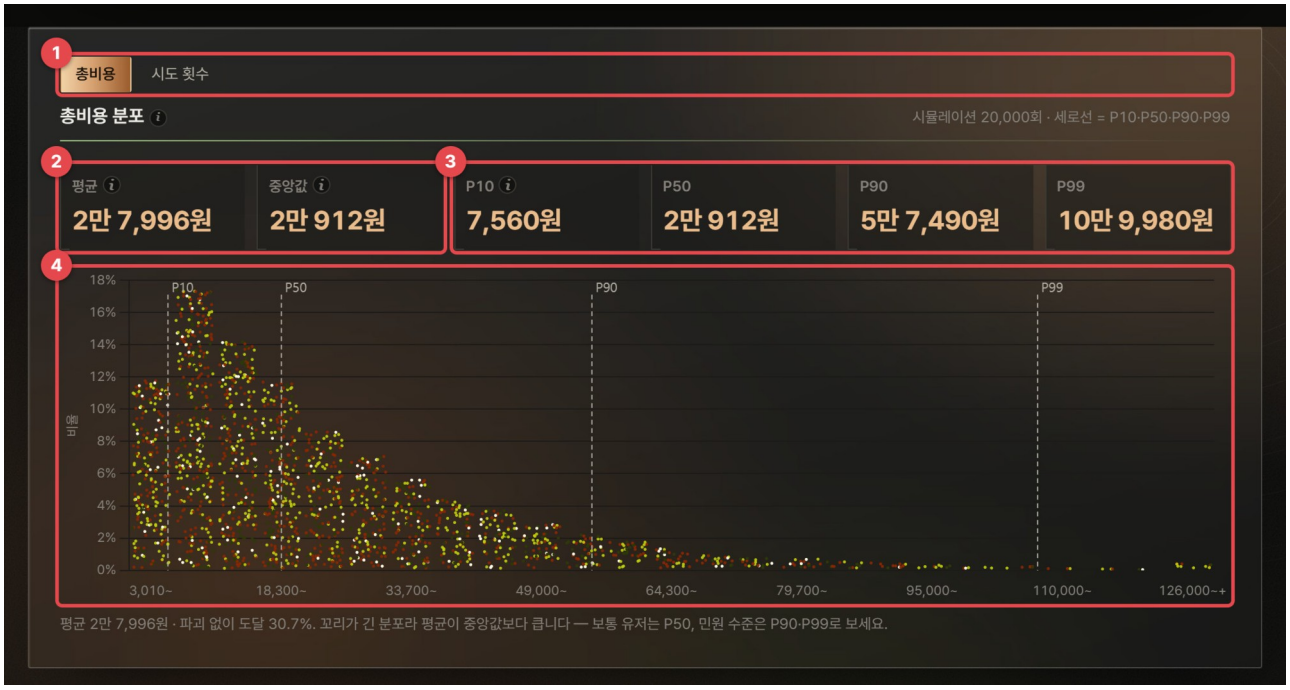


그림 7. 결과 분포 — 강화 기본 템플릿의 총비용 (시뮬레이션 2만 회)

1	**총비용 · 시도 횟수** — 그릴 값을 고릅니다(강화). 가챠는 선택한 목표의 뽑기 수, 전투 시뮬레이터는 클리어턴(초)을 그립니다.
2	**평균 · 중앙값** — 평균은 총량(매출·재화 소모), 중앙값은 보통 유저의 체감입니다.
3	**P10 · P50 · P90 · P99** — 운 좋은 10%의 경계, 가운데, 운 나쁜 10%와 1%의 경계입니다.
4	**히스토그램**(점묘) — 칸마다 뿌린 점의 양 = 그 구간에 든 비율(점 하나 ≈ 표본 몇 개), 점선 = 백분위 위치. 맨 위쪽 0.5%는 마지막 칸에 모읍니다.

강화 기본 템플릿은 평균 2만 7,996원(시뮬레이션), P10 7,560원, 중앙값 2만 912원, P90 5만 7,490원, P99 10만 9,980원입니다. P99가 중앙값의 5배가 넘는 긴 꼬리 분포라 평균 하나로는 "보통 유저"도 "운 나쁜 유저"도 설명하지 못합니다. 2.8절 달성 분위별 수치가 0.1%~99.9%를 자세히 보여 준다면, 이 카드는 네 개의 기준점과 분포 모양을 한눈에 보여 줍니다.

프리셋 저장과 A/B 비교

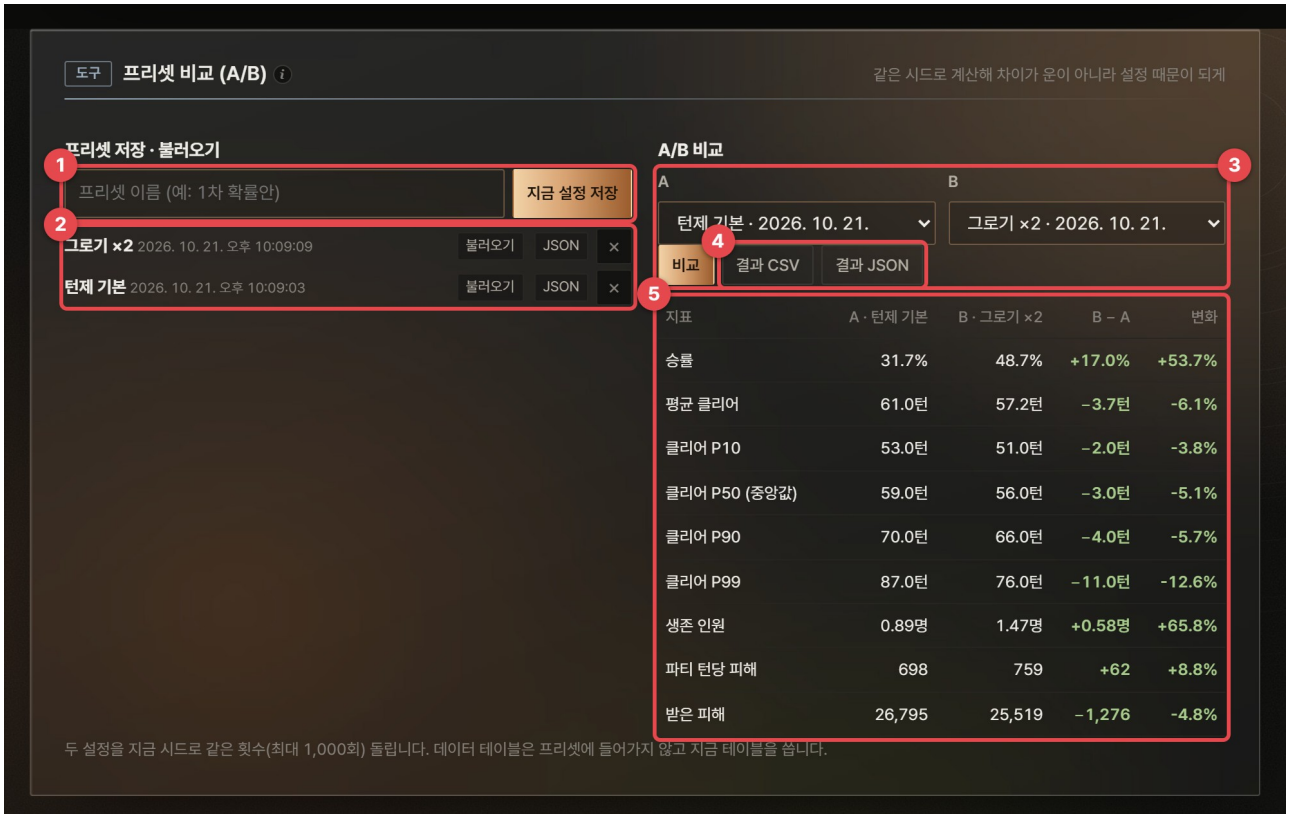


그림 8. 프리셋 비교 — 전투 시뮬레이터, 그로기 받는 피해 배율 1.5 → 2.0

1	**프리셋 이름 · 지금 설정 저장** — 지금 설정을 이름 붙여 이 브라우저에 저장합니다(탭마다 최대 30개).
2	**프리셋 목록** — 불러오기(되돌리기 Ctrl+Z로 취소 가능), JSON으로 내보내기, 삭제.
3	**A · B · 비교** — 두 프리셋(또는 "지금 설정")을 고르고 비교합니다. 두 설정을 같은 시드·같은 횟수로 다시 계산합니다.
4	**결과 CSV · 결과 JSON** — 비교 표를 파일로 저장합니다.
5	**비교 표** — 지표별 A, B, B − A, 변화율. 좋아진 쪽은 초록, 나빠진 쪽은 빨강입니다.

그림은 전투 시뮬레이터(6장)의 파티 전투에서 지금 설정을 "턴제 기본"으로 저장하고, 그로기 받는 피해 배율(흐트러진 보스가 받는 피해의 배율, 6.6절)을 2.0으로 바꿔 "그로기 ×2"로 저장한 뒤 비교한 결과입니다. 승률이 31.7%에서 48.7%로 오르고, 클리어 P50은 59턴에서 56턴, P99는 87턴에서 76턴으로 줄었습니다. 그로기 배율 하나가 긴 전투(P99)를 가장 많이 줄인다는 것이 숫자로 보입니다. 강화는 기대 총비용·P10~P99·표준편차·파괴 지표를, 가챠는 목표별 기대 뽑기·비용과 분위를 같은 방식으로

비교합니다.

> **팁 기준안 · 스냅샷 · 프리셋은 무엇이 다른가**
>
> **기준안**은 탭 하나의 "직전 상태" 하나를 KPI 옆에 놓고 비교합니다. **스냅샷**은 모든 탭의 설정을 한 번에 묶어 저장합니다(1.6절). **프리셋**은 탭 하나의 설정을 여러 벌 이름 붙여 두고, 그중 아무 둘을 같은 시드로 다시 계산해 표로 비교합니다. 전투 시뮬레이터의 프리셋에는 데이터 테이블이 들어가지 않으므로, 테이블을 바꾼 안끼리는 스냅샷으로 비교하세요.

기획서 영역

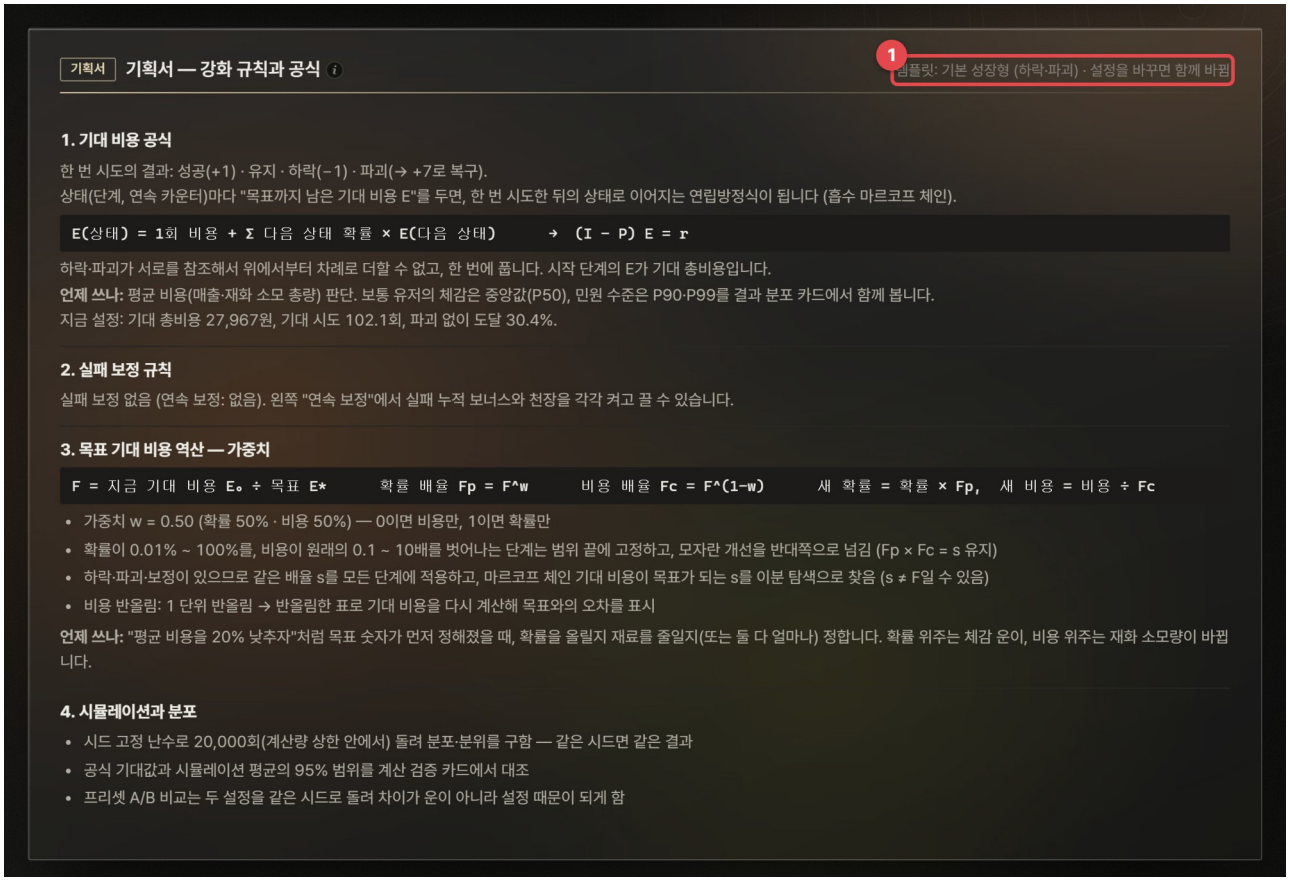


그림 9. 기획서 영역 — 강화 (목표 비용 역산 가중치 50%, 반올림 1 단위)

1	**지금 설정으로 만든 설명** — 템플릿과 설정을 바꾸면 문장·공식·숫자가 함께 바뀝니다.

기획서 영역은 탭 맨 아래에서 지금 설정의 공식·계산 순서·규칙을 기획서 문장으로 정리합니다. 강화는

기대 비용 공식, 실패 보정 규칙, 목표 비용 역산 가중치를, 가챠는 소환 레벨 ON/OFF에 맞춘 뽑기 구조와 천장·픽업·10연 보장, 계산 방법을, 전투 시뮬레이터는 최종 스탯·데미지 공식·계산 순서·상성·그로기·타겟팅·시너지·기여도를 설명합니다(6.12절). 화면에서 선택해 복사하면 기획서 초안이 되고, 숫자와 문장이 어긋날 걱정이 없습니다.

2. 강화 확률·비용 설계

하락·파괴·천장이 섞인 강화 시스템에서 목표 단계까지 드는 기대 총비용과 편차, 파괴 위험, 예산별 도달 확률을 계산하고, 보조 아이템(BM)의 적정 가격을 검토하는 도구입니다.

2.1 무엇을 하는 도구인가

단계별로 성공·대성공·유지·하락·파괴 확률과 재화 비용을 넣으면, "+0에서 +15까지 평균 얼마가 들고, 운 나쁜 1%는 얼마를 쓰는가"를 계산합니다. 하락했다가 다시 올라오는 경로, 파괴 후 복구 단계에서 다시 시작하는 경로, 실패가 쌓이면 오르는 보너스와 천장까지 모두 반영합니다.

이런 질문에 답합니다

- +15까지 평균 비용은? 중앙값·상위 10%·상위 1% 유저는 각각 얼마를 쓰나?
- 예산 3만 원으로 목표에 닿는 유저는 몇 %인가?
- 어느 단계가 총비용을 가장 많이 차지하고, 어느 단계의 성공률을 1%p 올리면 비용이 가장 많이 줄어드나?
- 파괴방지권 가격이 적정한가? 합리적인 유저는 어느 단계부터 어떤 아이템을 쓰나?
- 목표 기대 비용(예: 2만 원)에 맞추려면 성공률과 비용을 각각 얼마나 바꿔야 하나?

활용 예

- **BM 기획** — 보조 아이템을 판매하기 전에, 아이템을 최적으로 쓰는 유저의 비용 절감률과 기대 매출을 가격별로 비교해 가격을 정합니다.
- **시스템 기획** — 강화 확률표를 확정하기 전에 단계별 비용 비중과 민감도를 보고, 체감이 급격히 나빠지는 단계를 찾습니다.
- **확률 공시 검토** — 공시할 확률표로 실제 기대 비용이 얼마인지, 파괴 없이 도달할 확률이 몇 %인지 미리 확인합니다.
- **이벤트 기획** — 성공 확률 1.5배·비용 0.8배 같은 이벤트가 기대 비용을 얼마나 줄이는지 미리 봅니다.

2.2 화면 구성

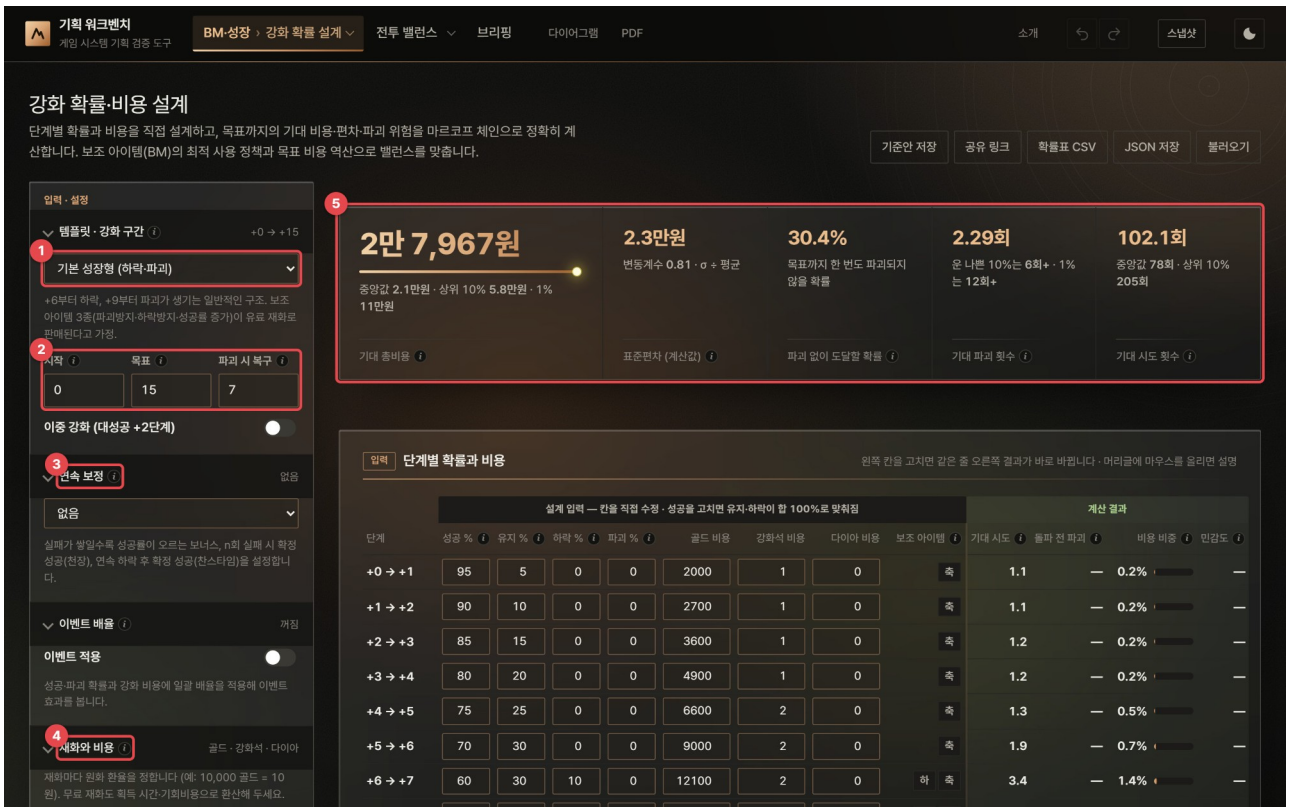


그림 10. 강화 확률·비용 설계 — 처음 화면

1	**템플릿** — 기본 성장형(하락·파괴), 실패 누적형(보너스·천장), 고위험 고보상(이중 강화·찬스타임), 빈 표 중에서 출발점을 고릅니다. 템플릿 아래 회색 글은 그 템플릿의 구조 설명입니다.
2	**시작 · 목표 · 파괴 시 복구** — 몇 강에서 몇 강까지 계산할지와, 파괴되면 몇 강으로 돌아가는지(복구 단계)를 정합니다. 바로 아래 스위치로 **이중 강화**(대성공 시 +2단계)를 켭니다.
3	**연속 보정** — 천장·실패 누적 보너스·찬스타임을 설정합니다 (2.4절).
4	**재화와 비용** — 골드·강화석·다이아처럼 강화에 드는 재화와 원화 환율을 정합니다. 그 아래에 **보조 아이템(BM)** 설정이 이어집니다.
5	**핵심 지표** — 기대 총비용(중앙값·상위 10%·1%), 표준편차, 파괴 없이 도달할 확률, 기대 파괴 횟수, 기대 시도 횟수. 값은 공식 계산이고, 중앙값·상위 n%는 시뮬레이션 결과입니다.

2.3 빠르게 시작하기

1. 왼쪽 **템플릿**에서 우리 게임과 가장 비슷한 구조를 고릅니다. 새로 설계한다면 "빈 표 (직접 입력)".
2. **시작·목표·파괴 시 복구** 단계를 맞춥니다. 목표를 바꾸면 단계 표의 행이 늘거나 줄어듭니다.

3. **재화와 비용**에서 재화 이름과 환율(예: 10,000 골드 = 10원)을 넣습니다. 무료 재화도 획득 시간·기회비용으로 환산해 두면 "체감 비용"을 원 단위로 비교할 수 있습니다.
4. 오른쪽 **단계별 확률과 비용** 표의 성공 %와 비용을 우리 수치로 바꿉니다. **성공 %를 고치면 파괴 %는 그대로 두고 유지·하락이 합 100%에 맞게 자동으로 바뀝니다.**
5. 위의 핵심 지표와 아래 분포·분위·검증 카드가 즉시 다시 계산됩니다. 비교용으로 고치기 전에 **기준안 저장**을 눌러 두세요.

2.4 설정 — 강화 구조

시작·목표·복구와 이중 강화

- **시작**: 강화를 시작하는 단계(보통 0). **목표**: 도달하면 끝나는 단계.
- **파괴 시 복구**: 파괴되면 이 단계부터 다시 시작합니다. 파괴 복구 비용은 재화별 "파괴 시 추가" 칸에서 정합니다.
- **이중 강화**를 켜면 표에 "대성공 %" 열이 생기고, 대성공 시 +2단계로 오릅니다.

연속 보정 (천장·보너스·찬스타임)

방식	카운터가 오르는 때	초기화	쓰는 곳
없음	—	—	단계별 확률만 사용
연속 실패 기준	유지·하락할 때마다 +1	성공·파괴 시	실패 누적 보너스(장인의 기운식), n회 실패 시 확정 성공(천장)
연속 하락 기준	하락할 때만 +1 (유지는 그대로)	성공·파괴 시	찬스타임: n연속 하락 후 다음 시도 확정 성공

- **실패 1회당 보너스**: 카운터 1당 성공률이 얼마나 오르는지 (상한까지). **보너스 기준**으로 "기본 성공률의 %"(예: 성공률 40%에 10% → +4%p)와 "%p (고정)"을 고릅니다.
- **천장 (회, 0=없음)**: 카운터가 이 값에 닿으면 다음 시도는 확정 성공입니다.
- **실패 누적 보너스 · 천장 스위치**: 연속 보정 방식을 고른 뒤 두 보정을 따로 켜고 끕니다. 꺼진 쪽은 0으로 계산하고(입력한 값은 지워지지 않음), 둘 다 끄면 "없음"과 같습니다. 목표 비용 역산과 프리셋 비교도 스위치 상태를 따릅니다.

이벤트 배율

이벤트 적용을 켜고 성공 배율·파괴 배율·비용 배율을 넣으면 모든 단계에 일괄 적용됩니다. 예: 성공 1.3배, 파괴 0.5배 이벤트. 성공 배율은 성공(+대성공) 확률에 곱하고, 늘어난 만큼 실패(유지·하락·파괴)에서 비율대로 뺍니다. 파괴 배율은 파괴 확률에 곱하고, 차이는 유지 확률과 주고받습니다. 적용 순서는 천장 → 이벤트 배율 → 연속 보너스 → 성공률 아이템 → 파괴방지·하락방지입니다.

재화와 비용

재화마다 "몇 개 = 몇 원" 환율을 정합니다. **결과 표시 단위**에서 기대 비용·분포·예산을 원으로 볼지 특정 재화로 볼지 고릅니다. 재화를 추가하면 단계 표에 그 재화의 비용 열이 생깁니다.

보조 아이템 (BM)

효과	동작
파괴방지 (파괴→유지)	사용한 시도에서 파괴가 나오면 유지로 바뀝니다.
하락방지 (하락→유지)	사용한 시도에서 하락이 나오면 유지로 바뀝니다.
성공률 증가 (%p)	성공률에 %p를 더하고, 그만큼 실패에서 비율대로 뺍니다. "1개 사용(켜기/끄기)" 또는 "개수 단위 투입"(개당 +n%p, 최대 개수)을 고릅니다.

아이템마다 1회(1개) 비용과 적용 단계 범위를 넣습니다. 실제로 어느 단계에서 쓸지는 단계 표의 **보조 아이템** 칸에서 켜거나 개수를 넣습니다. 칸의 글자는 아이템 이름의 첫 글자(예: 파·하·축)입니다.

2.5 단계별 확률과 비용 표



그림 11. 단계별 확률과 비용 — 왼쪽은 입력, 오른쪽(초록 머리)은 같은 줄의 계산 결과

1	**성공 %** — 고치면 파괴 %는 그대로 두고, 유지·하락이 입력을 시작할 때의 비율대로 나뉘어 합이 100%가 됩니다. 대성공 %도 같은 방식입니다. 유지·하락·파괴는 직접 고칠 수 있고, 합이 100%가 아니면 그 줄이 빨갛게 표시됩니다.
2	**파괴 %** — 성공을 고쳐도 바뀌지 않습니다. 성공 + 파괴가 100%를 넘으면 유지·하락이 0이 되고 합계 경고가 뜹니다.
3	**보조 아이템** — 아이템 칩을 눌러 이 단계의 모든 시도에 사용/해제합니다. 수량형은 숫자 칸에 1회 투입 개수를 넣습니다. 별표(*)는 최적화 결과처럼 연속 카운터마다 사용 여부가 다른 "상황별 사용"입니다.
4	**기대 시도** — 목표까지 가는 동안 이 단계에서 평균 몇 번 시도하는지. 하락 후 재도전과 파괴 후 복구 경로까지 포함합니다.
5	**돌파 전 파괴** — 이 단계에서 출발해 다음 단계에 닿기 전에 파괴될 확률. 하락해서 다시 올라오는 경로와 연속 보정까지 포함한 값입니다.

6	**민감도** — 이 단계 성공률을 1%p 올리면 총 기대 비용이 몇 % 바뀌는지. 절댓값이 큰 상위 3개 단계가 강조됩니다. 비용 비중 막대는 총 기대 비용 중 이 단계의 몫입니다.
7	**+ 단계 추가 / 마지막 단계 삭제** — 목표보다 높은 단계를 미리 설계할 때 씁니다. "경로 밖 단계도 표시"를 켜면 시작~목표 밖의 단계도 보입니다.

> **팁 민감도로 손볼 단계 고르기**
>
> 민감도가 −3.1%인 +14 단계는 성공률 1%p만 올려도 총비용이 3.1% 줄어듭니다. 체감 비용을 낮추고 싶을 때 민감도가 큰 단계부터 조정하면 적은 변화로 큰 효과를 얻습니다. 반대로 민감도가 거의 0인 초반 단계는 확률을 바꿔도 총비용에 거의 영향이 없습니다.

알고리즘 흡수 마르코프 체인 — 기대 비용·시도·파괴를 공식으로 계산

어디에 쓰였나: 핵심 지표의 기대 총비용, 표준편차, 파괴 없이 도달할 확률, 기대 파괴·시도 횟수, 그리고 단계 표의 기대 시도·돌파 전 파괴·비용 비중·민감도 전부.

모델: 상태 = (단계 i, 연속 카운터 c). 한 번 시도하면 성공(+1)·대성공(+2)·유지·하락(−1)·파괴(복구 단계로) 중 하나로 옮겨 갑니다. 목표 단계는 한번 들어가면 나오지 않는 "흡수 상태"입니다. 상태별 기대 비용 벡터 E는 다음 연립방정식을 만족합니다.

```
(I − P) E = r
  P : 상태 사이의 1회 시도 전이 확률 행렬 (흡수 상태 제외)
  r : 상태별 1회 시도 기대 비용 (파괴 시 복구 비용 포함)
```

빠르게 푸는 방법: 천장 수십 회 × 단계 수십 개도 즉시 풀리도록 행렬 구조(띠 행렬, 파괴 복구 열)를 활용합니다. 천장과 파괴 복구가 이 구조를 어떻게 만드는지는 구현 노트 I.1.

표준편차: 비용 제곱의 기대값(2차 모멘트) S2도 같은 행렬로 풉니다. $S2_s = \Sigma_t P_{st} [w_{st}^2 + 2 w_{st} E_t] + \Sigma_t P_{st} S2_t$, 분산 = $S2 - E^2$. 시뮬레이션 없이 정확한 표준편차가 나옵니다.

민감도(수반 방법): 시작 상태에서 각 상태를 방문하는 기대 횟수 $N = (I - P)^{-T} e_{시작}$ 을 한 번 구하면, 모든 단계의 성공률을 조금씩 바꿨을 때의 비용 변화를 한꺼번에 계산할 수 있습니다. 단계마다 다시 풀 필요가 없습니다.

부록 A.2.1(마르코프 체인), 구현 노트 I.1(LU·Sherman–Morrison), A.2.2(수반 민감도) 참고.

2.6 총비용 분포와 예산



그림 12. 총비용 분포와 예산 — 시뮬레이션 2만 회

1	**총비용 분포** — 가로축은 총비용, 세로축은 그 비용 구간에 든 유저 비율. 점선은 50%·90%·99% 지점입니다. 상위 1%를 넘는 꼬리는 그래프에서 생략합니다.
2	**예산** — 금액을 넣으면 그 예산 안에 목표에 닿는 유저 비율을 보여 줍니다. 처음에는 기대 총비용이 들어 있습니다. 평균이 중앙값보다 크기 때문에 평균 예산으로 도달하는 비율은 50%보다 조금 높습니다 (기본 템플릿 63.2%).
3	**50% / 90% / 99% 도달 예산** — 유저의 절반·90%·99%가 목표에 닿는 데 필요한 예산.

평균(기대 비용)은 운 나쁜 소수가 끌어올리기 때문에 보통 중앙값보다 큽니다. 기본 템플릿에서 평균은 2만 7,967원이지만 중앙값은 약 2.1만 원이고, 상위 1%는 11만 원을 씁니다. 과금 체감을 이야기할 때는 평균보다 **중앙값과 90%·99% 도달 예산**을 함께 보세요.

그 아래 결과 분포 카드(1.8절)는 총비용과 시도 횟수를 평균·중앙값·P10·P50·P90·P99와 히스토그램으로 보여 줍니다. 기본 템플릿은 P10 7,560원, P50 2만 912원, P90 5만 7,490원, P99 10만 9,980원입니다.

2.7 강화 경로 예시



그림 13. 강화 경로 예시 — 시뮬레이션 앞 12번의 단계 변화

가로축은 시도 횟수, 세로축은 현재 단계입니다. 회색 선은 각 시행, **탄색 선**(끝의 노란 점)은 시도 횟수가 중간인 시행, **청회색 선**은 가장 오래 걸린 시행, **빨간 ×**는 그 두 시행에서 파괴가 일어난 단계입니다. 표의 숫자만으로는 와닿지 않는 "하락·파괴로 몇 번씩 되돌아가는 체감"을 보여 줄 때 씁니다.

2.8 달성 분위별 수치

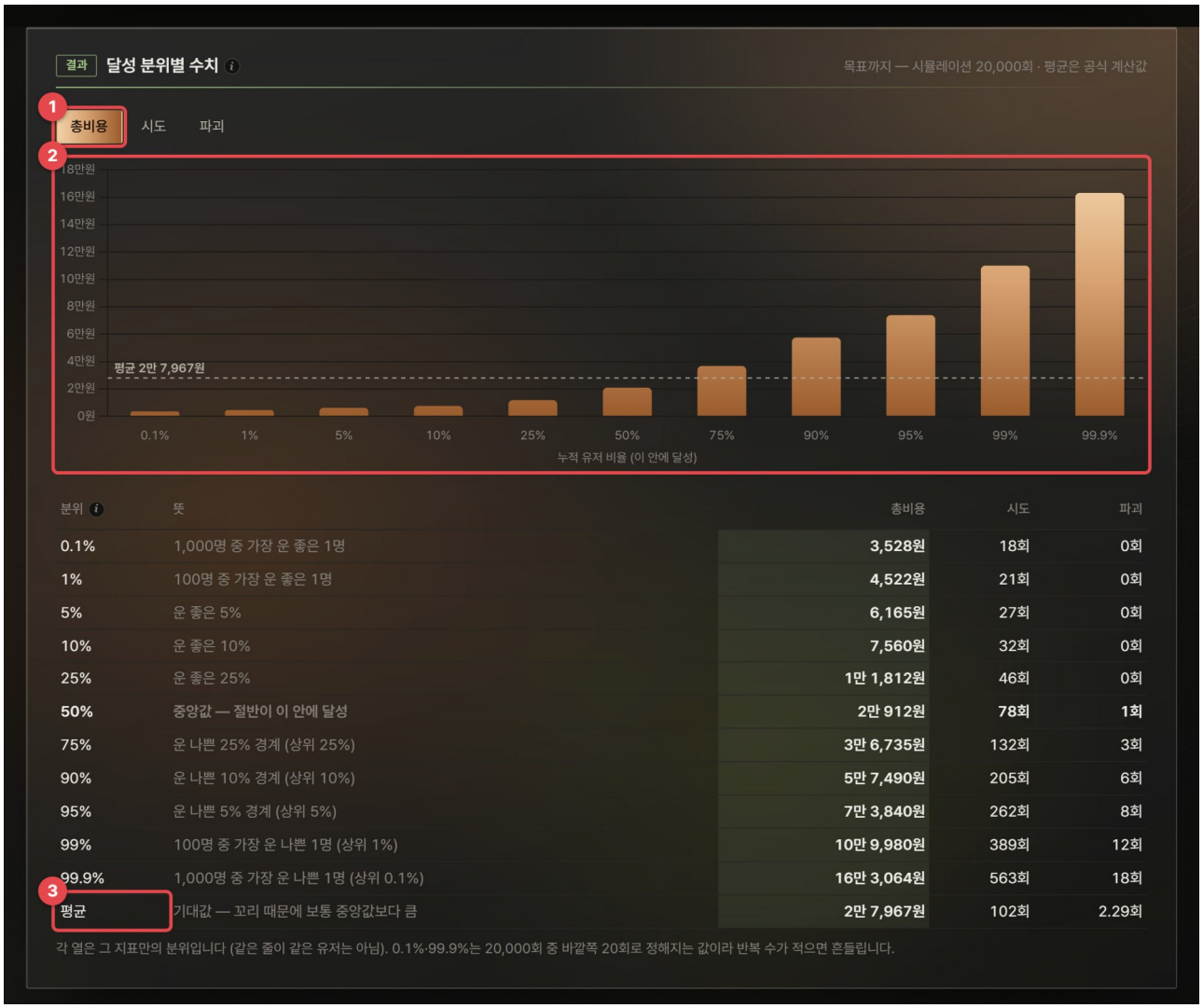


그림 14. 달성 분위별 수치 — 0.1%~99.9% 분위의 총비용·시도·파괴

1	**지표 선택** — 총비용 / 시도 / 파괴 중 그래프에 그릴 지표를 고릅니다. 표에는 세 지표가 모두 나옵니다.
2	**분위 막대** — 가로축은 누적 유저 비율(이 안에 달성), 점선은 평균입니다.
3	**평균** — 맨 아래 줄. 공식 계산값입니다. 분위 값은 시뮬레이션 결과입니다.

"분위 90%"는 "유저 100명을 비용 순으로 줄 세웠을 때 90번째 사람의 값", 즉 운 나쁜 10%가 시작되는 경계입니다. 각 열은 그 지표만의 분위라서, 같은 줄의 총비용과 파괴 횟수가 같은 유저의 값은 아닙니다. 0.1%·99.9%는 2만 회 중 바깥쪽 20회로 정해지는 값이라 설정을 조금만 바꿔도 흔들릴 수 있습니다.

> 알고리즘 **분위수(경험적 분위) — 운 좋은/나쁜 유저의 값**
>
> **어디에 쓰였나**: 중앙값, 상위 10%·1%, 50/90/99% 도달 예산, 달성 분위표.
>
> **방법**: 시뮬레이션 결과 n개를 작은 순으로 정렬하고, p 분위는 ⌈p·n⌉번째 값(최근접 순위 방식)을 씁니다. 예: n = 20,000에서 99% 분위는 19,800번째 값. 보간을 하지 않으므로 표의 값은 실제로 어떤 시행에서 나온 값입니다. 부록 A.1.2.

2.9 재화별 기대 소모와 계산 검증

재화별 기대 소모 카드는 목표까지 재화마다 평균 얼마를 쓰는지(파괴 복구·보조 아이템 포함)와 원환산, 비중을 보여 줍니다. 그 아래 **보조 아이템 기대 지출**은 목표 도달 1회당 BM 기대 매출, **파괴 복구 기대 비용**은 파괴 때문에 추가로 드는 비용입니다.

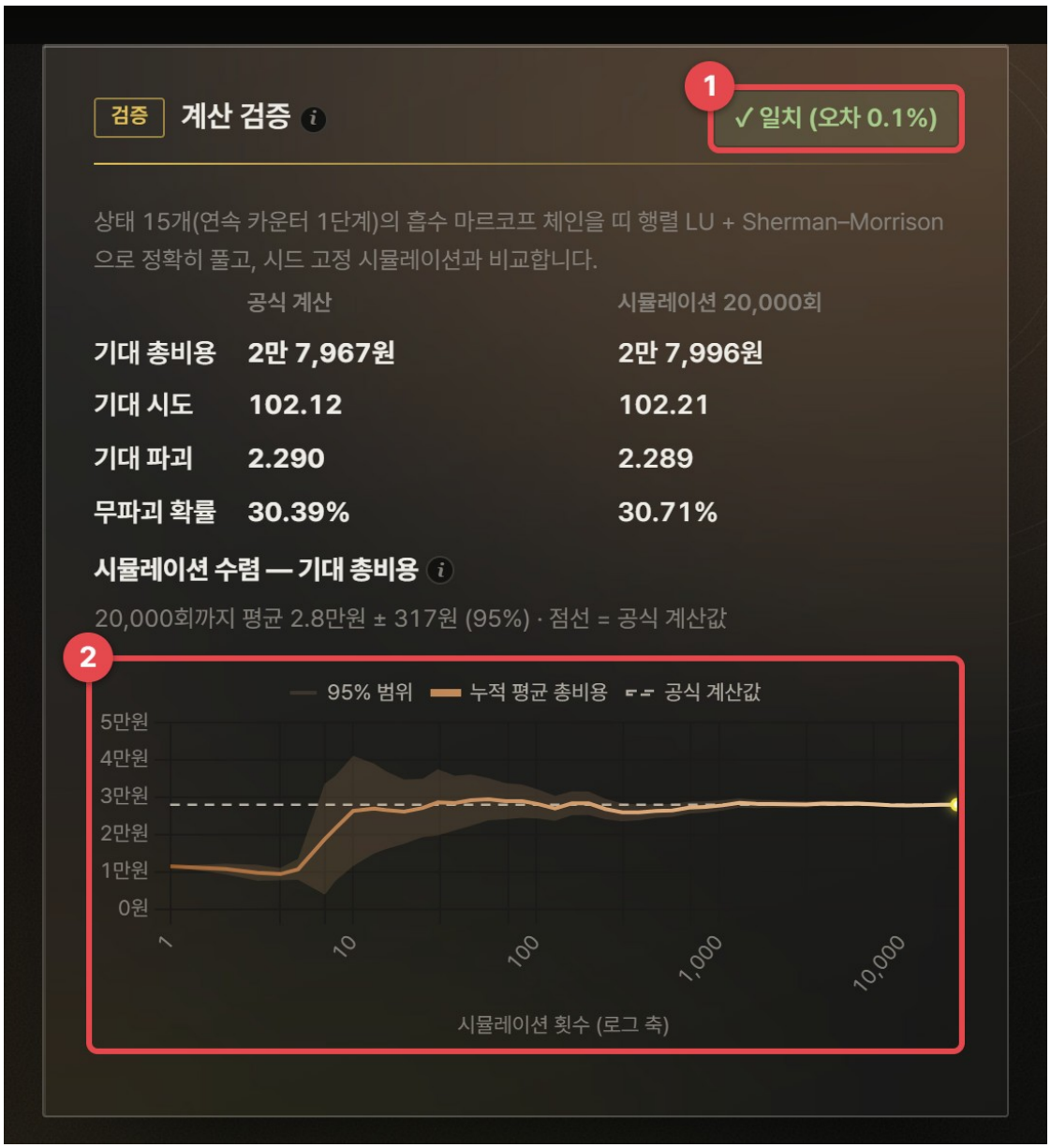


그림 15. 계산 검증 — 공식 계산과 시뮬레이션 대조, 수렴 그래프

1	**일치 표시** — 공식 계산과 시뮬레이션 2만 회의 기대 총비용 차이가 몇 %인지. 시뮬레이션의 95% 범위 안이면 일치로 봅니다.
2	**시뮬레이션 수렴** — 시뮬레이션 횟수(로그 축)에 따라 누적 평균과 95% 범위(띠)가 어떻게 좁아지는지, 점선(공식 계산값)에 다가가는지 보여 줍니다.

공식 계산과 시뮬레이션은 서로 독립적으로 구현되어 있습니다(체인은 행렬로, 시뮬레이션은 게임 규칙을 그대로 따라 한 번씩 굴림). 둘이 맞는다는 것은 설정 해석이 맞다는 근거가 됩니다. 만약 크게 어긋난다면 설정에 모델이 다루지 않는 규칙이 있는지 확인하세요.

알고리즘 **시드 고정 난수와 수렴 범위**

난수: 난수에 고정된 시드(씨앗값)를 넣어, 같은 설정이면 항상 같은 2만 개의 결과가 나옵니다. 그래서 설정을 조금 바꿨을 때의 차이가 "운"이 아니라 설정 때문임을 비교할 수 있습니다.

95% 범위: n회까지의 누적 평균 m과 표본 표준편차 s로 m ± 1.96·s/√n. 중심극한정리에 따라 평균의 분포가 정규분포에 가까워지는 성질을 씁니다. 부록 A.1.1.

2.10 밸런스 도구

목표 비용 역산

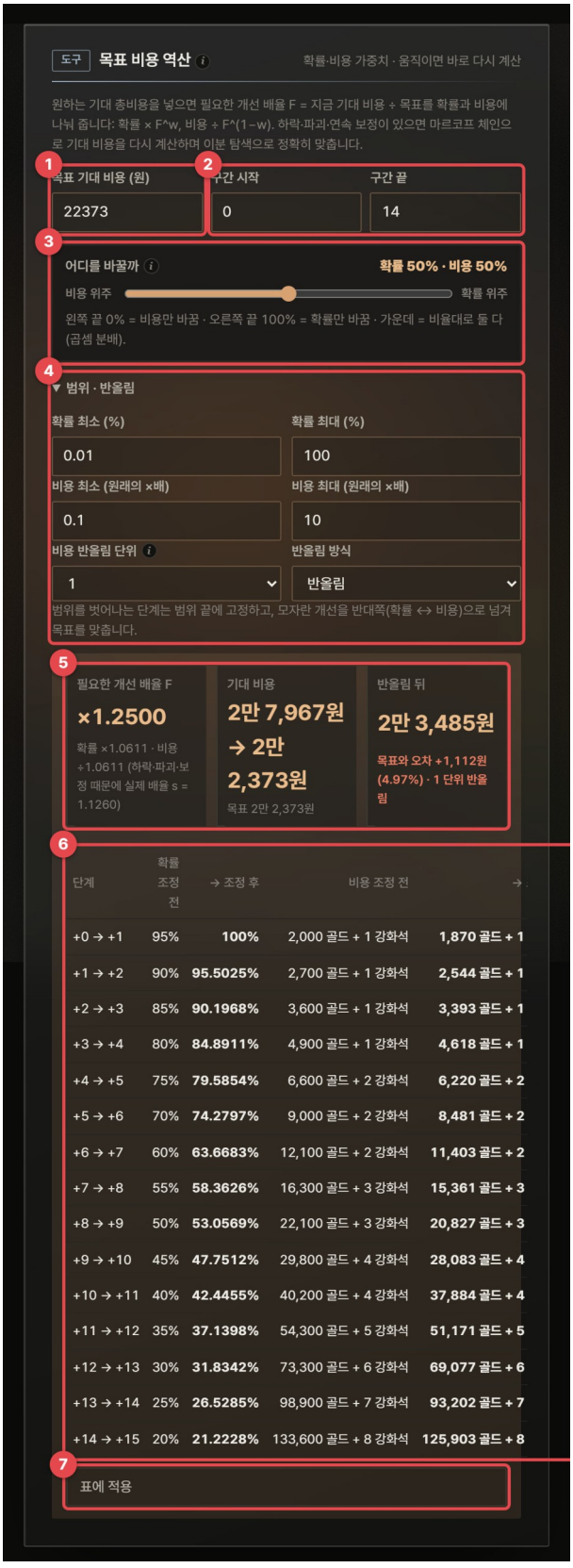


그림 16. 목표 비용 역산 — 확률·비용 가중치 50%, 비용 반올림 1 단위

1	**목표 기대 비용** — 원하는 기대 총비용(결과 표시 단위). 처음에는 지금 기대 비용의 80%가 들어 있습니다.
2	**구간 시작 · 구간 끝** — 확률·비용을 조정할 단계 범위. 범위 밖 단계는 그대로 둡니다.
3	**어디를 바꿀까 (가중치)** — 왼쪽 끝 0% = 비용만, 오른쪽 끝 100% = 확률만, 그 사이 = 비율대로 둘 다. 움직이면 바로 다시 계산합니다.
4	**범위 · 반올림** — 확률 최소·최대(%), 비용 최소·최대(원래 비용의 몇 배), 비용 반올림 단위(안 함·1·10·100·1,000·10,000)와 방식(반올림·올림·내림).
5	**결과 요약** — 필요한 개선 배율 F와 확률·비용 배율, 기대 비용 전 → 후, 반올림한 표의 기대 비용과 목표와의 오차(1%를 넘으면 빨강).
6	**단계별 조정 표** — 단계마다 확률·비용 조정 전 → 후(반올림 적용). 범위 끝에 걸린 단계에는 "확률 범위 끝 → 비용으로 넘김"처럼 표시합니다.
7	**표에 적용** — 조정된 확률·비용을 단계 표에 넣습니다. 적용 전 상태는 기준안으로 저장되어 KPI에서 비교됩니다.

기본 템플릿(지금 2만 7,967원)에서 목표를 2만 2,373원(80%)으로 두면 필요한 개선 배율은 F = 27,967 ÷ 22,373 = 1.25입니다. 가중치 50%면 확률과 비용에 같은 몫씩 나눠, 구간의 모든 단계에 성공 확률 ×1.0611, 비용 ÷1.0611을 적용해 목표에 정확히 닿습니다. 확률 배율과 비용 배율을 곱한 실제 개선 배율 s = 1.0611 × 1.0611 = 1.126이 F(1.25)보다 작은데도 목표에 닿는 이유는 하락·파괴 때문입니다. 성공률을 올리면 하락·파괴로 되돌아가는 경로도 함께 줄어 기대 비용이 배율보다 더 많이 내려갑니다. +0 → +1은 95% × 1.0611이 100%를 넘으므로 100%에 고정하고, 모자란 몫을 비용으로 넘겨 2,000골드를 1,870골드로 더 내립니다.

가중치를 바꾸면 같은 목표를 다른 방법으로 만듭니다. 0%(비용만)는 단계 비용 ÷1.2526(파괴 복구 비용은 그대로라 F보다 조금 더 내림), 100%(확률만)는 성공 확률 ×1.0848입니다. 확률 위주는 유저가 느끼는 운이, 비용 위주는 재화 소모량이 바뀝니다.

반올림 단위를 1, 방식을 반올림으로 두면 표의 기대 비용이 2만 3,485원으로 목표보다 1,112원(4.97%) 비싸집니다. 강화석처럼 개수가 작은 재화가 0.9개 → 1개로 올라가기 때문입니다. 반올림 단위는 모든 재화에 똑같이 적용되므로, 단위가 작은 재화가 섞여 있으면 오차가 커집니다(같은 설정에서 내림이면 1만 9,205원, −14.2%). 오차가 1%를 넘으면 빨갛게 표시되니 반올림 방식·단위나 범위를 바꿔 다시 봅니다.

알고리즘 목표 비용 역산 — 곱셈 분배와 범위 넘김

어디에 쓰였나: 목표 비용 역산(가중치 슬라이더, 범위, 반올림), 기획서 영역의 역산 설명.

```
F = 지금 기대 비용 ÷ 목표
확률 배율 Fp = s^w,  비용 배율 Fc = s^(1−w)        (Fp × Fc = s)
새 확률 = 확률 × Fp   (범위 밖이면 끝에 고정, 모자란 몫은 Fc로)
새 비용 = 비용 ÷ Fc   (범위 밖이면 끝에 고정, 모자란 몫은 Fp로)
```

곱셈 분배: 성공만 있는 단계에서 기대 비용 = 비용 ÷ 확률이므로, "25% 싸게"는 확률 ×1.25 또는 비용 ÷1.25이고 둘을 섞으면 곱이 1.25가 되어야 합니다. 그래서 더하기가 아니라 지수 w로 나눕니다(하락·파괴가 없다면 w = 0.5에서 둘 다 √1.25 = 1.118배).

s 찾기: 하락·파괴·연속 보정이 있으면 s = F가 아닙니다. 구간의 모든 단계에 같은 s를 적용한 표의 기대 비용을 마르코프 체인(2.5절)으로 다시 계산하고, s를 로그 이분 탐색으로 좁혀(최대 80회) 목표에 맞춥니다. 범위 끝에 걸린 단계가 있어도 단계마다 Fp × Fc = s가 유지되므로 같은 방법으로 풉니다. 부록 A.2.4.

보조 아이템 최적 사용 정책

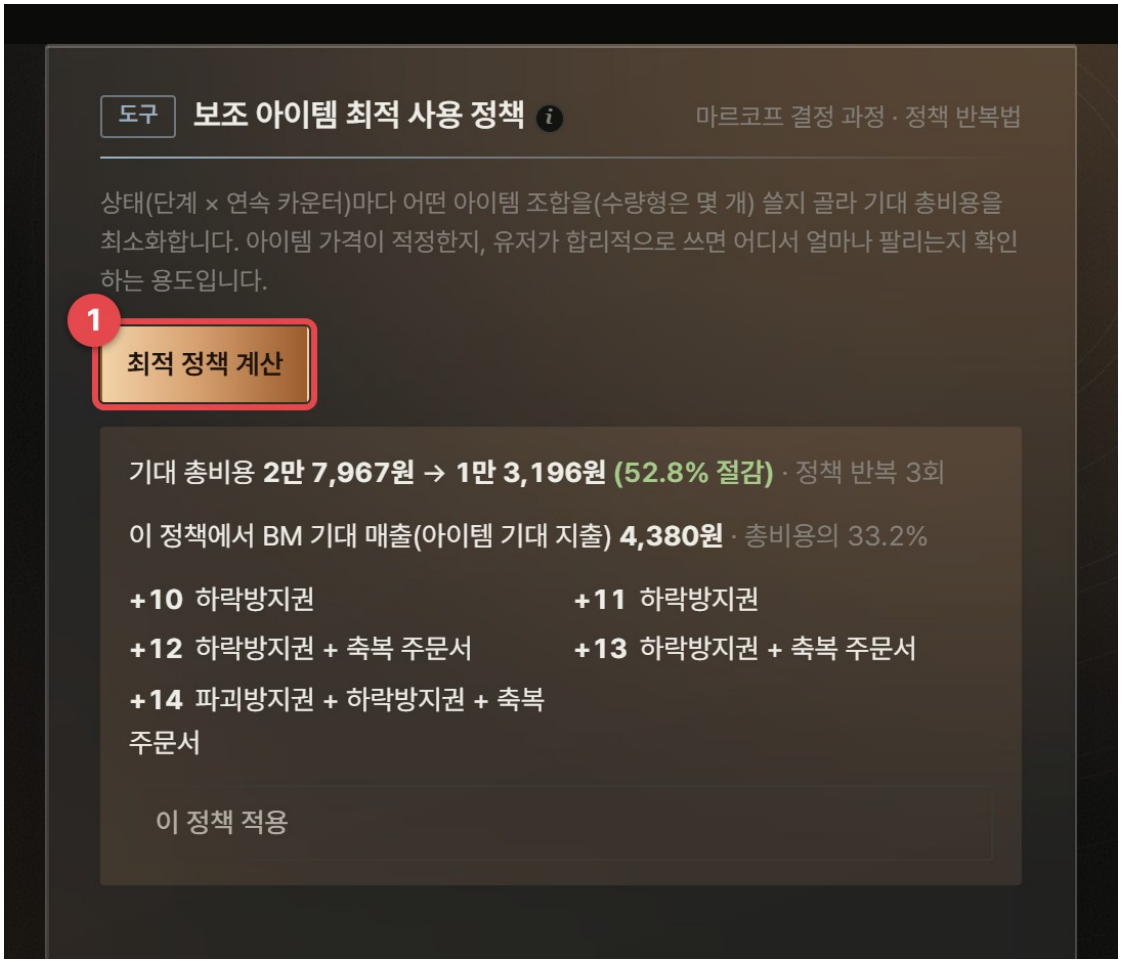


그림 17. 보조 아이템 최적 사용 정책 — "최적 정책 계산" 결과

1	**최적 정책 계산** — 상태(단계 × 연속 카운터)마다 어떤 아이템 조합을(수량형은 몇 개) 쓰면 목표까지 기대 총비용이 가장 낮은지 계산합니다. 결과로 절감액, BM 기대 매출, 단계별 추천 조합이 나오고, **이 정책 적용**을 누르면 단계 표의 아이템 칸에 반영됩니다.

기본 템플릿에서는 아이템 없이 2만 7,967원이던 비용이 최적 정책으로 1만 3,196원(52.8% 절감)이 됩니다. 합리적인 유저가 쓰면 비용이 절반이 되는 아이템은 "선택"이 아니라 "필수품"이라는 뜻이고, 가격이 너무 싸다는 신호입니다. 절감 폭이 의도한 수준(예: 15% 안쪽)이 되도록 아래 가격 곡선으로 가격을 찾습니다.

알고리즘 **마르코프 결정 과정과 정책 반복법**

어디에 쓰였나: 보조 아이템 최적 사용 정책, 아이템 가격 곡선(가격마다 다시 풂).

모델: 마르코프 체인에 "선택"을 더한 것이 마르코프 결정 과정(MDP)입니다. 상태마다 고를 수 있는 행동 = 아이템 투입 조합, 행동에 따라 전이 확률(파괴→유지 등)과 1회 비용(아이템 값 포함)이 바뀝니다. 목표는 시작 상태에서 목표까지의 기대 총비용을 최소화하는 정책(상태 → 행동)입니다.

정책 반복법: ① 지금 정책으로 고정하고 (I − P)E = r을 풀어 상태별 기대 비용을 구합니다(정책 평가). ② 상태마다 "이번 한 번만 다른 행동을 하고 이후는 지금 정책대로" 했을 때의 기대 비용(Q값)이 가장 작은 행동으로 바꿉니다(정책 개선). ③ 바뀌는 행동이 없을 때까지 반복합니다. 기본 템플릿은 3회 만에 끝납니다.

계산량 줄이기: 효과가 같은 아이템 조합을 하나로 합쳐 계산량을 줄입니다(구현 노트 I.2). 부록 A.2.3.

아이템 가격 곡선

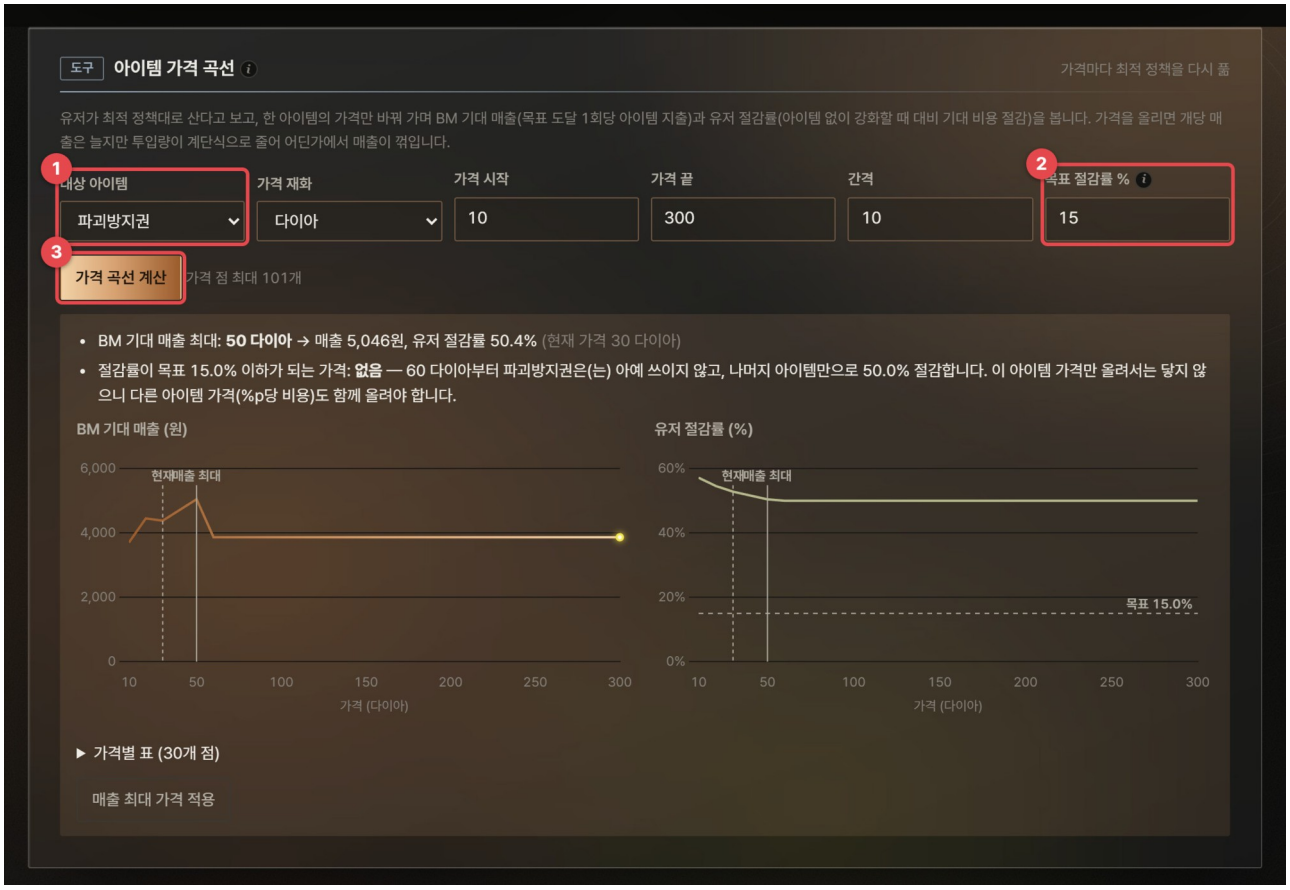


그림 18. 아이템 가격 곡선 — 가격별 BM 기대 매출과 유저 절감률

1	**대상 아이템 · 가격 재화 · 가격 범위 · 간격** — 가격을 바꿔 볼 아이템과 재화, 시작·끝·간격(최대 101개 가격 점).
2	**목표 절감률 %** — "아이템을 최적으로 써도 비용이 이만큼만 줄어야 한다"는 기준. 오른쪽 그래프에 가로 점선으로 그려집니다.
3	**가격 곡선 계산** — 가격마다 최적 정책을 다시 풀어 BM 기대 매출(왼쪽)과 유저 절감률(오른쪽)을 그립니다. 결과 문장에 매출이 가장 큰 가격과, 절감률이 목표 이하가 되는 가격이 나오고 버튼 하나로 적용합니다.

가격을 올리면 개당 매출은 늘지만, 어느 가격부터 합리적인 유저가 그 아이템을 쓰지 않는 단계가 생겨 투입량이 계단식으로 줄어듭니다. 그래서 매출 곡선은 어딘가에서 꺾입니다. 그림의 예처럼 "60 다이아부터 파괴방지권을 아예 쓰지 않고, 나머지 아이템만으로 50% 절감"이라면, 이 아이템 가격만 올려서는 목표 절감률에 닿지 않으므로 다른 아이템 가격도 함께 올려야 한다는 결론이 나옵니다.

> **주의** **최적 정책은 "합리적인 유저"의 상한입니다**
>
> 실제 유저는 최적 정책보다 아이템을 덜 쓰거나 더 많이 씁니다. 최적 정책의 절감률은 "아이템을 가장 영리하게 썼을 때 얼마나 이득인가"의 상한이고, BM 기대 매출은 그 유저가 쓰는 금액입니다. 실제 매출 예측보다는 가격이 지나치게 싸거나 비싼지 판단하는 기준으로 쓰세요.

2.11 다른 탭과 연결

- **확률표 CSV** — 단계별 확률·비용·결과를 표로 내보냅니다.
- **브리핑** — 기대 총비용·기대 시도·기대 파괴·파괴 없이 도달 확률이 브리핑 탭의 지표가 됩니다.
- 프리셋 비교 · 기획서 — 탭 아래쪽의 프리셋 비교(A/B)와 기획서 영역(1.8절)은 실패 보정 스위치와 역산 가중치까지 반영합니다.

3. 가챠·수집 설계

등급 확률, 소환 레벨, 천장, 픽업, 마일리지, 중복 분해 교환을 넣고 "이 유닛을 이만큼 모으려면 실제로 얼마가 드나"를 계산하는 도구입니다. "필요 장수 ÷ 확률"로 하는 단순 계산이 중복과 확률 구조 때문에 얼마나 빗나가는지 원인별로 나눠 보여 주고, 확률 공시표를 만듭니다.

3.1 무엇을 하는 도구인가

같은 유닛을 여러 장 모아야 하는 초월·각성 구조에서는, 이미 다 모은 유닛이 또 나오는 **중복** 때문에 실제 뽑기 수가 단순 계산보다 훨씬 커집니다. 또 소환 레벨처럼 처음에는 상위 등급 확률이 낮은 구간이 있으면 그만큼 더 듭니다. 이 도구는 이런 효과를 모두 넣어 목표별 기대 뽑기·비용·분포·도달 기간을 계산합니다.

이런 질문에 답합니다

- 레전드 4종을 6장씩(풀 초월) 모으려면 실제로 몇 번 뽑나? 단순 계산의 몇 배인가?
- 천장·마일리지·조각 교환 중 무엇이 운 나쁜 1%의 꼬리를 가장 잘 자르나?
- 무과금 유저와 월 3만 원 과금 유저는 목표까지 며칠이 걸리나?
- 공시해야 할 캐릭터별 확률과 천장 확률 변동표는? 반올림하면 0%로 보이는 캐릭터는 없나?

활용 예

- **BM 기획** — 신규 가챠의 목표(픽업 1장, 풀 초월 등)별 기대 비용과 99% 비용을 보고 천장·마일리지 수치를 정합니다.
- **라이브 운영** — "레전드 확률 0.02%"를 그대로 두고 마일리지로 운 나쁜 유저의 상한을 보장하는 안을 비교합니다.
- **확률 공시** — 소환 레벨별·캐릭터별 확률과 확률이 바뀌는 조건 문장을 만들어 CSV로 내보냅니다.

3.2 화면 구성

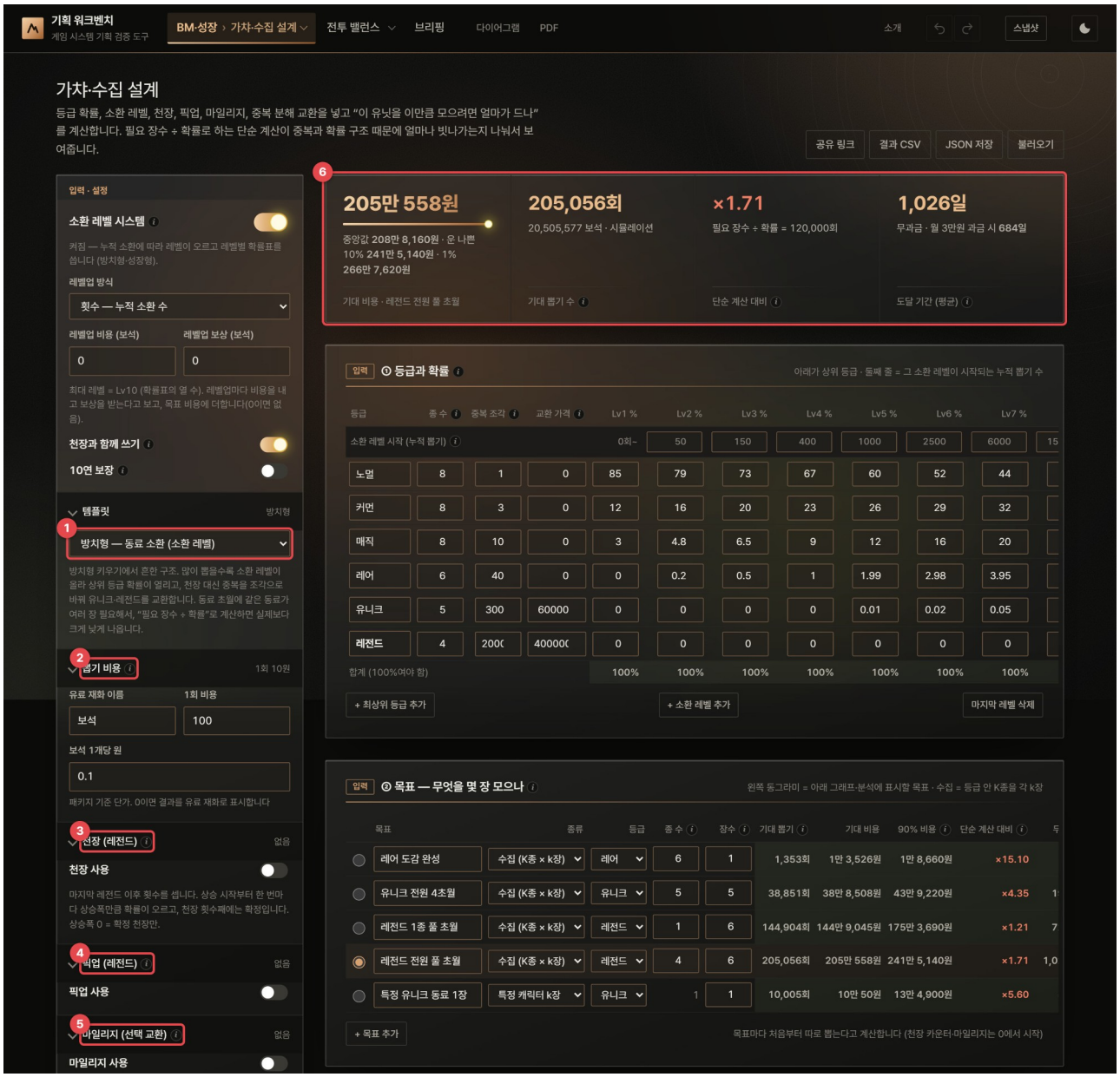


그림 19. 가챠·수집 설계 — 처음 화면

1	**템플릿** — 방치형(동료 소환·소환 레벨), 수집형 RPG(천장·픽업·마일리지), 빈 설계.
2	**뽑기 비용** — 유료 재화 이름, 1회 비용(재화 개수), 재화 1개당 원(패키지 기준 단가). 단가를 0으로 두면 결과를 재화 개수로 표시합니다.
3	**천장 (최상위 등급)** — 상승 시작, 상승폭 %p, 천장 횟수.
4	**픽업** — 최상위가 나왔을 때 픽업 유닛일 비율, "놓치면 다음은 픽업 확정" 보장.

5	**마일리지 (선택 교환)** — 뽑기당 적립량과 교환 가격. 모이면 최상위 중 원하는 1장(픽업 포함)과 바로 교환한다고 봅니다. 그 아래 **기준 유저**(무과금 하루 획득, 기준 과금 유저 월 과금)와 **계산**(목표당 시뮬레이션 횟수·랜덤 시드)가 이어집니다.
6	**핵심 지표** — 선택한 목표의 기대 비용(중앙값·운 나쁜 10%·1%), 기대 뽑기 수, 단순 계산 대비 배율, 평균 도달 기간(무과금 / 기준 과금 유저).

3.3 빠르게 시작하기

1. **템플릿**을 고르고, **뽑기 비용**에 1회 비용과 원 단가를 넣습니다.
2. ① **등급과 확률** 표에 등급별 종 수와 확률을 넣습니다. 소환 레벨이 없는 고정 확률 가챠라면 왼쪽 맨 위 소환 레벨 시스템 스위치를 끕니다(3.5절).
3. 천장·픽업·마일리지가 있다면 왼쪽 설정에서 켜고 수치를 넣습니다.
4. ② **목표** 표에 "무엇을 몇 장 모으나"를 넣습니다. 왼쪽 동그라미로 그래프·분석에 표시할 목표를 고릅니다.
5. 핵심 지표, 달성 확률과 예산, 분위표, "왜 이만큼 드나"에서 결과를 읽고, 공시가 필요하면 **확률 공시표**를 CSV로 받습니다.

3.4 ① 등급과 확률



그림 20. ① 등급과 확률 — 아래로 갈수록 상위 등급

1	**종 수** — 그 등급에 속한 유닛 종류 수. 같은 등급 안의 유닛은 같은 확률로 나온다고 봅니다.
2	**중복 조각** — 필요 없는 중복 1장을 분해하면 얻는 조각 수.
3	**교환 가격** — 그 등급의 원하는 유닛 1장과 바꾸는 데 필요한 조각 수. 비워 두면 교환 없음.

4	**Lv1 % … Lv10 %** — 소환 레벨별 등급 확률. 레벨 열은 "+ 소환 레벨 추가"로 늘립니다. 소환 레벨 시스템이 켜져 있을 때만 여러 열이 보이고 늘릴 수 있습니다(3.5절).
5	**소환 레벨 시작 (누적 뽑기)** — 그 레벨이 시작되는 누적 뽑기 수. 많이 뽑을수록 상위 등급 확률이 열리는 방치형 구조를 표현합니다. 경험치 방식이면 "레벨 시작 (필요 누적 경험치)"가 됩니다.
6	**합계** — 레벨마다 등급 확률 합이 100%여야 합니다. 아니면 빨갛게 표시됩니다.

3.5 소환 레벨 스위치와 10연 보장

가챠는 크게 두 갈래입니다. 서브컬처 가챠는 등급별 확률이 고정이고 천장·픽업으로 운을 보정하며, 방치형·성장형 가챠는 많이 뽑을수록 소환 레벨이 올라 상위 등급 확률이 열립니다. 왼쪽 맨 위의 **소환 레벨 시스템** 스위치가 이 둘을 나눕니다.

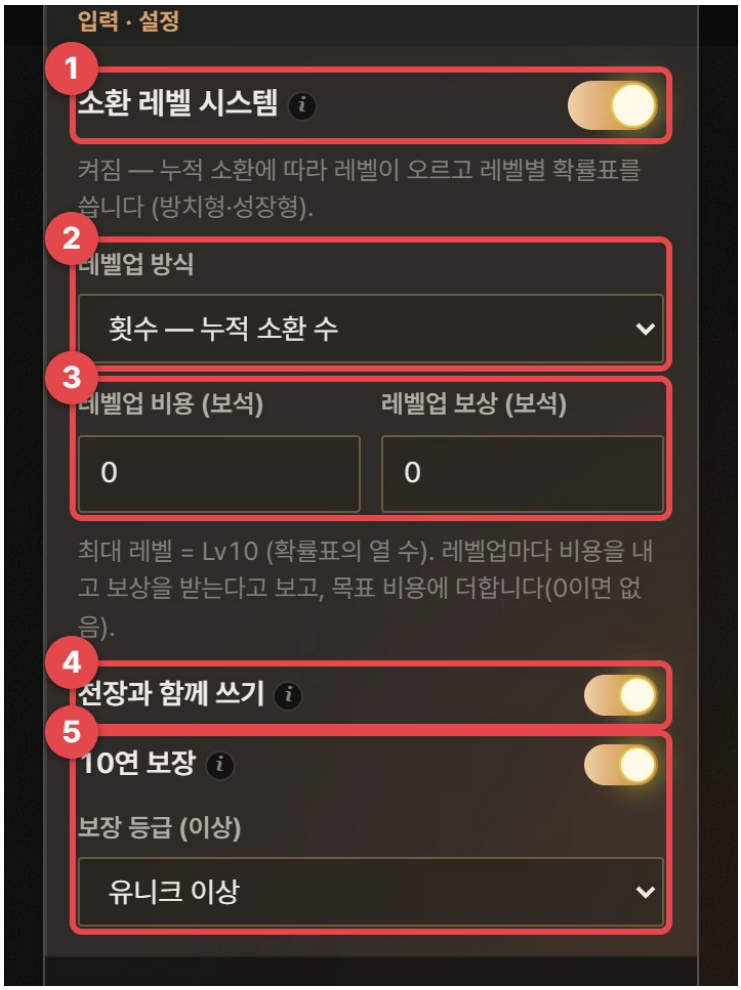


그림 21. 소환 레벨 스위치와 10연 보장 (방치형 템플릿, 10연 보장을 켠 상태)

1	**소환 레벨 시스템** — 켜짐 = 누적 소환에 따라 레벨별 확률표를 씀, 꺼짐 = 등급별 고정 확률 한 열만 씀. 끄고 켜도 각 모드에서 넣었던 확률표·레벨 기준은 따로 보관했다가 되살립니다.
2	**레벨업 방식** — 횟수(누적 소환 수가 기준에 닿으면) 또는 경험치(소환 1회당 경험치를 쌓아 "필요 누적 경험치"에 닿으면). 경험치 방식은 뽑기 수 = 필요 경험치 ÷ 소환당 경험치(올림)로 바꿔 계산합니다.
3	**레벨업 비용 · 보상** — 레벨이 오를 때마다 내는 비용과 받는 보상(유료 재화). 목표를 이루기 전에 지나간 레벨만큼 기대값으로 목표 비용에 더합니다.
4	**천장과 함께 쓰기** — 끄면 소환 레벨이 켜진 동안 천장을 쓰지 않습니다.
5	**10연 보장 · 보장 등급** — 10번째 뽑기까지 보장 등급 이상이 한 번도 없으면 10번째는 그 등급 이상. 소환 레벨과 상관없이 켤 수 있습니다.

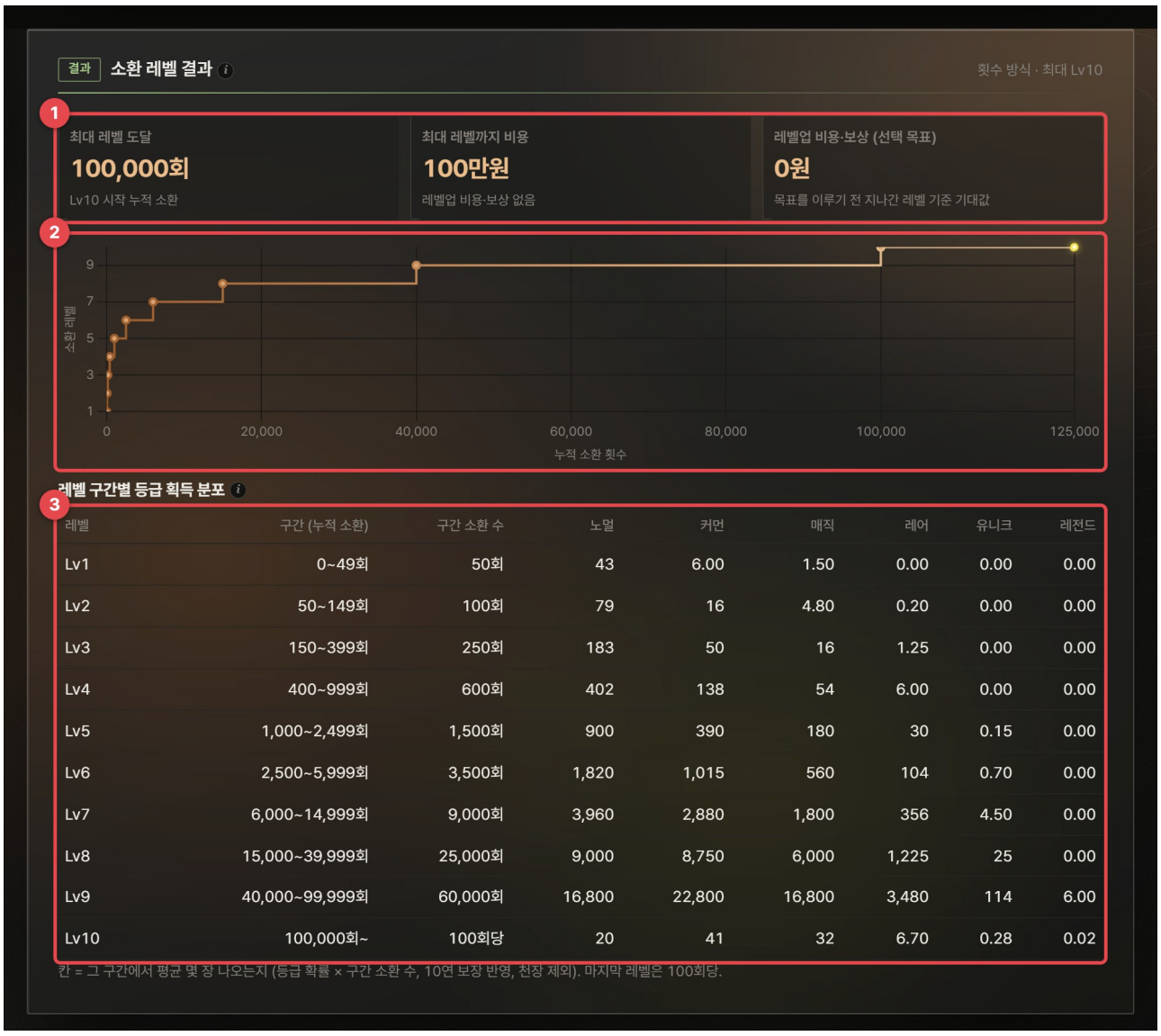


그림 22. 소환 레벨 결과 — 방치형 템플릿 (최대 Lv10)

1	**최대 레벨 도달 · 비용 · 레벨업 비용·보상** — 최대 레벨이 시작되는 누적 소환 수와 그때까지의 비용, 선택한 목표의 레벨업 비용·보상 기대값.
2	**레벨 진행 그래프** — 누적 소환 횟수(가로)에 따른 소환 레벨(세로) 계단.
3	**레벨 구간별 등급 획득 분포** — 레벨 구간마다 평균 몇 장씩 나오는지(등급 확률 × 구간 소환 수, 10연 보장 반영, 천장 제외). 칸에 마우스를 올리면 1회당 확률이 나옵니다.

방치형 템플릿은 Lv10이 누적 10만 회에 열리고, 거기까지 100만 원이 듭니다. 레벨 구간표를 보면 레전드는 Lv9 구간(4만~99,999회) 6만 회 동안 평균 6장이 나오고, 그 전 레벨에서는 0장입니다. 레전드 목표 비용의 대부분이 "레벨을 올리는 비용"이라는 뜻입니다.

레벨업 방식과 비용을 바꾸면 효과가 바로 보입니다. 같은 확률표에서 경험치 방식으로 소환 1회당 2 경험치를 주면 레벨 기준이 절반으로 당겨져, "레전드 전원 풀 초월" 목표(3.6절)의 기대 뽑기가 205,056회에서 172,305회로 16% 줄어듭니다. 레벨업마다 비용 500·보상 200 보석을 두면 아홉 번의 레벨업으로 순 2,700 보석(270원)이 목표 비용에 더해집니다.



그림 23. 고정 확률 결과 — 수집형 RPG 템플릿, 소환 레벨 꺼짐, 10연 보장 SR 이상

1	**목표 · 천장 도달 · 10연 보장** — 선택한 목표의 기대 뽑기와 비용, 천장까지 가는 비율, 10연 보장 상태.
2	**등급별 획득** — 표기 확률, 10연 보장을 반영한 실제 1회당 비율, 100회당 장수, 목표까지 평균 장수.

수집형 RPG 템플릿(R 91.2% · SR 8% · SSR 0.8%)에 "SR 이상 10연 보장"을 켜면 실제 비율은 R 87.219%,

SR 11.619%, SSR 1.162%가 됩니다. 앞 9번이 모두 R일 확률 $(1 - 0.088)^9 = 43.7\%$만큼 10번째의 R이 SR 이상으로 바뀌기 때문입니다. 그 결과 "SR 도감(20종)" 기대 뽑기는 899회에서 620회로 31% 줄고, "픽업 SSR 1장"은 80.4회에서 72.2회가 됩니다. 10연 보장이 결과에 영향을 주는 목표(보장 등급 이상의 목표)는 공식 대신 한 번씩 뽑는 시뮬레이션으로 계산합니다.

알고리즘 10연 보장의 실제 비율과 레벨업 비용 기대값

어디에 쓰였나: 고정 확률 결과·소환 레벨 결과의 실제 1회당 비율, 레벨 구간별 등급 분포, 기획서 영역, 목표 비용의 레벨업 비용·보상.

```
q = 보장 등급 이상 확률의 합,   a = (1 − q)⁹      ← 앞 9번에 보장 등급 이상이 없을 확률
보장 등급 이상 등급:  (9p + (1 − a)·p + a·p ÷ q) ÷ 10
그 아래 등급:         (9p + (1 − a)·p) ÷ 10
```

10번 중 앞 9번은 원래 확률 그대로이고, 10번째만 "앞 9번이 모두 꽝이면(a) 보장 등급 이상을 원래 비율대로(p ÷ q), 아니면 원래 확률"로 나옵니다. 이것을 10번 평균한 값입니다.

레벨업 비용 기대값 = Σ 레벨 (비용 − 보상) × P(목표를 이루기 전에 그 레벨 기준 뽑기 수에 닿음). 달성 분포(3.7절)에서 각 레벨 기준까지 끝내지 못할 확률을 읽어 더합니다. 부록 A.3.2.

3.6 ② 목표 — 무엇을 몇 장 모으나



그림 24. ② 목표 표

1	**목표** — 이름. 왼쪽 동그라미로 아래 그래프·분석에 표시할 목표를 고릅니다.
2	**종류** — 수집(등급 안 K종을 각 k장), 픽업 유닛 k장, 특정 캐릭터 k장.
3	**장수 (k)** — 한 종당 필요한 장수. 예: 6초월 = 6장.
4	**기대 뽑기 · 기대 비용 · 90% 비용** — 목표를 처음부터 따로 뽑는다고 할 때의 평균과 90% 유저가 달성하는 비용.
5	**단순 계산 대비** — 실제 기대 뽑기 ÷ (필요 장수 ÷ 최고 확률). ×1.71이면 단순 계산보다 71% 더 듭니다.
6	**계산** — 그 목표를 공식(정확해)으로 풀었는지 시뮬레이션으로 계산했는지 표시합니다.

목표마다 처음부터 따로 뽑는다고 계산합니다(천장 카운터·마일리지는 0에서 시작). "무과금 / 과금 유저" 열은 기대 비용을 하루 획득량(무과금 획득 + 월 과금 ÷ 30 ÷ 단가)으로 나눈 평균 도달일입니다.

3.7 달성 확률·예산과 분위



그림 25. 달성 확률과 예산 — 선택한 목표의 누적 달성 곡선

1	**달성 곡선** — 가로축 비용(또는 뽑기 수), 세로축 그 예산 안에 목표를 달성하는 비율.
2	**예산** — 금액을 넣으면 그 예산으로 달성하는 비율과 그 예산의 뽑기 수를 보여 줍니다.

그 아래 **달성 분위별 수치** 카드는 강화 탭과 같은 형식으로 0.1%~99.9% 분위의 뽑기·비용·무과금 도달일을 보여 줍니다. 천장이 있으면 "천장 구간 당첨 횟수" 분위도 함께 나옵니다. 시뮬레이션으로 계산한 목표는 3,000회 기본이라 0.1%·99.9%가 바깥쪽 3회로 정해집니다. 꼬리가 중요하면 왼쪽 **계산**에서 횟수를 늘리세요.

3.8 왜 이만큼 드나 — 비용 원인 분해



그림 26. 왜 이만큼 드나 — 레전드 전원 풀 초월 예시

1	① **단순 계산** — 필요 장수 ÷ 최고 확률. 기획서에서 흔히 쓰는 값 (120,000회).
2	② **중복이 없다면** — 소환 레벨 구간(초반 확률이 낮음)·천장·조각 교환 효과까지 넣되, 중복은 없다고 했을 때 (176,479회, ×1.47).
3	③ **실제** — 이미 다 모은 유닛의 중복까지 넣은 실제 기대값 (205,056회, ×1.16).
4	**획득 경로** — 시뮬레이션 평균으로 몇 장이 뽑기에서 직접, 몇 장이 조각 교환·마일리지에서 왔는지.

①→②의 배율이 크면 **확률 구조**(소환 레벨, 천장 시작 시점)가 원인이니 초반 확률이나 레벨 기준을 조정합니다. ②→③의 배율이 크면 **중복**이 원인이니 확률을 올리기보다 중복을 조각·교환으로 돌려주는 편이 효과적입니다. 검증 결과에도 같은 내용이 문장으로 나옵니다.

3.9 계산 검증과 수렴



그림 27. 계산 검증 — 공식으로 풀 수 있는 목표는 공식 계산과 시뮬레이션을 대조

공식으로 풀 수 있는 목표(확률이 고정되고 교환이 없는 수집, 분해 교환이 없는 픽업·특정 캐릭터)는 공식 계산값과 시뮬레이션을 나란히 보여 줍니다. 기본 템플릿처럼 소환 레벨이 여러 개거나 교환이 있는 수집 목표는 시뮬레이션만 표시하고, 수렴 그래프로 평균의 95% 범위를 보여 줍니다.

알고리즘 포아송화 — 수집 목표의 기대 뽑기 수를 적분으로 계산

어디에 쓰였나: 수집 목표(등급 안 K종을 각 k장), 확률 고정·교환 없음일 때의 공식 계산.

문제: 뽑기 횟수를 n으로 고정하면 각 유닛이 나온 횟수들이 서로 얽혀(합이 n) 계산이 어렵습니다. **포아송화**는 "뽑기가 시간 당 1회 비율로 무작위 시각에 일어난다"고 바꿔 생각하는 기법입니다. 그러면 각 유닛이 나오는 횟수가 서로 독립인 포아송 과정이 되어, "시각 t까지 K종이 모두 k장 이상"일 확률이 P(k, λt)^K 로 간단해집니다.

```
E[N] = (1/λ) · ∫₀^∞ [ 1 − P(k, u)^K ] du
  λ = 등급 확률 ÷ 종 수  (유닛 하나의 1회 확률)
  P(k, u) = 포아송(u) 값이 k 이상일 확률 = 정칙화 하부 불완전 감마 함수
```

연속 시간에서 완료 시각의 기대값이 원래 뽑기 횟수의 기대값과 같다는 성질을 씁니다. 적분은 수치로 계산합니다(구현 노트 I.4). k = 1이면 잘 알려진 쿠폰 수집 문제 $n \cdot H_n/p$와 일치하는지 테스트로 확인합니다. 부록 A.3.1.

알고리즘 상태 분포 동적 계획법 — 픽업·특정 캐릭터 목표의 분포 전체

어디에 쓰였나: 픽업 유닛 k장, 특정 캐릭터 k장 목표(분해 교환이 없을 때)의 기대값·분위수·예산별 달성 확률.

방법: 상태 = (모은 장수, 천장 카운터, 보장 여부). 뽑기 0회에서 모든 확률이 "0장, 카운터 0, 보장 없음"에 있습니다. 뽑기 한 번마다 각 상태의 확률을 다음 상태들로 나눠 보냅니다(최상위 당첨 → 픽업이면 장수+1·카운터 0, 아니면 보장 켜기 등). 목표 장수에 닿은 확률은 그 회차의 "달성 확률"로 빼 둡니다. 이렇게 하면 n회째에 처음 달성할 확률이 모두 나와서 평균뿐 아니라 분위수와 예산별 달성 확률까지 오차 없이 구해집니다. 마일리지는 뽑기 수만으로 정해지는 시점에 1장을 더하므로 그대로 반영됩니다. 부록 A.1.3.

알고리즘 기하분포 건너뛰기 시뮬레이션

어디에 쓰였나: 공식으로 풀 수 없는 목표(소환 레벨이 여러 개, 조각 교환 등)의 시뮬레이션.

방법: 한 번씩 굴리는 대신 "다음 당첨까지 몇 번 뽑나"를 기하분포에서 한 번에 뽑고, 확률이 바뀌는 지점(소환 레벨 경계, 천장 상승 구간, 마일리지·조각이 교환 가격에 닿는 시점)에서는 끊고 다시 뽑습니다. 결과 분포는 한 번씩 굴린 것과 같습니다. 다른 등급의 중복 조각은 뽑기당 기대값으로 쌓습니다. 덕분에 0.005% 확률 목표도 빠르게 계산됩니다. 부록 A.1.1, A.3.1, 구현 노트 I.3.

3.10 확률 공시표



그림 28. 확률 공시표

1	**소환 레벨** — 캐릭터별 확률을 볼 소환 레벨을 고릅니다.

2	**소수 자릿수** — 표시할 소수 자리(2~6자리). 반올림하면 0%로 보이거나 합계가 100%에서 어긋나는 경우를 아래 "참고"에서 알려 줍니다.
3	**공시표 CSV** — 등급별·캐릭터별 확률과 확률이 바뀌는 조건을 CSV로 저장합니다 (엑셀에서 바로 열림).
4	**① 등급별 확률 / ② 캐릭터별 확률 / ③ 실효 확률** — 레벨별 등급 확률, 캐릭터마다의 개별 확률과 1장 기대 뽑기·비용, 천장·보장을 포함한 실효 확률.

캐릭터 확률 = 등급 확률 × 등급 안 비중(균등, 픽업 등급은 픽업 비율로 나눔)입니다. **실효 확률**은 천장과 보장까지 포함해 "최상위 1장을 얻는 데 평균 몇 번 뽑나"의 역수로, 실제 체감 확률에 해당합니다. 천장이 있으면 **천장 확률 변동표**(j번째 뽑기의 확률과 누적 확률)도 나옵니다. **확률이 바뀌는 조건** 문장은 그대로 공시 문구 초안으로 쓸 수 있습니다. 아래의 "캐릭터 이름 입력"에 쉼표로 이름을 넣으면 공시표에 실제 이름이 나옵니다.

> **주의 공시 전에 반드시 확인**
>
> - 레벨마다 캐릭터 확률 합이 100%인지 (도구가 자동 검사)
> - 반올림 때문에 0.0000%로 보이는 캐릭터가 없는지 — 있다면 자릿수를 늘리세요
> - 천장·보장·소환 레벨처럼 확률이 바뀌는 조건을 모두 문장으로 밝혔는지

3.11 판단 사례 — 마일리지가 꼬리를 자르는 방식

방치형 동료 소환 템플릿에서 레전드 전원 풀 초월의 99% 비용은 266,762회입니다. 레전드 천장 8천 회는 평균을 26% 줄이지만, 레전드가 나오는 속도 자체를 바꿔 전투 밸런스까지 흔듭니다. 2만 회마다 레전드 선택권을 주는 마일리지는 평균을 20% 줄이면서 99% 꼬리를 정확히 200,000회로 자릅니다. 공시 확률은 그대로 두고 "아무리 운이 나빠도 여기까지"를 약속할 수 있어 1차안으로 고른 사례입니다 (docs/case-study.md).

4. 스탯·전투 밸런스

엔티티(캐릭터·몬스터) × 스탯 × 규칙 수식으로 레벨별 전투력(CP)과 1:1 승률을 계산하는 도구입니다. "CP는 높은데 실제로는 지는" 조합을 찾고, 상성을 원하는 승률로 맞추는 최소 변경을 계산하며, 결과를 Unity·Unreal로 내보냅니다.

4.1 무엇을 하는 도구인가

대부분의 게임은 스탯을 가중합한 **CP(Combat Power, 전투력)** 숫자를 화면에 보여 줍니다. 그런데 CP 수식이 공격 속도나 방어 감쇠 같은 상성을 제대로 반영하지 못하면, CP가 높은 캐릭터가 실제 1:1에서는 지는 일이 생깁니다. 이 도구는 실제 전투 규칙으로 1:1 승률을 계산해 CP 순서와 비교합니다. CP는 승패 계산에 쓰이지 않고, **CP 순서가 실제 승률 순서와 맞는지 비교하는 데만** 씁니다.

이런 질문에 답합니다

- CP가 높은데 실제로는 지는 캐릭터가 있나? CP와 실제 강함의 순위 상관은?
- 레벨 구간(Lv1~10, 11~20...)마다 누가 메타를 지배하나? 상성 순환(가위바위보)이 있나?
- 전사 대 궁수를 50%로 맞추려면 어느 스탯을 몇 배 바꾸면 되나?
- 흡혈·가시·광폭화 같은 특성이나 아이템이 승률을 몇 %p 바꾸나?
- 공격 속도 4배인 캐릭터는 턴 교대 방식과 시간 기반 방식에서 승률이 얼마나 다른가?

활용 예

- **신규 직업 추가** — 기존 직업들과의 승률 행렬을 보고, 지배하거나 완전히 밀리는 상대가 없도록 스탯을 맞춥니다.
- **CP 수식 검토** — CP 순위와 실제 순위의 상관이 낮으면 CP 수식(가중치)을 고칩니다.
- **아이템·특성 밸런스** — 아이템 하나를 껐다 켰을 때의 점수 변화로 아이템의 실제 가치를 봅니다.
- **Kalivra 사용자** — Kalivra 프로젝트(.kal)를 그대로 불러와 같은 규칙의 결과를 공식 계산으로 확인합니다.

> **팁 스탯·전투와 전투 시뮬레이터(6장) 중 무엇을 쓰나**
>
> **스탯·전투**는 사용자 정의 스탯과 규칙 수식으로 1:1 승률을 공식으로 정확히 풀고(특성·데미지 변동이 있으면 시뮬레이션), 실제 전투 흐름은 배틀 시뮬레이터 카드(4.10절)로 보여 주며, 레벨 곡선·CP 순위·최소 변경 역산까지 해 줍니다. 스탯 설계와 CP 수식 검토에 맞습니다. **전투 시뮬레이터**는 데이터 테이블의 스킬·효과·상성·시너지로 1:1과 파티 전투를 시뮬레이션합니다. 스킬 구성, 버프·디버프, 보스 패턴, 파티 조합을 볼 때 씁니다. 두 탭은 데이터를 공유하지 않습니다.

4.2 화면 구성

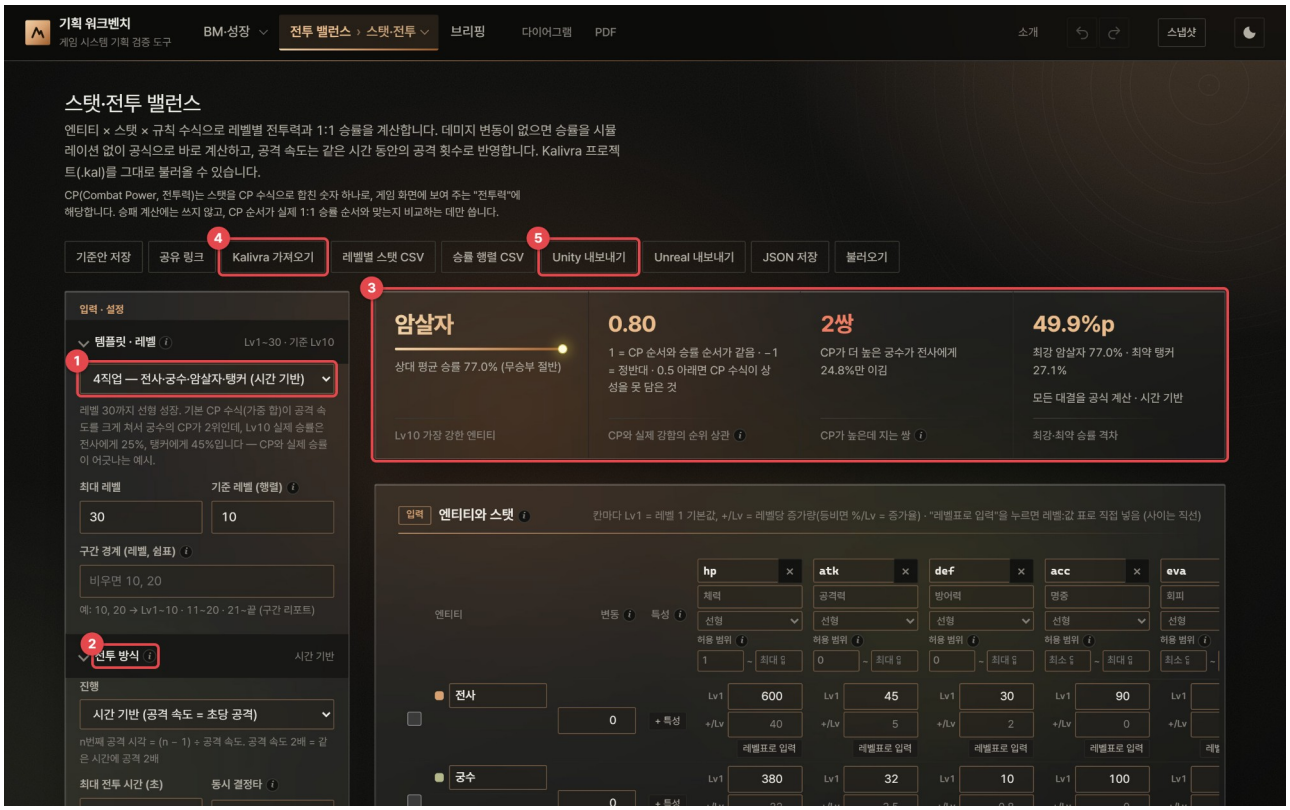


그림 29. 스탯·전투 밸런스 — 처음 화면

1	**템플릿 · 레벨** — 4직업(시간 기반), Kalivra 기본 샘플(턴 교대), 특성 예시, 빈 설계. 아래에서 최대 레벨, 기준 레벨(승률 행렬을 볼 레벨), 구간 경계(구간 리포트용, 예: 10, 20)를 정합니다.
2	**전투 방식** — 진행 방식(시간 기반 / 턴 교대), 최대 전투 시간, 동시 결정타 처리, 명중 하한·상한, 데미지 반올림, 시뮬레이션 횟수. 그 아래 **규칙 수식**이 있습니다.

3	**핵심 지표** — 기준 레벨에서 가장 강한 엔티티(상대 평균 승률), CP와 실제 강함의 순위 상관, CP가 높은데 지는 쌍 수, 최강·최약 승률 격차(아래 줄에 계산 방식: 공식 계산 / 시뮬레이션, 시간 기반 / 턴 교대).
4	**Kalivra 가져오기** — Kalivra v1.0.1 프로젝트 파일(.kal/JSON)을 이 탭 설정으로 바꿉니다. 바뀌거나 빠진 항목은 알림으로 보여 줍니다.
5	**Unity / Unreal 내보내기** — 엔진에 바로 넣을 데이터와 규칙 코드를 ZIP으로 받습니다 (4.14절).

머리의 설명 아래에 CP가 무엇인지 한 줄 설명이 있고, KPI·레벨 곡선·검증 카드에도 같은 뜻이 반복되어 처음 보는 사람도 CP를 승률과 혼동하지 않게 했습니다.

4.3 빠르게 시작하기

1. **템플릿**을 고릅니다. 4직업 템플릿은 CP 2위 궁수가 전사에게 24.8%밖에 못 이기는 "CP와 승률이 어긋나는" 예시입니다.
2. **전투 방식**과 **규칙 수식**을 우리 게임 규칙에 맞춥니다 (데미지·명중·치명·공격 속도·체력·CP 수식).
3. **엔티티와 스탯** 표에 캐릭터와 스탯(기본값·레벨당 증가)을 넣습니다.
4. **승률 행렬**에서 칸을 눌러 **1:1 상세**를 보고, **CP와 실제 승률**에서 어긋나는 쌍을 확인합니다. 실제로 싸우는 흐름은 그 아래 배틀 시뮬레이터(4.10절)의 FIGHT·M.C로 봅니다.
5. **밸런스 도구**에서 대상·상대·목표 점수를 정하면 필요한 스탯 배율이 나오고, **적용**으로 반영합니다.

4.4 전투 방식

설정	설명
시간 기반 (기본)	n번째 공격 시각 = (n − 1) ÷ 공격 속도. 공격 속도가 2배면 같은 시간에 2배 공격합니다. 두 결정타가 같은 시각이면 "동시 결정타" 설정(동전 반반 / 무승부)을 따릅니다.
턴 교대 (Kalivra 방식)	공격 속도 비율로 선공 확률을 정하고(A 선공 = A 속도 ÷ (A + B 속도)), 한 번씩 번갈아 공격합니다. 공격 속도는 선공 확률에만 쓰입니다.
최대 전투 시간 / 최대 턴	그 안에 끝나지 않으면 무승부로 따로 셉니다 (Kalivra는 양쪽 모두 패배로 셈).
명중 하한·상한	명중 수식 결과를 이 범위로 자릅니다. 치명 확률은 0~100%로 자릅니다.
데미지 반올림	내림 / 반올림 / 없음. 치명 배율까지 곱한 뒤 적용합니다.

설정	설명
시뮬레이션 횟수	데미지 변동·특성이 있을 때와 공식 계산 검증용 시뮬레이션 횟수.

두 방식의 차이는 결론을 뒤집을 만큼 큽니다. 공격 속도만 4배인 두 캐릭터(체력·공격 같음)는 턴 교대에서 80%, 시간 기반에서 100% 이깁니다. 4직업 템플릿의 전사(Lv10)는 탱커에게 시간 기반으로 84.3%, 턴 교대로 3.7%를 이깁니다. 우리 게임의 실제 전투가 어느 쪽에 가까운지 먼저 정하세요.

4.5 규칙 수식

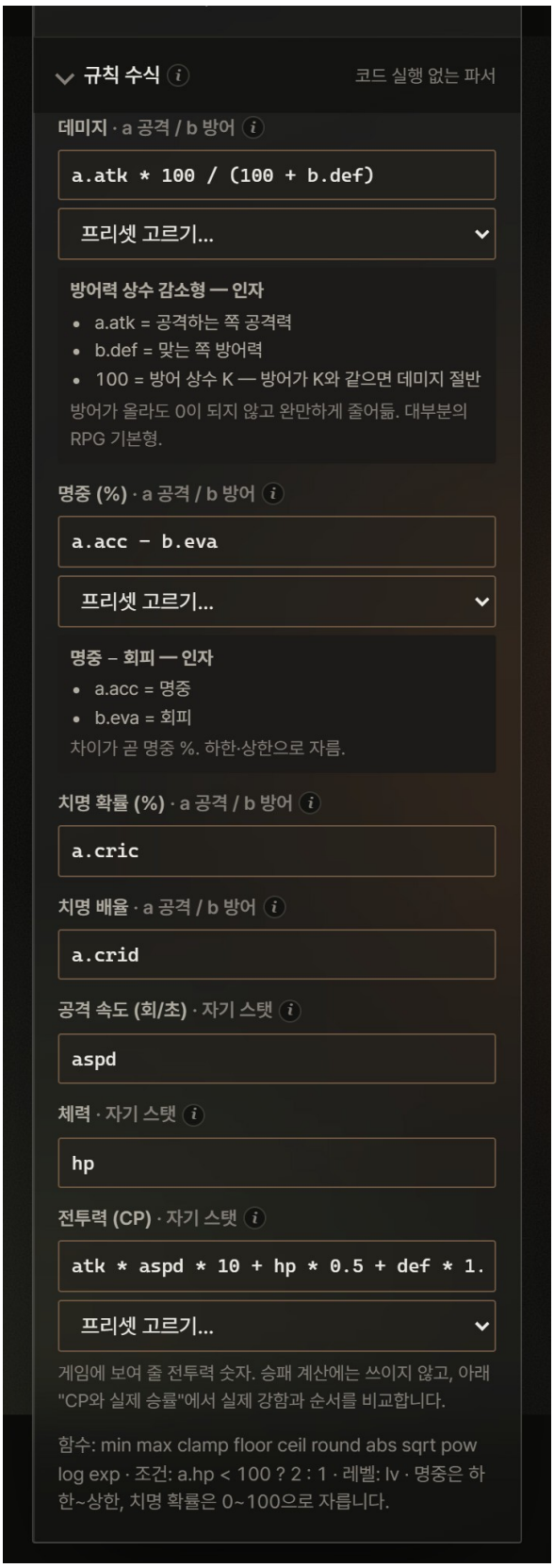


그림 30. 규칙 수식 — 데미지·명중·치명·공격 속도·체력·CP

- **데미지 · a 공격 / b 방어** — 공격하는 쪽 스탯은 `a.atk`, 맞는 쪽은 `b.def`처럼 씁니다. 프리셋: 방어 감쇠형 `a.atk * (100 / (100 + b.def))`, 차감형(최소 10%), 비율형(공 ÷ 방, 상한 3배), 레벨 차 보정.
- **명중 (%)** — 프리셋: 명중 − 회피, 비율(명중 ÷ (명중 + 회피)), 항상 명중.
- **치명 확률 (%) · 치명 배율** — a·b 스탯을 모두 쓸 수 있습니다.
- **공격 속도 (회/초) · 체력 · 전투력(CP)** — 자기 스탯만 씁니다 (`atk`, `hp`, `lv`). CP 프리셋: 가중 합 (Kalivra 기본), 기대 DPS × 체력 제곱근.
- 함수: `min max clamp floor ceil round abs sqrt pow log exp`, 조건식 `a.hp < 100 ? 2 : 1`, 레벨 `lv`. 거듭제곱은 `^` 또는 `**`.

수식에 오류가 있으면 그 칸 아래에 위치와 함께 오류가 표시되고 계산이 멈춥니다. 다른 사람이 보낸 설정 파일의 수식 칸에 코드가 들어 있어도 실행되지 않습니다.

4.6 엔티티와 스탯

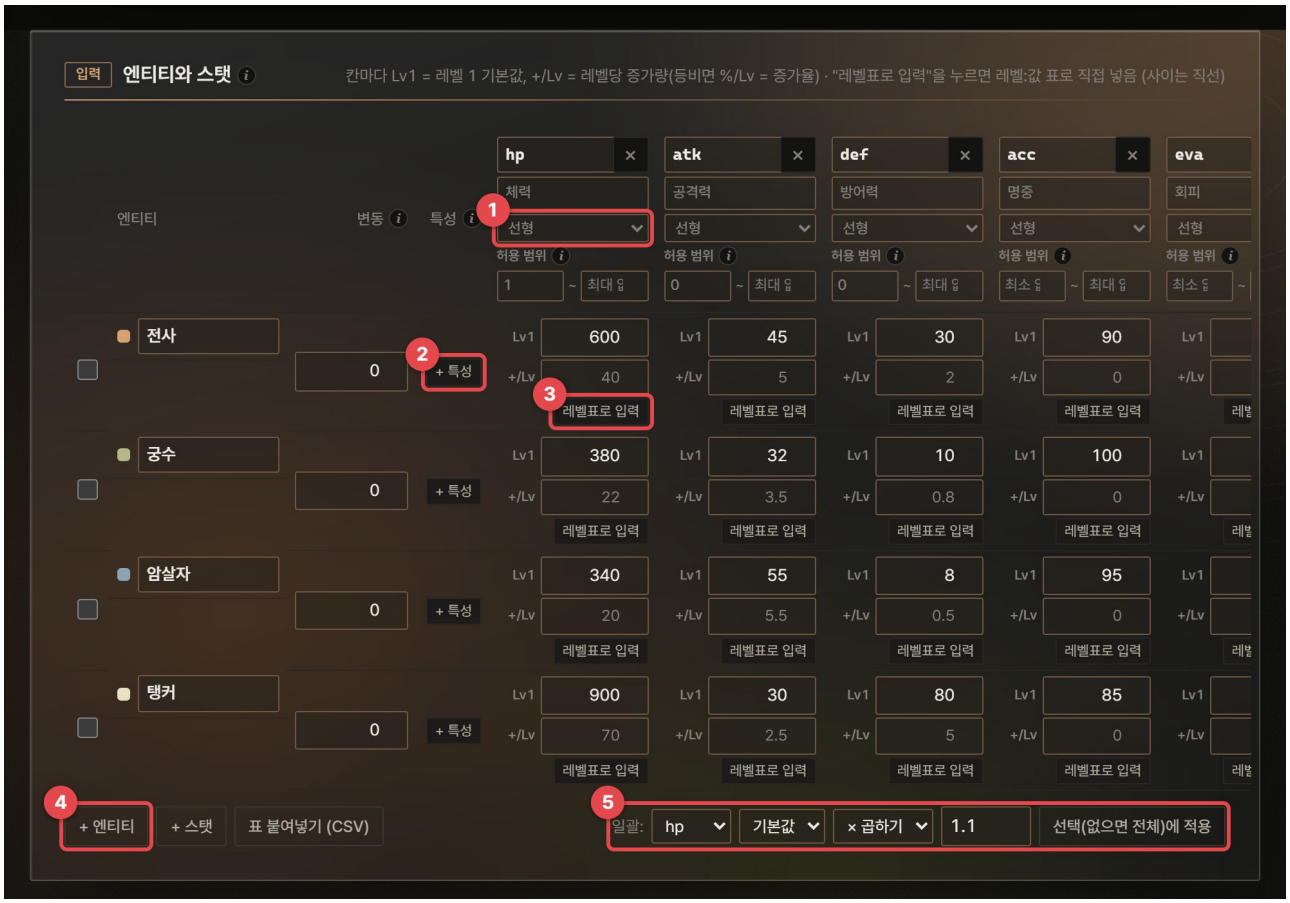


그림 31. 엔티티와 스탯 — 칸마다 위 = 기본값(Lv1), 아래 = 레벨당 증가

1	**성장 방식** — 스탯 열마다 선형(기본 + (L−1) × 증가) 또는 등비(기본 × (1 + g%)^(L−1))를 고릅니다.
2	**+ 특성** — 이 엔티티에 특성(발동 조건 + 효과)을 붙입니다. 표 아래에 특성 편집 패널이 열립니다.
3	**표** — 레벨별 값을 직접 넣습니다. 띄엄띄엄 넣으면 사이는 직선으로 잇고, 양 끝 밖은 끝값을 씁니다(구간 성장).
4	**+ 엔티티 / + 스탯 / 표 붙여넣기 (CSV)** — 행·열 추가와 엑셀 표 붙여넣기.
5	**일괄** — 고른 스탯의 기본값·성장값에 곱하기·더하기·바꾸기를 선택한 엔티티(없으면 전체)에 한 번에 적용합니다.

"변동" 열은 데미지 변동 폭(%)입니다. 변동이 0이면 공식 계산, 0보다 크면 그 엔티티가 낀 대결은 시뮬레이션으로 계산합니다.

특성 (트리거)

구성	선택지
발동 시점	전투 시작 · 매 턴(시간 기반은 매 1초) · 공격 직전 · 공격 적중 · 치명타 · 공격 빗나감 · 회피 성공 · 피격
조건	발동 확률(%), 체력 조건(자신/상대 체력 n% 이하), 최대 발동 횟수(0 = 무제한)
효과	회복 · 추가 데미지 · 이번 공격 데미지 × · 이번 공격 데미지 + · 스탯 버프
효과 기준	고정값 · 이번 공격 데미지의 % · 대상 최대 체력의 % · 대상 기본 스탯의 %
지속	스탯 버프의 지속 — 시간 기반은 초, 턴 교대는 자기 공격 수, 0 = 끝까지

프리셋으로 흡혈·가시·광폭화·선제 방어·분노·처형·재생·반격을 바로 넣을 수 있습니다. 특성이 있는 대결은 두 쪽의 공격 횟수가 서로 영향을 주므로 시뮬레이션으로 계산하고, 결과에 95% 범위가 붙습니다. 효과는 발동 한 번에 한 번만 적용하고, 확률은 발동마다 한 번만 굴리며, 쓰러뜨리는 공격 뒤에는 맞은 쪽의 피격 효과가 나오지 않습니다(이 세 가지는 Kalivra와 다르게 고친 규칙).

4.7 아이템과 빌드



그림 32. 아이템 (스탯 수정자)

1	**고정 +** — 스탯에 더하는 값.
2	**% +** — 퍼센트 증가. 같은 스탯의 %는 합산합니다.
3	**× 배율** — 곱하는 배율. 여러 아이템이면 곱합니다.
4	**+ 아이템** — 아이템 추가. 아이템마다 켜기, 대상(전체 또는 특정 엔티티), 특성(아이템에 붙는 트리거)을 정합니다.

```
최종 스탯 = (기본값 + Σ고정) × (1 + Σ% ÷ 100) × Π배율   → 그 뒤 스탯 최소·최대로 자름
(아이템 순서와 무관 — Kalivra는 순서에 따라 결과가 달랐음)
```

아이템 세트 (빌드) — "지금 아이템을 빌드로 저장"으로 지금 아이템 목록(켜기·대상·수정자·특성)에 이름을 붙여 두고, 빌드마다 엔티티 평균 승률을 표로 비교합니다. "장비 세트 A와 B 중 어느 쪽이 더 균형적인가"를 볼 때 씁니다.

4.8 레벨 곡선과 승률 행렬

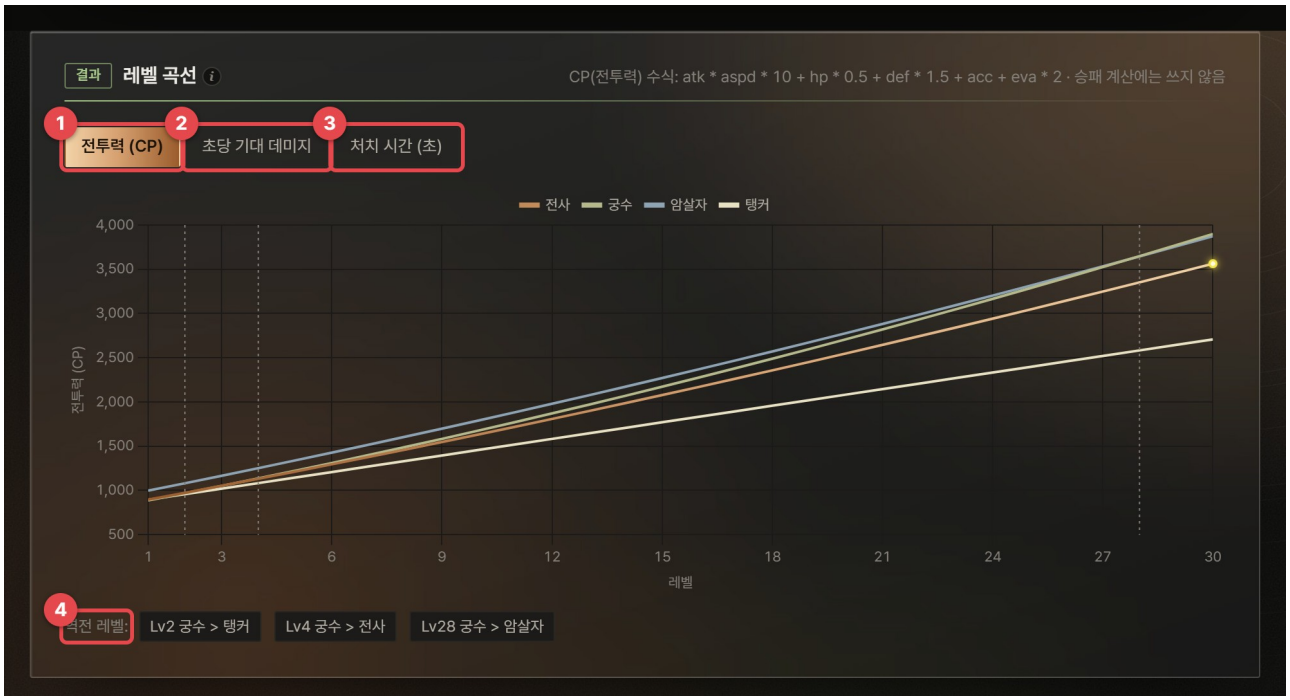


그림 33. 레벨 곡선 — 전투력(CP) / 초당 기대 데미지 / 처치 시간

1	**전투력 (CP)** — 엔티티별 레벨에 따른 CP.
2	**초당 기대 데미지** — 기준 상대(같은 레벨)에게 주는 초당 기대 데미지 (변동 제외).
3	**처치 시간** — 기준 상대를 쓰러뜨리는 데 걸리는 기대 시간.
4	**역전 레벨** — 두 곡선이 교차하는 레벨 (세로 점선). 예: Lv4부터 궁수 CP가 전사보다 높아짐.

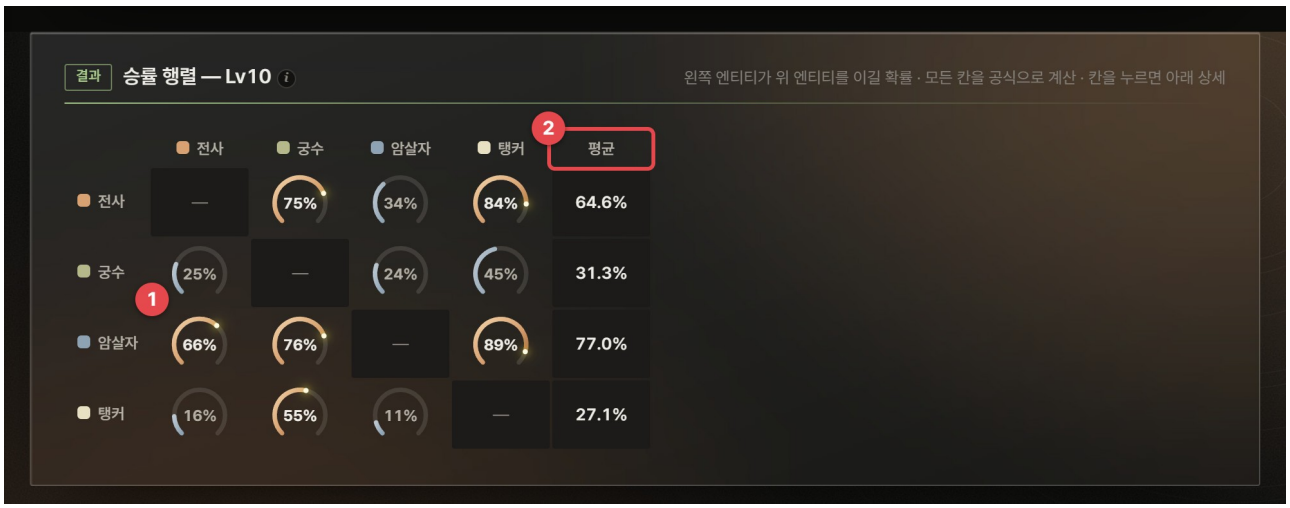


그림 34. 승률 행렬 — 왼쪽 엔티티가 위 엔티티를 이길 확률 (칸의 아치 = 승률, 탄색 = 이김 · 청회색 = 짐)

1	**칸** — 누르면 아래 1:1 상세가 그 대결로 바뀝니다. 공식 계산이 안 되는 칸(변동·특성이 있는 대결)은 시뮬레이션 값 아래에 ±오차가 함께 나오고, 95% 범위가 50%를 걸치면(어느 쪽이 강한지 확실하지 않으면) 흐리게 표시합니다. 칸에 마우스를 올리면 승·패·무와 95% 구간이 나옵니다.
2	**평균** — 그 엔티티의 모든 상대 평균 승률 (무승부는 절반 승으로 셈). 이 값으로 "실제 순위"를 정합니다.

4.9 1:1 상세



그림 35. 1:1 상세 — 승률 게이지·무·패, 공격 프로필, 공식과 시뮬레이션 대조, 수렴·전투 길이·전투 로그

1	**대결 선택** — 두 엔티티를 고릅니다 (승률 행렬 칸을 눌러도 바뀜).

2 **공식 계산 vs 시뮬레이션** — 승률·무승부·실현 명중·치명·평균 전투 길이를 나란히 비교하고, 계산값이 시뮬레이션 95% 범위 안에 있는지 표시합니다.

- **공격 프로필** — 서로에 대한 명중률, 치명 확률, 1회 데미지(일반/치명), 공격 속도, 상대 체력. 그 아래 "쓰러뜨리는 공격: 평균 10.9회(9.04초) · 중앙값 11회 · 운 나쁜 1% 17회"처럼 처치에 필요한 공격 횟수 분포가 요약됩니다.
- **시뮬레이션 수렴** — 시뮬레이션 횟수에 따른 누적 승률과 Wilson 95% 범위, 점선(공식 계산값). 범위가 ±1%p보다 넓으면 "아직 ±1%p보다 넓음"이라고 알려 줍니다.
- **전투 길이 분포** — 이긴 쪽별로 나눈 전투 길이 히스토그램(점묘, 색 = 이긴 쪽) (시간 기반은 초, 턴 교대는 턴).
- **전투 로그** — 같은 규칙의 시뮬레이션 20전투 중 하나를 골라 체력 타임라인(▲ 치명타, ◆ 특성 발동)과 사건 목록(시각, 공격자, 결과, 남은 체력)을 봅니다. "왜 이런 승률이 나오는지"를 설명할 때 유용합니다.

알고리즘 공격 횟수 분포 동적 계획법 — 1:1 승률을 시뮬레이션 없이 계산

어디에 쓰였나: 승률 행렬의 "공식 계산" 칸, 1:1 상세의 승·무·패와 처치 공격 횟수 분포, 레벨별 승률, 밸런스 도구 전부(변동·특성이 없을 때).

아이디어: 공격 한 번의 결과는 빗나감(1 − h), 일반 적중(h·(1 − c), 데미지 d), 치명(h·c, 데미지 d·m) 세 가지입니다. 상태를 (일반 적중 수 j, 치명 수 k)로 두면 "아직 살아 있음 ⇔ j·d + k·d·m < 상대 체력"이고, 공격마다 상태 확률을 다음 상태로 전진시키면 **n번째 공격에서 처음 쓰러뜨릴 확률 P(N = n)**이 모두 나옵니다.

```
시간 기반:  P(A 승) = Σₙ P(N_A = n) · P(B가 A의 n번째 공격 시각 전에 못 쓰러뜨림)
            A의 n번째 공격 시각 = (n − 1) ÷ A 공격 속도   (같은 시각이면 동전 반반 또는 무승부)
턴 교대:    P(A 승) = Σₙ P(N_A = n) · [ 선공 · S_B(n − 1) + (1 − 선공) · S_B(n) ]
            S_B(m) = P(B가 m번 공격 안에 못 쓰러뜨림),  선공 = A 속도 ÷ (A 속도 + B 속도)
```

A와 B의 처치 공격 횟수는 서로 독립(특성·버프가 없을 때)이라 이렇게 곱해서 더할 수 있습니다. 결과는 같은 설정이면 언제나 같은 값이고, Kalivra v1.0.1 엔진을 그대로 돌린 40만~140만 회 결과와 3σ 안에서 일치합니다(예: 0.72643 대 0.72676). 부록 A.4.1.

알고리즘 **Wilson 점수 구간 — 승률의 95% 범위**

어디에 쓰였나: 시뮬레이션으로 계산한 승률 칸, 1:1 상세의 시뮬레이션 열, 승률 수렴 그래프.

$$\text{중심} = (\hat{p} + z^2/2n) \div (1 + z^2/n)$$

$$\text{반폭} = z \cdot \sqrt{\hat{p}(1-\hat{p})/n + z^2/4n^2} \div (1 + z^2/n), \quad z = 1.96$$

흔히 쓰는 $\hat{p} \pm 1.96\cdot\sqrt{(\hat{p}(1-\hat{p})/n}$은 승률이 0%·100%에 가까우면 범위가 0~100%를 벗어나거나 지나치게 좁아집니다. Wilson 구간은 그런 경우에도 범위가 0~1 안에 있고 실제 포함 확률이 95%에 가깝습니다. 부록 A.4.2.

4.10 배틀 시뮬레이터 (FIGHT · M.C)

1:1 상세(4.9절)가 "이 대결의 승률은 정확히 몇 %인가"에 답한다면, 배틀 시뮬레이터는 "실제로 싸우면 어떻게 흘러가나"를 보여 줍니다. 스탯·전투 탭에서 1:1 상세 바로 아래에 있는 카드로, 이 탭의 엔티티·아이템·특성과 전투 방식·규칙 수식을 그대로 써서 전투를 한 판씩 돌립니다. 공격 쪽 하나를 골라 상대 한 명 또는 전체와 정해진 횟수만큼 싸우게 하고(FIGHT), 한 쌍을 1만 번 돌려 선공·남은 체력·오버킬까지 봅니다(M.C). 난수는 시드를 고정해 같은 설정이면 같은 숫자가 나옵니다. 화면 구성은 Kalivra의 Battle Simulator(FIGHT · M.C 버튼)를 따랐습니다.

팁 **이름이 비슷한 6장 전투 시뮬레이터와 다른 점**

이 카드는 스탯·전투 탭의 1:1 대결만 다루고 스킬·버프·파티가 없습니다. 데이터 테이블(5장)의 스킬·효과·시너지로 파티 전투를 돌리는 것은 별도 탭인 전투 시뮬레이터(6장)입니다. 두 도구는 데이터를 공유하지 않습니다.

이런 질문에 답합니다

- 전사는 Lv10 궁수에게 몇 % 이기고, 보통 몇 초 만에 끝나나? 이기는 판과 지는 판은 어떻게 흘러가나?
- 턴 교대 규칙에서 선공을 잡으면 승률이 얼마나 오르나? 선공 운이 승패를 좌우하는 대결인가?

- 이길 때 체력을 얼마나 남기나 — 여유 있게 이기나, 아슬아슬하게 이기나?
- 초반·중반·후반 중 언제 강하고 언제 약한가?

FIGHT — 상대마다 정해진 횟수만큼



그림 36. 배틀 시뮬레이터 — FIGHT (4직업 템플릿, 전사 대 전체 Lv10, 상대마다 100전투)

1	**공격 쪽 · 상대** — 싸울 두 쪽. 상대에 "전체 (리그)"를 고르면 공격 쪽이 나머지 엔티티 모두와 차례로 싸웁니다.
2	**레벨 · 횟수** — 두 엔티티에 함께 쓰는 레벨(처음에는 기준 레벨)과, FIGHT에서 상대마다 돌릴 전투 수(1~5,000).
3	**M.C · FIGHT** — M.C는 상대 한 명과 10,000번, FIGHT는 상대마다 "횟수"만큼 싸웁니다. 규칙이나 스탯을 바꾸면 결과는 지워집니다.

4	**승률 · 평균 길이** — 상대마다 공격 쪽 승률(초록, 왼쪽 축)과 평균 전투 길이(청회색, 오른쪽 축 — 시간 기반은 초, 턴 교대는 턴). 막대에 마우스를 올리면 무승부 비율과 이긴 판만의 평균 길이가 나옵니다.
5	**VS 상대** — 상대별 묶음. 평균 길이와 승률(50% 이상 초록, 미만 빨강). 눌러서 접고 폅니다.
6	**전투 로그 ◀ ▶** — 앞 100전투를 하나씩 넘겨 봅니다. 결과·길이(턴 교대면 선공도)가 나오고, 아래에 시각(또는 턴)마다 공격·빗나감·치명타·특성 발동과 두 쪽의 남은 체력이 이어집니다.
7	**스탯** — 그 레벨의 최종 스탯(아이템 포함)을 엔티티별 카드로 보여 줍니다.

4직업 템플릿의 처음 상태(시간 기반, 기준 레벨 10)에서 전사로 전체와 100번씩 싸우면 궁수에게 75번(평균 8.71초), 암살자에게 32번(7.71초), 탱커에게 79번(36.56초) 이깁니다. 궁수 상대 첫 전투의 로그를 넘겨 보면 두 쪽 모두 0초에 첫 공격을 하고, 전사는 한 대에 76(치명타 115), 궁수는 49(치명타 83)를 넣습니다. 궁수가 공격 속도 1.68로 전사(1.09)보다 1.5배 자주 치지만, 체력 578인 궁수는 여덟 대면 쓰러지고 체력 960인 전사는 스무 대를 버텨, 첫 전투는 전사가 체력 221을 남기고 7.34초에 이깁니다. 암살자에게 지는 이유도 스탯 카드에서 보입니다 — 암살자의 회피 34.5 때문에 전사의 명중은 90 − 34.5 = 55.5%로 떨어지고, 암살자는 치명 확률 35%에 치명 배율 2배입니다.

상대를 한 명(예: 궁수)으로 고르고 FIGHT를 누르면 구간별 승률 카드가 함께 나옵니다. 구간마다 레벨별 승률을 평균한 값으로, 공식으로 풀리는 레벨은 공식 계산, 아니면 시뮬레이션이고 구간 경계는 왼쪽 "템플릿 · 레벨"을 따릅니다(4.11절의 구간 리포트와 같은 경계). 전사 대 궁수는 Lv1~10 73.5%, Lv11~20 75.9%, Lv21~30 73.8%로, 레벨과 상관없이 전사가 비슷하게 앞섭니다.

M.C — 한 쌍을 1만 번



그림 37. 배틀 시뮬레이터 — M.C (전사 대 궁수 Lv10, 10,000번)

1	**전체 승률** — 공격 쪽 승률과 Wilson 95% 범위. 공식으로 풀리는 대결이면 공식 계산값도 함께 나오고 (특성·데미지 변동이 있으면 시뮬레이션만), 무승부가 있으면 그 비율도.
2	**선공일 때 승률** — 턴 교대에서 공격 쪽이 먼저 공격한 판만의 승률, 선공 비율, 후공일 때 승률. 시간 기반은 둘 다 0초에 시작하므로 이 예처럼 "—"로 표시합니다. 두 승률이 20%p 넘게 벌어지면 빨갛게.
3	**평균 길이 · 계산 시간** — 모든 판의 평균 전투 길이와 공격 쪽이 이긴 판만의 평균, 1만 번에 걸린 시간.
4	**실현 치명타율 · 회피율** — 치명타율은 적중 중 비율(아래 작은 글씨는 빗나간 공격까지 분모에 넣은 Kalivra 표기), 회피율은 상대의 공격 중 빗나간 비율.
5	**평균 DPS(DPT) · 평균 오버킬** — 공격 쪽이 가한 데미지 합 ÷ 전투 길이 합. 시간 기반은 초당(DPS), 턴 교대는 턴당(DPT)입니다. 오버킬 = 공격 쪽이 이긴 판에서 마지막 공격이 상대 체력을 넘긴 양의 평균.
6	**전투 길이 분포** — 이긴 쪽별로 쌓은 막대 — 1:1 상세의 전투 길이 분포와 같은 형식.
7	**승리 시 남은 체력 분포** — 이긴 쪽이 전투 끝에 남긴 체력(절댓값)을 10칸으로. 0 근처에 몰리면 아슬아슬한 승리, 오른쪽이면 여유 있는 승리입니다.

4.3절에서 본 "CP 2위 궁수가 전사에게 24.8%밖에 못 이기는" 대결을 1만 번 돌리면 전사 승률은 75.13%(95% 74.3~76.0%)이고, 공식 계산 75.16%가 범위 안에 있습니다. FIGHT 100번의 75.0%와 같은 값을 훨씬 좁은 범위로 보여 주는 셈입니다. 실현 치명타율 10.1%는 전사의 치명 확률 10%, 회피율 5.0%는 궁수 명중 100 − 전사 회피 5 = 95%와 맞습니다. 작은 글씨의 공격 중 7.3%는 빗나간 공격까지 분모에 넣은 값으로, 10.1% × 전사 명중률 72%(90 − 17.7)와 같습니다.

남은 체력 분포에서 전사의 승리는 0부터 530 근처까지 퍼져 있고 중앙값이 187입니다. 최대 체력 960의 5분의 1 정도만 남기고 이기는 판이 보통이라는 뜻입니다. 반대로 궁수의 승리는 절반 이상(55%)이 첫 칸(0~58)에 몰려 있어 이겨도 거의 아슬아슬하게 이기고, 전사도 전체 판의 9.8%는 58 이하만 남기고 이깁니다. 두 쪽 모두 한두 대 차이로 갈리는 판이 많으니, 궁수의 체력이나 공격력을 조금만 바꿔도 이 판들이 뒤집혀 승률이 움직입니다. 평균 DPS 68.8은 전사가 1초에 평균 69를 넣는다는 뜻으로, 궁수 체력 578을 깎는 데 약 8.4초가 필요합니다 — 전사가 이긴 판의 평균 길이 8.25초와 맞습니다.

선공 효과는 턴 교대에서만 나타납니다. 왼쪽 전투 방식의 진행 방식을 턴 교대로 바꾸고 전사 대 암살자로 M.C를 돌리면, 전사는 45.1%의 판에서 먼저 공격하고 선공일 때 58.6%, 후공일 때 47.6%를 이겨 전체 승률이 52.6%입니다. 11%p 차이는 첫 공격 한 번이 결과를 꽤 바꾼다는 뜻입니다. 두 승률이 20%p 넘게 벌어지면 선공 운이 승패를 좌우한다는 신호이므로, 선공 확률을 정하는 공격 속도나 첫 공격의 데미지를 다시 봅니다.

알고리즘 배틀 시뮬레이터 — 선공으로 나눈 승률과 Kalivra와 다른 점

어디에 쓰였나: M.C의 선공일 때 승률, FIGHT·M.C 전체.

```
승률 = 선공 비율 × 선공일 때 승률 + (1 − 선공 비율) × 후공일 때 승률
     = 0.451 × 0.586 + 0.549 × 0.476 ≈ 0.526      (턴 교대, 전사 대 암살자 Lv10)
선공 비율 ≈ A 공격 속도 ÷ (A 공격 속도 + B 공격 속도) = 1.09 ÷ (1.09 + 1.29) ≈ 0.458
```

전체 확률의 법칙: 승률을 "선공한 판"과 "후공한 판"으로 나눠 가중 평균한 것입니다. 두 승률의 차이가 선공 운의 영향이고, 선공 비율은 턴 교대 규칙대로 공격 속도 비에서 나옵니다(4.4절). 1만 번 중 실제 선공 비율 45.1%는 이 비 45.8%와 시뮬레이션 오차 안에서 맞습니다.

Kalivra와 다른 점: ① 시드를 고정해 같은 설정이면 같은 숫자(Kalivra는 Math.random이라 누를 때마다 바뀜). ② 무승부를 따로 셈. ③ Kalivra의 "Realized Crit %"는 빗나간 공격까지 분모에 넣어 치명 확률보다 낮게 나오므로, 적중 중 비율과 나눠 보여 줌. ④ 구간별 승률을 레벨당 10전투 대신 공식 계산(또는 더 많은 시뮬레이션)으로. 부록 A.1.1(몬테카를로), A.4.2(Wilson 구간).

4.11 레벨별 승률 (구간 리포트)



그림 38. 레벨별 승률 — 엔티티별 평균 승률 곡선과 구간 표, 구간 지배 경고

모든 레벨에서 승률 행렬을 계산해 엔티티별 평균 점수(승 + 무승부 ÷ 2)를 곡선으로 그리고, "템플릿 · 레벨"의 구간 경계로 나눈 구간별 평균을 표로 보여 줍니다. 구간 평균으로 모든 상대에게 55% 이상 이기는 엔티티는 "지배"로 경고합니다. 시뮬레이션이 필요한 쌍이 많으면 전투 6만 회 예산 안에서 반복 수를 줄이거나 레벨을 건너뛰고, 화면이 멈추지 않도록 레벨마다 나눠 계산합니다.

4직업 템플릿(Lv1~30, 구간 Lv1~10 · 11~20 · 21~30)에서는 세 구간 모두 암살자가 구간 평균으로 모든 상대에게 55% 이상 이겨(평균 승률 82.4% → 76.0% → 72.5%) "지배" 경고가 세 번 나옵니다. 전사는 58.7%에서 69.7%로 후반에 강해지고, 탱커는 29.9%에서 21.3%로 약해집니다.

상성 순환: A가 B를, B가 C를, C가 A를 55% 이상 이기는 세 엔티티를 찾아 "의도한 가위바위보 구조인지 확인하세요"라고 알려 줍니다.

4.12 CP와 실제 승률

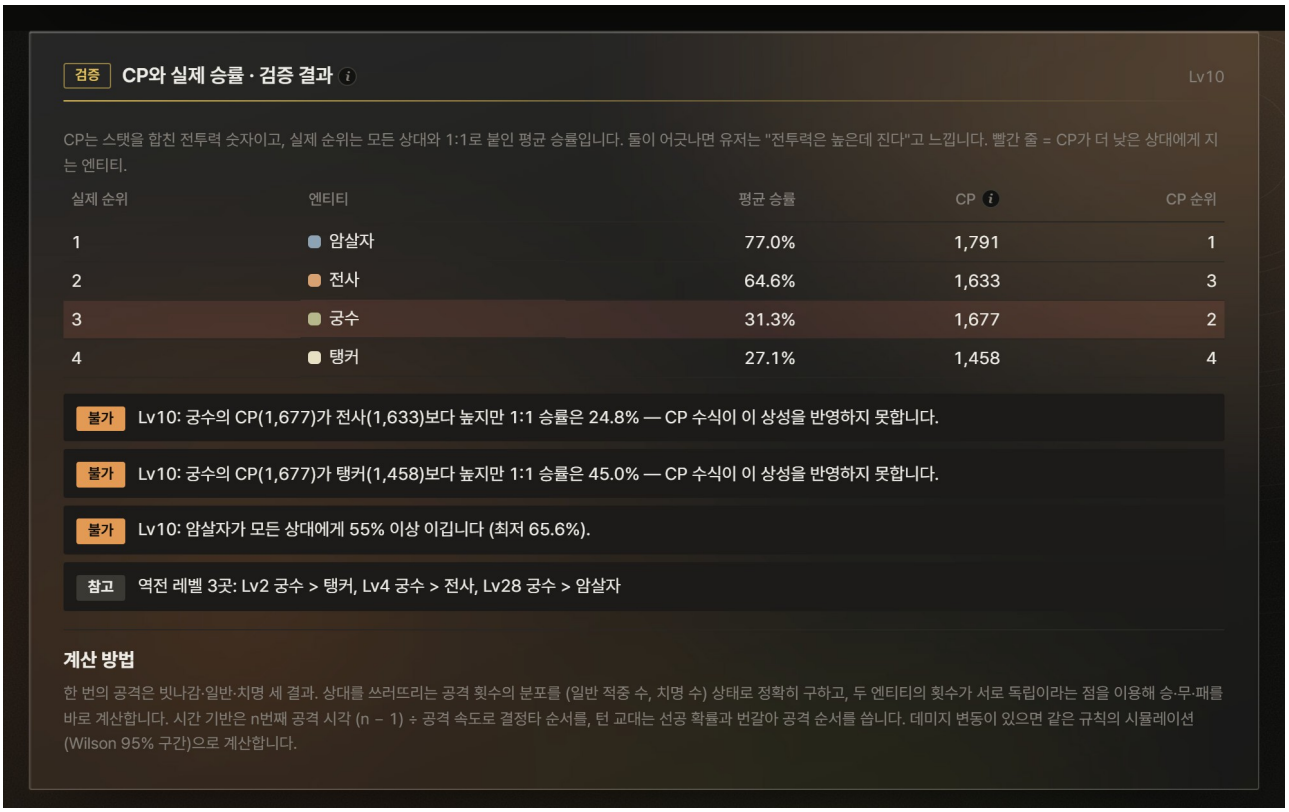


그림 39. CP와 실제 승률 · 검증 결과 — 빨간 줄 = CP가 더 낮은 상대에게 지는 엔티티

실제 순위(모든 상대와의 평균 승률)와 CP 순위를 나란히 놓고, CP가 높은데 지는 쌍을 빨간 줄과 검증 문장으로 보여 줍니다. 기본 템플릿: CP 2위 궁수(1,677)가 CP 3위 전사(1,633)에게 24.8%, CP 4위 탱커(1,458)에게 45.0%만 이깁니다. 기본 CP 수식이 공격 속도를 크게 쳐서 생긴 어긋남입니다.

> **알고리즘 스피어만 순위 상관**
>
> **어디에 쓰였나**: 핵심 지표 "CP와 실제 강함의 순위 상관", 브리핑 지표.
>
> ρ = (CP 순위, 실제 순위) 두 순위 사이의 피어슨 상관계수 — 동점은 평균 순위
>
> 동점이 없으면 $\rho = 1 - 6\,\Sigma\, d_i^2 \div \left(n(n^2 - 1) \right)$, d_i = 엔티티 i의 CP 순위 − 실제 순위
>
> 값의 크기가 아니라 **순서**가 얼마나 같은지만 봅니다. 1이면 CP 순서와 실제 순서가 완전히 같고, −1이면 정반대입니다. 기본 템플릿은 0.80 — 대체로 맞지만 한 쌍 이상이 뒤집혀 있다는 뜻입니다. 부록 A.4.3.

4.13 밸런스 도구



그림 40. 밸런스 도구 — 점수를 목표로 만드는 최소 변경과 민감도

1	**대상 · 상대 · 레벨** — 점수를 볼 엔티티와 상대(한 명 또는 모든 상대 평균), 레벨.
2	**목표 점수 (%)** — 점수 = 승 + 무승부 ÷ 2. 보통 50%. 민감도 변동폭(±5/10/20%)도 여기서 고릅니다.

3	**적용** — 그 스탯을 필요 배율만큼 바꿉니다 (기본값·성장값·표 값을 모두 같은 배율로).

- **점수를 목표로 만드는 최소 변경** — 대상의 스탯 하나만 ×k 했을 때 목표 점수에 닿는 k를, 필요한 변화가 작은 순으로 보여 줍니다. 공격 횟수가 정수라 점수는 계단 모양이므로 "목표를 처음 넘는 값"과 그때의 점수를 함께 표시합니다. 지금 점수가 목표보다 높으면 목표까지 내리는 배율(여유)이 나옵니다.
- **민감도** — 대상과 상대의 스탯을 하나씩 ±10% 바꿨을 때 대상 점수가 몇 %p 바뀌는지 토네이도로 보여 줍니다. 오른쪽 = 대상에게 유리.

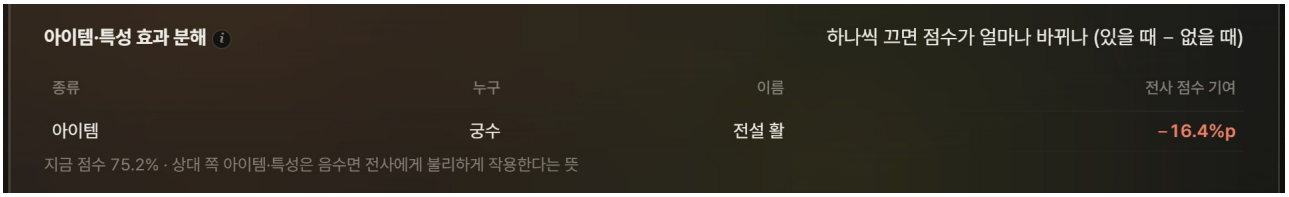


그림 41. 아이템·특성 효과 분해 — 하나씩 끄면 점수가 얼마나 바뀌나

대상에게 걸린 아이템·특성(상대가 한 명이면 상대 것도)을 하나씩 껐을 때의 점수 차이(있을 때 − 없을 때)입니다. 상대 쪽 아이템이 음수면 대상에게 불리하게 작용한다는 뜻입니다. 그림에서 궁수의 "전설 활"이 전사 점수를 16.4%p 깎습니다.

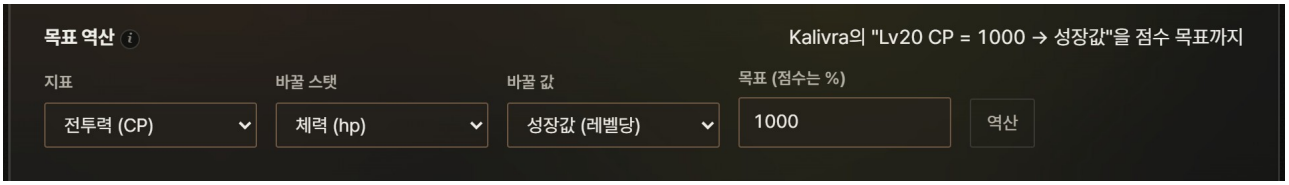


그림 42. 목표 역산 — Kalivra의 "Lv20 CP = 1000 → 성장값"을 점수 목표까지

지표(전투력 CP / 초당 기대 데미지 / 점수)와 바꿀 스탯, 바꿀 값(성장값 / 기본값 / 전체 배율), 목표를 넣고 **역산**을 누르면 목표에 맞는 값을 찾아 적용합니다. 예: "Lv20 전사 CP = 1000이 되려면 체력 성장값은?" 또는 "Lv20 전사 대 궁수 점수 50%가 되려면 공격력 성장값은?"

알고리즘 로그 이분 탐색과 같은 시드 비교

최소 변경·목표 역산: 스탯 배율 k를 키우면 점수는 단조로 변합니다. 1/64배~64배 범위에서 로그 축 이분 탐색으로 목표를 처음 넘는 k를 찾습니다(공식 계산 40회, 시뮬레이션 22회 반복). 그 범위에서 닿지 않으면 "이 스탯만으로는 닿지 않음"과 끝에서의 점수를 보여 줍니다.

시뮬레이션이 필요한 쌍: 배율마다 같은 시드(같은 난수열)로 시뮬레이션해서, 설정을 조금 바꿨을 때의 차이가 난수 잡음에 묻히지 않게 합니다(공통 난수 기법). 부록 A.1.4.

4.14 가져오기와 내보내기

버튼	내용
Kalivra 가져오기	Kalivra 프로젝트(.kal / JSON)를 읽어 엔티티·스탯·아이템·규칙·특성을 이 탭 설정으로 바꿉니다. 특성 트리거 이름을 발동 시점으로 대응시키고, 대응하지 못한 트리거·효과는 알림으로 알려 줍니다.
레벨별 스탯 CSV	엔티티 × 레벨 × 스탯 값(아이템 포함) 표.
승률 행렬 CSV	기준 레벨의 승률 행렬.
Unity 내보내기	ZIP: BattleData.json, Scripts/BattleRules.cs(규칙 수식을 C# 코드로), Scripts/BattleData.cs(ScriptableObject), Scripts/Editor/BattleDataImporter.cs, README.txt
Unreal 내보내기	ZIP: DT_BattleEntities.csv/json, DT_BattleItems.csv/json(DataTable 행 배열), Source/BattleRows.h(행 구조체), Source/BattleRules.h(규칙 수식을 C++로), README.txt

주의 엔진 내보내기는 출발점입니다

내보낸 C#·C++ 코드는 수식 파서의 나무를 그대로 옮긴 것으로, 실제 엔진 프로젝트에서 컴파일해 보지 않았습니다. 프로젝트의 네임스페이스·빌드 설정에 맞게 확인한 뒤 쓰세요. ID는 문자열로 내보내 Kalivra의 정수 넘침 문제를 피했고, 특성의 트리거·효과는 모두 내보냅니다.

5. 데이터 테이블

전투 시뮬레이터(6장)가 읽는 직업·캐릭터·몬스터·스킬·효과·속성 상성·시너지 일곱 개 테이블을 편집하는 탭입니다. 테이블끼리는 ID로 참조하고, 엑셀에서 고친 CSV를 그대로 불러오며, 없는 ID 참조·중복 ID·빈 필수값·범위 밖 숫자를 칸 단위로 표시합니다.

5.1 무엇을 하는 도구인가

전투 기획 데이터는 보통 엑셀 시트 여러 장에 나뉘어 있고, 시트끼리는 ID로 이어져 있습니다(캐릭터 시트의 ClassID → 직업 시트). 시트를 고치다 보면 지운 스킬을 아직 쓰는 캐릭터, 오타 난 효과 ID, 100%를 넘는 확률이 생기는데, 엑셀만으로는 전투를 돌려 보기 전까지 잘 드러나지 않습니다. 데이터 테이블 탭은 그 시트 구조를 그대로 옮겨 와 참조와 값을 검사하고, 고친 내용이 전투 시뮬레이터의 선택 목록과 계산에 바로 반영되게 합니다.

이런 질문에 답합니다

- 엑셀에서 고친 스킬 시트에 없는 효과 ID를 참조하는 행은 없나?
- 캐릭터를 하나 추가하면 직업 기본·레벨 성장·개인 보정을 거쳐 최종 스탯이 얼마가 되나?
- 스킬 ID 이름을 바꾸면 그 스킬을 쓰던 직업·몬스터·보스 페이즈는 어떻게 되나?
- 속성 상성과 시너지 조건을 표 하나에서 관리할 수 있나?

활용 예

- 전투 데이터 검수 — 기획 엑셀을 CSV로 저장해 불러오고, 검증 결과의 빨간 칸만 고쳐 다시 내보냅니다.
- 신규 캐릭터 추가 — 캐릭터 행 하나를 복제해 수치를 바꾸면 전투 시뮬레이터 목록에 바로 나타나 기존 파티와 비교할 수 있습니다.
- 보스 설계 — 몬스터 행에 그로기 최대치와 페이즈(체력%=스킬)를 적어 페이즈 전환을 시뮬레이션합니다.

5.2 화면 구성

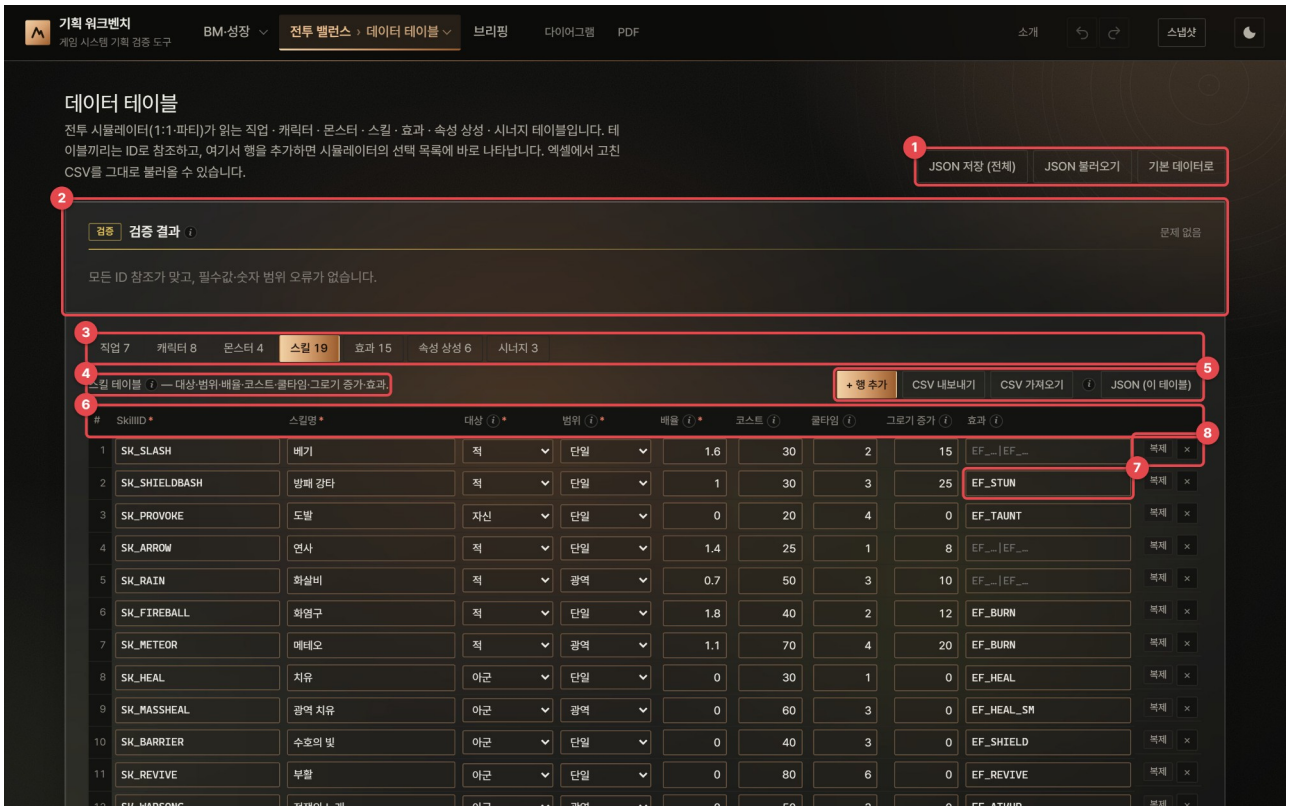


그림 43. 데이터 테이블 — 처음 화면 (스킬 테이블을 연 상태)

1	**JSON 저장 (전체) · JSON 불러오기 · 기본 데이터로** — 일곱 개 테이블 전체를 JSON 파일 하나로 저장하거나 불러옵니다. 불러오는 JSON은 전체 또는 테이블 하나 모두 됩니다. 기본 데이터로는 예시 데이터로 되돌립니다(Ctrl+Z로 취소 가능).
2	**검증 결과** — 모든 테이블의 오류·경고를 최대 12건 보여 줍니다. 항목을 누르면 그 테이블의 그 칸으로 이동합니다. 문제가 없으면 "문제 없음".
3	**테이블 탭** — 직업 7 · 캐릭터 8 · 몬스터 4 · 스킬 19 · 효과 15 · 속성 상성 6 · 시너지 3(숫자 = 행 수). 오류가 있는 테이블에는 빨간 숫자 배지가 붙습니다.
4	**테이블 설명** — 지금 테이블이 무엇을 담는지 한 줄 설명. (i)에 자세한 뜻이 있습니다.
5	**+ 행 추가 · CSV 내보내기 · CSV 가져오기 · JSON (이 테이블)** — 지금 테이블만 다룹니다. 새 행은 겹치지 않는 ID(예: SK_20)와 기본값으로 생깁니다.
6	**열 머리** — 빨간 *는 필수 열, (i)는 열 설명. 머리에 마우스를 올리면 CSV에 쓰는 열 키(예: mult)가 나옵니다.
7	**참조 칸** — 다른 테이블의 ID를 | 로 이어 적습니다(예: EF_DEFDOWN|EF_VULN). 입력하는 동안 ID 자동 완성 목록이 뜹니다. 하나만 고르는 참조(캐릭터의 ClassID·속성 등)는 목록에서 고릅니다.

8	**복제** · × — 행을 바로 아래에 복제(새 ID)하거나 삭제합니다. 삭제한 ID를 참조하던 칸은 오류로 표시됩니다.

화면 아래 테이블 사용법 카드에 최종 스탯 계산 순서, 참조 관계, 검증 기준, CSV 규칙이 요약되어 있습니다.

5.3 빠르게 시작하기

1. 기본 데이터(전사·기사·궁수·마법사·사제·음유시인·주술사 7직업, 캐릭터 8명, 몬스터 4종)를 둘러보며 열의 뜻을 (i)로 확인합니다.
2. 우리 게임의 시트가 있다면 테이블 탭을 고르고 **CSV 가져오기**로 엑셀 저장본을 불러옵니다. 머리글은 열 키(hp)나 열 이름(체력) 모두 됩니다.
3. **검증 결과**의 오류를 눌러 빨간 칸으로 이동해 고칩니다. 오류가 0이 될 때까지 반복합니다.
4. 전투 시뮬레이터(6장) 탭을 열면 추가한 캐릭터·몬스터가 선택 목록에 이미 들어 있습니다.
5. 고친 테이블은 **JSON 저장 (전체)**로 팀과 나누거나, 테이블마다 **CSV 내보내기**로 다시 엑셀에 붙입니다.

5.4 일곱 개 테이블과 ID 참조

테이블	한 행	주요 열	참조
직업	직업 하나 (ClassID)	역할군(탱커·딜러·힐러·버퍼·디버퍼), 기본 배치(전열/후열), Lv1 스탯 8종, 스탯별 레벨당 성장, 기본 스킬	→ 스킬
캐릭터	캐릭터 하나 (CharID)	ClassID, 속성, 등급, 보정 방식(합/곱), 스탯 보정, 고유 스킬	→ 직업, 속성, 스킬
몬스터	몬스터 하나 (MonsterID)	속성, 배치, 스탯 8종(레벨 없음), 스킬, 보스, 그로기 최대치, 페이즈	→ 속성, 스킬
스킬	스킬 하나 (SkillID)	대상(적/아군/자신), 범위(단일/광역), 배율, 코스트, 쿨타임, 그로기 증가, 효과	→ 효과

테이블	한 행	주요 열	참조
효과	효과 하나 (EffectID)	종류(버프·디버프·스턴·회복·보호막·지속 피해·도발·부활), 대상 스탯, 수치, 연산(합/곱), 발동 확률, 지속, 중첩	—
속성 상성	공격 속성 하나	방어 속성마다 배율(기본 1). 행을 더하면 열도 생김	← 캐릭터·몬스터의 속성
시너지	시너지 하나 (SynergyID)	조건(같은 속성 N명 / 역할군 조합 / 특정 캐릭터), 조건 값, 인원 N, 효과	→ 효과, 속성, 캐릭터

스탯 8종은 체력·공격력·방어력·속도·치명타 확률(%)·치명타 배율·명중·회피입니다. 여러 ID는 | 로 잇고(세미콜론·쉼표도 됨), 보스 페이즈는 "70=SK_FLAME|SK_SLAM; 30=SK_ERUPT|SK_FLAME|SK_ENRAGE"처럼 "체력%=스킬 목록"을 ; 로 나눠 적습니다. 체력이 70% 이하가 되면 둘째 목록, 30% 이하가 되면 셋째 목록으로 스킬이 바뀝니다.



그림 44. 몬스터 테이블 — 보스(화염 골렘)의 그로기 최대치 300과 페이즈 두 개

1	**스킬** — 평소에 쓰는 스킬. 화염 골렘은 내려찍기(SK_SLAM).
2	**보스** — 체크하면 선택 목록에 [보스]가 붙고, 그로기 최대치가 없으면 경고합니다.
3	**그로기 최대치** — 스킬을 맞을 때마다 쌓이는 그로기 게이지의 상한. 가득 차면 흐트러짐(6.6절).
4	**페이즈** — 체력% 기준과 그 페이즈의 스킬 목록. 페이즈가 바뀌면 쿨타임이 초기화됩니다.

ID 바꾸기

ID 칸을 고치면 다른 테이블의 참조도 함께 바뀝니다. 예를 들어 스킬 SK_SLAM을 SK_SMASH로 바꾸면 오크 전사의 스킬 칸과 화염 골렘의 스킬·페이즈 칸("70=SK_FLAME|SK_SMASH; ...")이 한 번에 바뀌고,

화면 아래에 "ID SK_SLAM → SK_SMASH (참조도 함께 바꿈)"이 뜹니다. 이미 있는 ID로 바꾸면 참조는 그대로 두고 중복 ID 오류로 표시합니다. 속성 이름을 바꾸면 상성 표의 열과 "같은 속성" 시너지의 조건 값도 함께 바뀝니다.

5.5 최종 스탯 계산 순서

캐릭터의 전투 시작 스탯은 네 단계로 정해집니다. 전투 시뮬레이터 화면의 계산 단계(6.2절 그림의 ⑤)가 단계별 값을 그대로 보여 줍니다.

수식 최종 스탯 — 아렌(전사, Lv30) 예

```
① 직업 기본 (Lv1)        = 직업 테이블의 값
② 레벨 성장          = ① + 성장 × (레벨 − 1)
③ 캐릭터 보정   합(add) = ② + 보정값
            곱(mul) = ② × (1 + 보정값 ÷ 100)
④ 장비·버프          = 전투 중 효과 (6.5절) → 스탯 하한·상한으로 자름
```

아렌은 전사(체력 900, 공격력 95, 체력 성장 60, 공격력 성장 6)에 합 보정 체력 +100·공격력 +10입니다. Lv30이면 ② 체력 900 + 60 × 29 = 2,640, 공격력 95 + 6 × 29 = 269, ③ 체력 2,740, 공격력 279가 됩니다. 곱 보정인 달리아(마법사, 공격력 +10%)는 ② 공격력 125 + 8.5 × 29 = 371.5에 ×1.1을 해 408.7입니다.

몬스터는 레벨 성장 없이 몬스터 테이블의 값을 그대로 씁니다. 치명타 확률은 0~100%, 치명타 배율은 1 이상, 체력은 1 이상으로 자릅니다.

5.6 검증 — 빨간 칸과 노란 칸

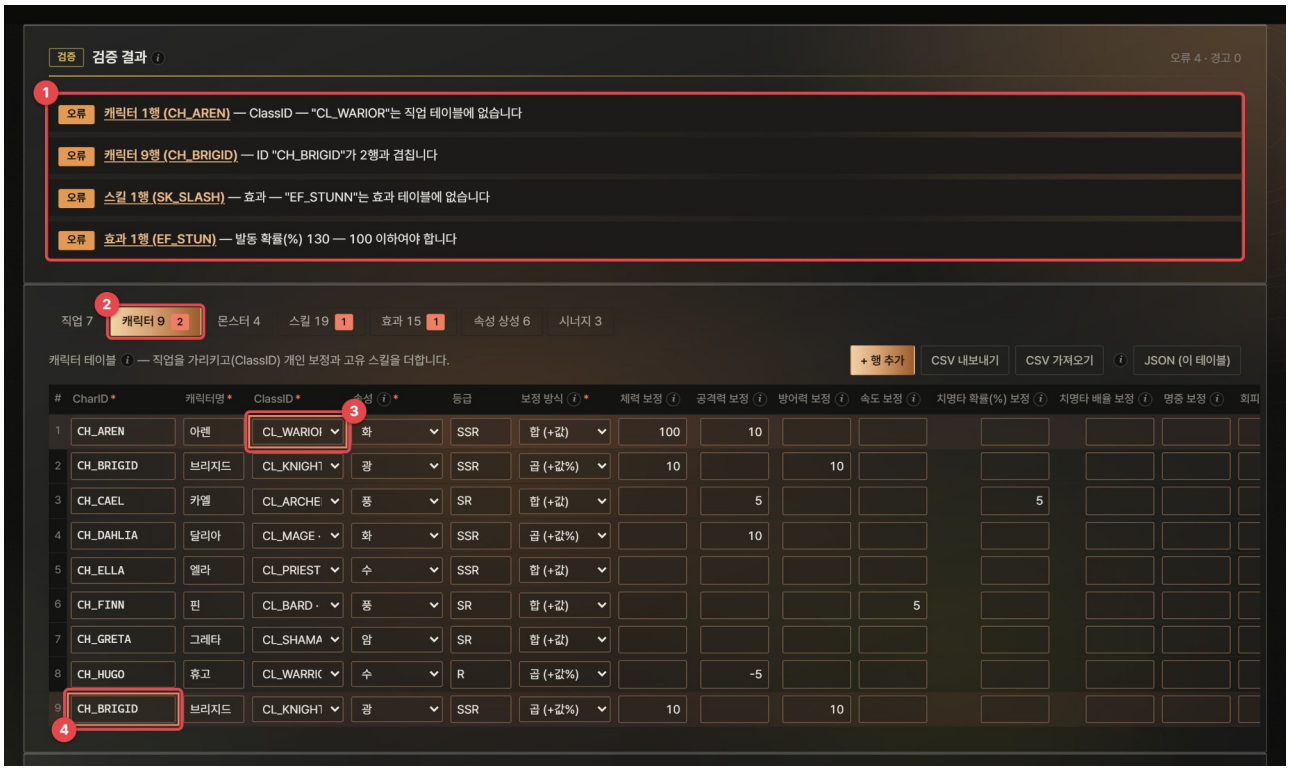


그림 45. 검증 결과 — 오류 네 가지를 일부러 넣은 예

1	**오류 목록** — 테이블·행·ID와 이유. 밑줄 친 글씨를 누르면 그 칸으로 이동합니다.
2	**테이블 배지** — 그 테이블의 오류 수.
3	**빨간 칸** — 문제가 있는 칸. 마우스를 올리면 이유가 나옵니다(예: ClassID — "CL_WARIOR"는 직업 테이블에 없습니다).
4	**중복 ID** — 같은 ID가 두 번 나오면 뒤쪽 행에 표시합니다(캐릭터 9행 CH_BRIGID가 2행과 겹침).

검사	등급	예
빈 필수값	오류	스킬명이 비었음
중복 ID	오류	CH_BRIGID가 2행과 겹침
없는 ID 참조	오류	효과 EF_STUNN이 효과 테이블에 없음 (페이즈 안의 스킬, 시너지 조건 값 포함)
숫자 아님 · 범위 밖	오류	발동 확률 130 — 100 이하여야 함, 체력 0 — 1 이상이어야 함
선택지 밖 값	오류	역할군 "서포터" — 탱커·딜러·힐러·버퍼·디버퍼 중 하나

검사	등급	예
효과 규칙	오류	버프·디버프에 대상 스탯·수치가 없음, 회복·보호막·지속 피해·부활에 수치가 없음
시너지 조건	오류	같은 속성 조건의 속성이 상성 테이블에 없음, 인원 N 없음, 없는 역할군·캐릭터
효과가 걸리지 않는 설정	경고	스턴·도발·지속 피해의 지속이 0, 그로기 최대치가 없는 보스

오류가 있어도 전투 시뮬레이터는 돌아갑니다. 다만 없는 ID를 가리키는 스킬·효과는 건너뛰고, 테이블에 없는 캐릭터를 고른 슬롯은 빠지므로 결과가 의도와 달라질 수 있습니다. 시뮬레이터 화면도 "테이블에 없는 ID입니다"라고 알립니다.

5.7 CSV와 JSON

- **CSV 내보내기** — 지금 테이블을 UTF-8 CSV로 저장합니다. 머리글은 열 키(id, name, mult …), 선택지는 한글 이름 첫 단어(적·단일·탱커), 참·거짓은 TRUE/FALSE, 여러 ID는 | 로 씁니다. 엑셀에서 바로 열립니다.
- **CSV 가져오기** — 엑셀이 저장한 CSV를 그대로 읽습니다. BOM, 따옴표 안 쉼표·줄바꿈, 쉼표·세미콜론·탭 구분자(첫 줄에서 많은 쪽)를 모두 처리합니다. 머리글은 열 키나 열 이름 모두 되고, 모르는 열은 무시한 뒤 알림으로 알려 줍니다. 숫자의 천 단위 쉼표와 끝의 %는 떼고 읽습니다.
- **속성 상성 CSV** — 첫 열이 공격 속성, 나머지 머리글이 방어 속성입니다(화, vs:화, → 화 모두 됨). 불러오는 파일의 속성 목록이 새 열이 됩니다.
- **JSON** — 전체 저장은 일곱 개 테이블을 파일 하나로, "이 테이블"은 지금 테이블만 저장합니다. 불러올 때는 파일에 들어 있는 테이블만 바꿉니다.

> **팁 엑셀과 오가며 쓰는 순서**
>
> ① 테이블마다 CSV 내보내기 → ② 엑셀에서 수정·행 추가 → ③ CSV(쉼표로 분리)로 저장 → ④ 같은 테이블에서 CSV 가져오기 → ⑤ 검증 결과 확인. 가져오기는 그 테이블의 행 전체를 바꾸므로, 다른 테이블이 참조하는 ID를 엑셀에서 지웠다면 검증 결과에 바로 나타납니다.

5.8 속성 상성



그림 46. 속성 상성 표 (전투 시뮬레이터 화면) — 행 = 공격 속성, 열 = 방어 속성

1	**상성 표** — 공격 속성(행)이 방어 속성(열)을 때릴 때의 데미지 배율. 1보다 크면 초록, 작으면 빨강입니다.

기본 데이터는 화 → 풍 → 수 → 화 순환(유리 ×1.3, 불리 ×0.8)과 광 ↔ 암 상호 ×1.3입니다. 같은 표를 데이터 테이블의 "속성 상성" 탭과 전투 시뮬레이터 아래 카드에서 모두 고칠 수 있고, 한쪽에서 고치면 다른 쪽에도 바로 반영됩니다. 속성을 추가하면(행 추가) 모든 행에 그 속성 열이 1로 생깁니다.

5.9 저장과 공유

- 테이블은 이 브라우저에 자동 저장되고, 전투 시뮬레이터가 같은 저장본을 읽습니다. 되돌리기(Ctrl+Z)도 편집마다 한 단계씩 됩니다.
- 스냅샷·프로젝트 파일(1.6절)에 데이터 테이블이 함께 들어갑니다.
- 전투 시뮬레이터의 공유 링크·JSON·프리셋에는 테이블이 들어가지 않습니다. 팀원과 같은 결과를 보려면 테이블을 JSON으로 함께 보내세요.

6. 전투 시뮬레이터

데이터 테이블(5장)의 캐릭터·몬스터·스킬·효과·상성·시너지를 읽어 1:1 전투와 파티 전투(N:N, N:보스)를 같은 규칙으로 수백 번 돌리는 도구입니다. 턴제와 실시간을 고르고 데미지 공식·명중·치명타·버프 합산·그로기 규칙을 바꿔 가며 승률, 클리어 시간 분포, 캐릭터별 기여도를 보고, 조합을 자동으로 만들어 순위를 매깁니다.

6.1 무엇을 하는 도구인가

4장 스탯·전투는 엔티티의 스탯과 규칙 수식으로 1:1 승률을 계산합니다(대부분 공식으로 정확히). 그 안의 배틀 시뮬레이터(4.10절)도 같은 1:1 규칙을 여러 번 돌릴 뿐, 스킬은 다루지 않습니다. 하지만 실제 전투에는 쿨타임과 코스트가 있는 스킬, 확률로 걸리는 스턴·화상·방어 감소, 아군을 살리는 회복·보호막·부활, 보스의 페이즈와 그로기가 섞입니다. 이런 상호작용은 식으로 풀기 어려워, 이 도구는 테이블에 적힌 규칙대로 전투를 한 판씩 진행하는 시뮬레이션으로 계산합니다. 같은 시드면 언제나 같은 결과가 나오므로, 규칙이나 수치를 하나 바꿨을 때의 차이를 그대로 비교할 수 있습니다.

이런 질문에 답합니다

- 탱커·딜러·힐러·버퍼로 짠 5인 파티가 화염 골렘을 몇 %로 잡고, 보통 몇 턴(초)이 걸리나? 운 나쁜 1%는?
- 같은 데이터인데 턴제와 실시간 중 무엇을 고르느냐에 따라 승률이 얼마나 달라지나?
- 방어력 상수 감소형과 비율 감소형 중 어떤 데미지 공식이 우리 성장 곡선에 맞나?
- 파티에서 누가 피해를 가장 많이 넣고, 누가 빠지면 클리어가 안 되나?
- 캐릭터 7명 중 4명을 고르는 35가지 조합 중 어느 조합이 가장 강한가?

활용 예

- 보스 난이도 설계 — 그로기 최대치·페이즈 체력%·광폭화 수치를 바꾸며 목표 승률과 클리어 시간(P50·P90)에 맞춥니다.
- 신규 캐릭터 검토 — 교체 비교로 기존 파티의 한 자리를 새 캐릭터로 바꿨을 때 승률 변화를 봅니다.

- 전투 공식 결정 — 같은 테이블에서 데미지 공식 4종과 버프 합·곱연산을 바꿔 보고, 프리셋 A/B 비교로 지표 차이를 정리합니다.
- 밸런스 이슈 설명 — 전투 로그를 CSV로 내보내 "왜 이 판을 졌는지"를 클라이언트·QA와 같은 숫자로 이야기합니다.

6.2 화면 구성



그림 47. 전투 시뮬레이터 — 처음 화면 (1:1 전투, 아렌 Lv30 대 오크 전사)

1	**1:1 전투 · 파티 전투 · 조합 비교** — 전투 종류. 파티 전투와 조합 비교는 같은 적 구성을 씁니다.
2	**전투 방식** — 턴제 / 실시간, 최대 턴(또는 최대 시간), 데미지 반올림(내림·반올림·없음).
3	**데미지 공식** — 공식 4종 중 고르면 아래 상자에 공식과 인자(공·방·스킬 배율·상수)의 뜻이 나오고, 공식마다 필요한 상수 칸(방어 상수 K, 최소 데미지 %, 방어율 상한)이 바뀝니다(6.5절).
4	**A · 레벨** — 대결할 캐릭터나 몬스터와 레벨. 목록은 데이터 테이블에서 만들어지며, 레벨은 캐릭터에만 쓰입니다.

5	**계산 단계** — 최종 스탯이 ① 직업 기본 → ② 레벨 성장 → ③ 캐릭터 보정을 거쳐 정해지는 과정(5.5절). 아래 줄에 직업·역할·속성·배치·스킬이 나옵니다.
6	**핵심 지표** — 승률(패배·무승부), 평균 클리어 턴(이긴 판 기준, 중앙값), 양쪽의 턴당 피해(실시간이면 DPS)와 치명타율, 받은 피해 총량.
7	**데이터 테이블 편집** — 데이터 테이블 탭으로 이동합니다. 옆의 공유 링크·JSON 저장·불러오기는 이 탭의 규칙·구성만 담고 테이블은 담지 않습니다.

그 아래로 클리어 분포(6.7절), 기여도 분석(6.8절), 파티 전투의 교체 비교(6.9절), 데미지 계산 순서 예시(6.5절), 전투 로그(6.11절), 속성 상성(5.8절), 프리셋 비교(1.8절), 기획서 — 전투 규칙(6.12절)이 이어집니다.

6.3 빠르게 시작하기

1. **1:1 전투**에서 A에 아렌(Lv30), B에 오크 전사를 고른 기본 상태가 승률 100%, 평균 23.4턴인 것을 확인합니다.
2. 왼쪽 **전투 방식**과 **데미지 공식**을 우리 게임 규칙에 맞춥니다. 공식 옆 (i)와 아래 설명 상자가 인자의 뜻을 알려 줍니다.
3. **파티 전투**로 바꿔 아군 5명(브리지드·아렌·달리아·엘라·핀)과 적(화염 골렘)을 확인합니다. 기본 상태의 승률은 31.7%입니다.
4. 클리어 분포에서 P50·P90을 보고, 기여도 분석·교체 비교로 누가 승패를 가르는지 찾습니다.
5. 보스·스킬 수치는 **데이터 테이블 편집**으로 가서 고칩니다. 돌아오면 이미 다시 계산되어 있습니다.
6. 비교할 안은 **프리셋 비교**에 이름을 붙여 저장해 두고 A/B로 비교합니다.

6.4 규칙 설정



그림 48. 전투 규칙 (파티 전투의 왼쪽 설정) — 왼쪽 단에 이어 오른쪽 단

1	**명중 · 치명타** — 명중 공식(명중 – 회피 / 명중 ÷ (명중 + 회피) × 100 / 항상 명중), 명중 하한·상한(기본 5~100%), 치명타 계산 켜기.

2	**버프 · 디버프 합산** — 같은 스탯에 걸린 % 효과가 여러 개일 때 합연산(기본 × (1 + Σ%)) / 곱연산(기본 × Π(1 + %)). 고정값(+10) 효과는 % 계산 뒤에 더합니다.
3	**자원 (코스트)** — 시작 40, 턴(초)마다 +20, 최대 100. 스킬은 코스트만큼 자원을 씁니다.
4	**그로기 (흐트러짐)** — 게이지 증가 배율, 기본 공격 그로기(5), 흐트러짐 지속(2턴), 받는 피해 배율(×1.5) (6.6절).
5	**파티 규칙** — 아군·적의 공격 대상(전열 우선·랜덤·체력 낮은 대상 우선·도발 우선), 아군 대상 스킬 규칙, 회복 사용 기준(체력 70% 미만), 시너지 적용, 부활 스킬 사용.
6	**시뮬레이션** — 판 수(기본 300)와 랜덤 시드(1.8절).

설정	턴제	실시간
진행	라운드마다 살아 있는 전원이 속도 순으로 한 번씩 행동 (같으면 아군 먼저)	초당 행동 = 속도 ÷ 기준 속도(100). 행동 간격이 짧을수록 자주 행동
쿨타임 · 지속	행동 횟수(턴)	초
지속 피해 · 자원 회복	자기 턴 시작마다	1초마다
제한	최대 턴(100) — 넘으면 무승부	최대 시간(180초) — 넘으면 무승부
결과 단위	클리어 턴 수, 턴당 피해	클리어 시간(초), DPS

같은 테이블이라도 방식에 따라 결론이 크게 달라집니다. 기본 파티 대 화염 골렘은 턴제에서 승률 31.7%·평균 61.0턴, 실시간에서 88.3%·평균 44.9초입니다. 턴제에서는 속도가 행동 순서만 정하지만, 실시간에서는 속도가 곧 행동 횟수라 속도 101~145인 아군이 속도 95인 골렘보다 자주 움직이기 때문입니다(실시간 한 판에 핀 57.6회, 골렘 36.4회 행동). 우리 게임이 어느 쪽에 가까운지 먼저 정하세요(부록 A.6.2).

스킬을 고르는 규칙

- 스킬 목록의 앞쪽(캐릭터 고유 스킬 → 직업 스킬)부터 보고, 쿨타임이 끝났고 자원이 코스트 이상인 첫 스킬을 씁니다. 쓸 수 있는 스킬이 없으면 기본 공격(배율 1)입니다.
- 회복 스킬은 체력이 회복 사용 기준 미만인 아군이 있을 때만, 부활은 쓰러진 아군이 있을 때만 씁니다. 이미 걸려 있는 영구 버프나 도발 중인 도발은 다시 쓰지 않습니다.

- 광역 스킬은 그 진영의 살아 있는 전원에게, 대상마다 명중·치명타를 따로 굴립니다.
- 보스는 체력이 페이즈 기준 이하가 되면 그 페이즈의 스킬 목록으로 바뀌고 쿨타임이 초기화됩니다.

6.5 데미지 공식과 계산 순서

공식	식	언제 쓰나
방어력 상수 감소형 (기본)	공 × K ÷ (K + 방), K = 100	방어가 올라도 0이 되지 않고 완만하게 줄어듦. 방어 = K일 때 절반. 대부분의 RPG 기본형
단순 차감형	max(공 − 방, 공 × 최소%)	직관적이지만 방어가 공격에 가까워지면 급격히 줄어 최소 데미지 하한이 필요. 고전 JRPG형
비율 감소형	공 × 공 ÷ (공 + 방)	공격과 방어의 비율만 중요 — 둘이 같은 배율로 성장하면 데미지도 같은 배율. 인플레이션이 큰 성장형
곱연산형 (방어율)	공 × (1 − min(방, 상한) ÷ 100)	방어력 = 받는 피해 감소 %. 읽기 쉽고 상한(90%)으로 무적을 막음

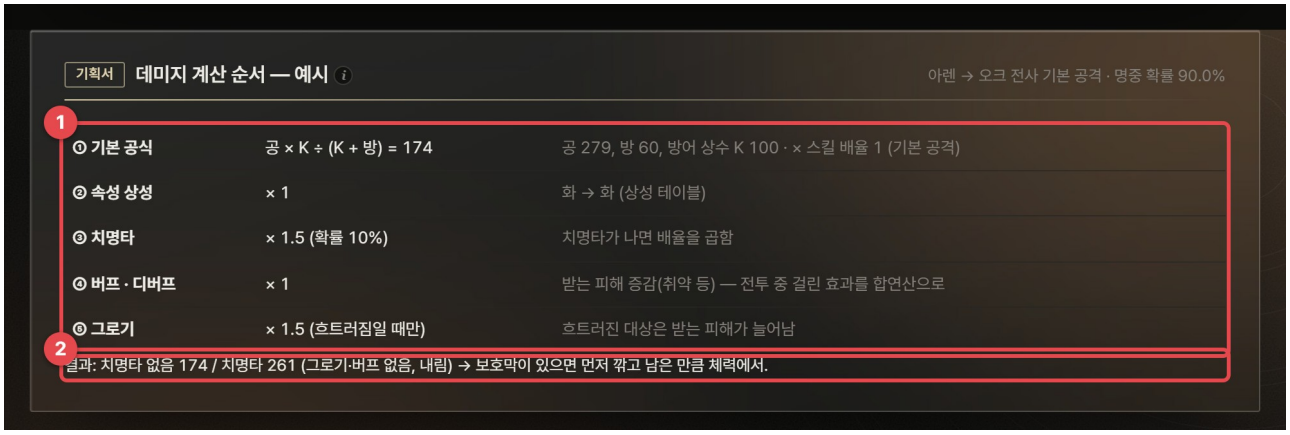


그림 49. 데미지 계산 순서 — 예시 (아렌 → 오크 전사 기본 공격)

1	**단계별 값** — ① 기본 공식 × 스킬 배율 → ② 속성 상성 → ③ 치명타 → ④ 받는 피해 증감(취약 등) → ⑤ 그로기. 지금 고른 공식과 첫 번째 아군·적으로 계산합니다.
2	**결과** — 치명타 없음 / 치명타일 때의 최종 데미지. 오른쪽 위에 명중 확률이 나옵니다. 반올림 뒤 보호막이 먼저 흡수하고 남은 만큼 체력이 줄어듭니다.

예에서 아렌(공격력 279)이 오크 전사(방어력 60)를 기본 공격하면 ① 279 × 100 ÷ 160 = 174.4, ② 화 → 화 ×1, ③ 치명타면 ×1.5이므로 내림해서 174 또는 261입니다. 명중 확률은 명중 95 − 회피 5 = 90%입니다. 같은 대결에서 공식만 바꾸면 ①이 단순 차감형 219, 비율 감소형 229.6, 곱연산형 111.6이 되어, 평균 클리어가 23.4턴에서 각각 19.0턴·18.2턴·34.3턴으로 바뀝니다.

알고리즘 한 대의 데미지 — 계산 순서와 버프 합산

어디에 쓰였나: 1:1·파티 전투의 모든 공격, 데미지 계산 순서 예시, 기획서 영역의 3절.

```
명중 확률 = 하한 ≤ (명중 − 회피) ≤ 상한          (명중 공식이 "명중 − 회피"일 때)
데미지  = 기본 공식(공, 방) × 스킬 배율 × 상성 × 치명 배율 × 받는 피해 배율 × 그로기 배율
        → 반올림(최소 1) → 보호막 먼저 → 체력
```

공·방은 버프·디버프가 걸린 실제 값입니다. 같은 스탯의 % 효과는 합연산이면 기본 × (1 + Σ%), 곱연산이면 기본 × Π(1 + %)로 모은 뒤 고정값을 더합니다. 공격력 279에 +20% 버프 두 개와 +10 고정 버프가 걸리면 합연산 279 × 1.40 + 10 = 400.6, 곱연산 279 × 1.44 + 10 = 411.8입니다. 곱연산은 버프가 많을수록 더 크게 불어나므로 버퍼가 여럿인 파티의 상한을 설계할 때 차이가 커집니다.

받는 피해 배율은 취약(디버프, +%)과 피해 감소(버프, −%)를 같은 규칙으로 모읍니다. 지속 피해는 건 쪽 공격력의 %에 대상의 받는 피해 배율만 곱합니다. 공식별 특성은 부록 A.6.1.

6.6 효과 · 그로기 · 페이즈 · 시너지

효과 종류	동작
버프 · 디버프	대상 스탯(공격력·방어력·속도·치명타·명중·회피·받는 피해)을 고정값(합) 또는 %(곱)로 올리거나 내림
스턴	지속 동안 행동 불가
회복 · 보호막	건 쪽 공격력의 %만큼. 회복·보호막 스킬은 체력 비율이 가장 낮은 아군에게
지속 피해	건 쪽 공격력의 %를 턴(초)마다. 중첩이면 겹칠 때마다 하나 더
도발	적의 공격 대상 규칙이 "도발 우선"이면 도발 중인 대상을 먼저 공격

효과 종류	동작
부활	쓰러진 아군을 최대 체력의 %로 되살림 (부활 스킬 사용 스위치)

모든 효과는 발동 확률로 굴려 걸립니다. 지속은 턴제에서 대상의 행동 횟수, 실시간에서 초이고, 0이면 전투 끝까지 갑니다. 중첩이 꺼진 효과가 다시 걸리면 수치는 그대로 두고 지속만 처음으로 돌립니다.

그로기 (흐트러짐)

그로기는 보스를 "때려서 눕히는" 구간을 만드는 규칙입니다. 스킬에 맞을 때마다 게이지에 스킬의 그로기 증가 × 게이지 증가 배율(기본 공격은 5)이 쌓이고, 몬스터 테이블의 그로기 최대치에 닿으면 흐트러짐 상태가 됩니다. 흐트러진 동안(기본 2턴) 보스는 행동하지 못하고 받는 피해가 ×1.5가 되며, 끝나면 게이지가 0으로 돌아갑니다.

기본 파티 대 화염 골렘(그로기 최대치 300) 첫 판의 로그를 보면 9·19·29·39턴에 흐트러짐이 일어나고, 15턴에 체력 70% 이하로 페이즈 2(화염 숨결·내려찍기), 37턴에 30% 이하로 페이즈 3(대분화·화염 숨결·광폭화)에 들어갑니다. 받는 피해 배율을 1.5에서 2.0으로 올리면 승률이 31.7%에서 48.7%로 오릅니다(1.8절의 프리셋 비교 예).

시너지

전투가 시작될 때 조건을 만족한 진영 전원에게 시너지 효과가 걸립니다(지속 0 = 전투 끝까지). 기본 파티는 화 속성 2명(아렌·달리아) → 화염 공명(공격력 +10%), 탱커·힐러·딜러 포함 → 삼위일체(방어력 +15%), 아렌과 엘라 함께 → 오랜 동료(치명타 확률 +10)가 모두 켜집니다. 파티 구성 아래에 지금 켜지는 시너지가 표시됩니다. 시너지 적용을 끄면 이 파티는 화염 골렘을 한 번도 잡지 못합니다(300판 중 0승).

6.7 파티 구성과 클리어 분포



그림 50. 파티 구성과 핵심 지표 — 기본 파티 5명 대 화염 골렘

1	**아군 · 인원** — 1~5명. 인원을 줄이면 아래 슬롯부터 빠집니다.
2	**슬롯** — 캐릭터(또는 몬스터)·레벨·배치. 배치 "기본"은 직업 테이블의 기본 배치(전열/후열)를 씁니다. "전열 우선" 타겟팅은 살아 있는 전열부터 노립니다.
3	**시너지** — 지금 구성에서 켜지는 시너지 이름.
4	**적** — N:N이면 여러 명, N:보스면 보스 1명. 몬스터 대신 캐릭터를 적으로 넣어 PvP처럼 볼 수도 있습니다.
5	**핵심 지표** — 아군 승률, 평균 클리어 턴(이긴 판 기준), 생존 인원(끝났을 때 아군 평균), 파티 총 턴당 피해, 받은 피해 총량과 회복량.

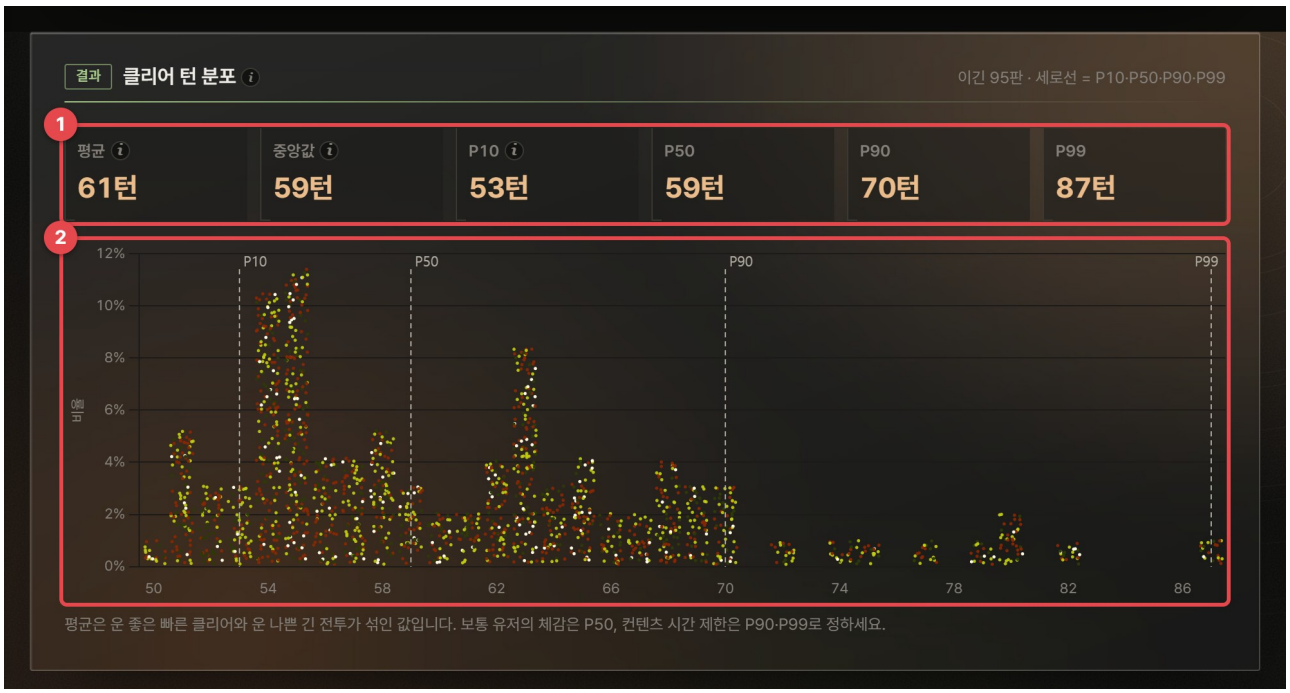


그림 51. 클리어 턴 분포 — 이긴 95판, 세로선 = P10·P50·P90·P99

1	**평균 · 중앙값 · P10 · P50 · P90 · P99** — 이긴 판만 모아 계산합니다(1.8절).
2	**히스토그램**(점묘) — 턴(초)별 비율(점의 양)과 백분위 세로선.

기본 파티는 300판 중 95판을 이기고, 이긴 판의 클리어는 평균 61턴, P10 53턴, 중앙값 59턴, P90 70턴, P99 87턴입니다. 보통 유저의 체감은 P50(59턴), 콘텐츠 제한 시간은 P90·P99로 정합니다. 예를 들어 제한을 70턴으로 두면 이긴 판의 약 10%가 시간 초과로 바뀝니다.

6.8 기여도 분석



진영	이름	역할	가한 피해	받은 피해	회복·보호막	버프·디버프 기여	생존 턴	생존율	치명타율	치치
아군	브리지드	탱커	3,693	5,993	6,309	0	66.2	31.7%	15.4%	0.10
아군	아렌	딜러	12,725	5,660	0	706	53.5	19.7%	19.8%	0.11
아군	달리아	딜러	20,071	5,102	0	640	47.3	9.0%	24.7%	0.04
아군	엘라	힐러	6,002	4,741	11,259	0	57.0	22.0%	15.6%	0.07
아군	핀	버퍼	3,495	5,300	0	2,000	48.8	6.3%	15.2%	0.01
적	화염 골렘	몬스터	26,795	45,986	0	0	66.7	68.3%	9.8%	4.11

그림 52. 기여도 분석 — 300판 평균

1	**가한 피해** — 한 판 평균 가한 피해와 막대(가장 큰 값 대비).
2	**회복·보호막** — 회복량 + 보호막 양.
3	**버프·디버프 기여** — 그 버프·디버프가 없었다면 줄었을 데미지(추정)를 효과를 건 사람에게 나눠 준 값.
4	**생존율** — 전투가 끝났을 때 살아 있던 비율. 옆의 생존 턴은 평균 생존 길이, 치명타율·처치는 판 평균.

기본 파티에서 피해는 달리아(20,071)와 아렌(12,725)이 대부분을 넣고, 엘라는 회복 11,259, 브리지드는 보호막 등 6,309를 맡습니다. 핀은 직접 피해가 3,495로 가장 적지만 전쟁의 노래(공격력 +20%)로 다른 사람의 데미지를 판당 약 2,000 늘립니다. 피해량만 보면 핀이 가장 약해 보이지만, 버프 기여를 더하면 역할이 드러납니다.

> **알고리즘 버프·디버프 기여 — 효과를 빼고 다시 계산**
>
> **어디에 쓰였나**: 기여도 분석의 "버프·디버프 기여" 열.
>
> **방법**: 공격 한 번마다 실제 데미지를 계산한 뒤, 다른 아군이 건 버프와 우리 진영이 대상에게 건 디버프만 뺀 공격력·방어력·받는 피해로 같은 공격을 다시 계산합니다(명중·치명타·상성·그로기는 그대로). 두 값의 차이가 그 효과들의 몫이고, 효과를 건 사람이 여럿이면 똑같이 나눕니다.
>
> **한계**: 효과 때문에 적이 일찍 쓰러져 생긴 이득(전투가 짧아짐, 덜 맞음)은 세지 않으므로 실제보다 작게 나옵니다. 그 전체 효과는 교체 비교(빼 보기)로 확인합니다. 부록 A.6.3.

6.9 교체 비교

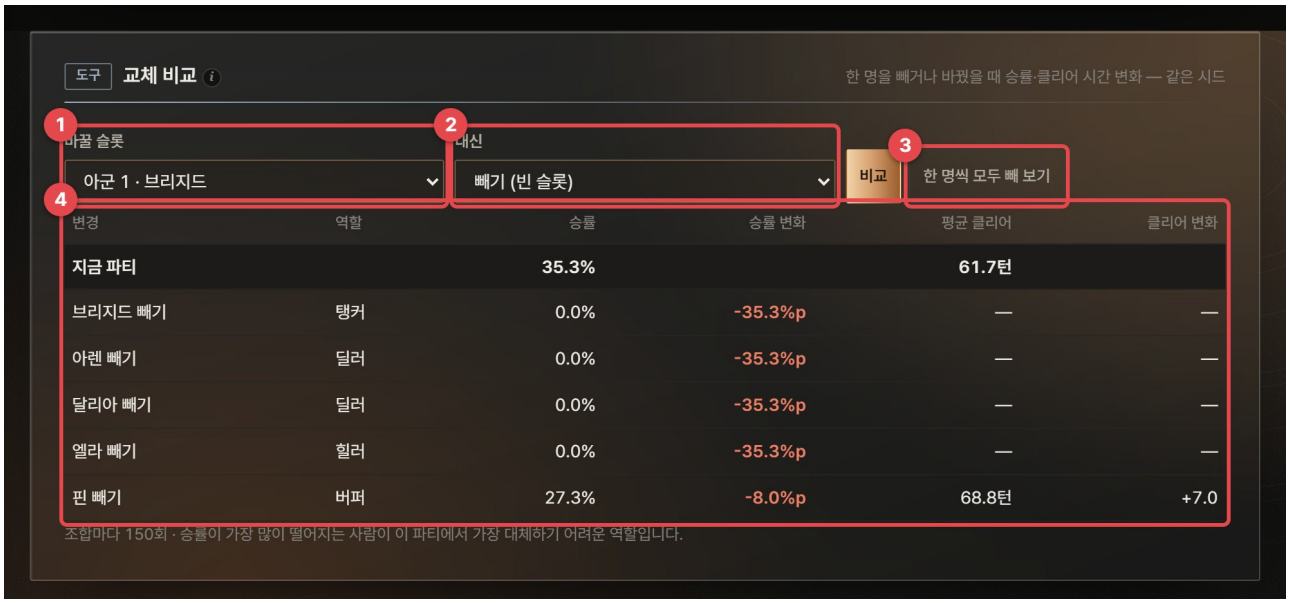


그림 53. 교체 비교 — "한 명씩 모두 빼 보기" 결과 (경우마다 150판, 같은 시드)

1	**바꿀 슬롯** — 바꾸거나 뺄 아군 슬롯.
2	**대신** — 넣을 캐릭터, 또는 "빼기 (빈 슬롯)". 비교를 누르면 그 한 경우만 계산합니다.
3	**한 명씩 모두 빼 보기** — 아군을 한 명씩 뺀 파티를 모두 계산합니다.
4	**결과** — 지금 파티와 바꾼 파티의 승률·평균 클리어와 변화. 승률이 가장 많이 떨어지는 순서로 정렬하고, 5%p 넘게 바뀌면 색으로 표시합니다.

교체 비교는 시뮬레이션 판 수의 절반(최소 50판)을 모든 경우에 같은 시드로 돌립니다. 기본 파티(150판 기준 승률 35.3% — 판 수가 절반이라 300판의 31.7%와 조금 다릅니다)에서 브리지드·아렌·달리아·엘라 중 누구를 빼도 승률이 0%가 되고, 핀을 빼면 27.3%(−8.0%p)로 떨어지면서 평균 클리어가 7턴 늘어납니다. 탱커·딜러·힐러는 하나도 빠질 수 없는 구성이고, 버퍼는 승률과 클리어 속도를 올리는 역할이라는 뜻입니다. 승률이 가장 많이 떨어지는 사람이 이 파티에서 가장 대체하기 어려운 역할입니다.

6.10 조합 비교



그림 54. 조합 비교 — 캐릭터 7명 중 4명 자동 조합 35개의 순위 (조합마다 80판)

1	**등록한 조합** — 파티 전투의 지금 파티를 "+ 지금 파티 등록"으로 이름 붙여 쌓고, "등록한 조합 비교"로 한꺼번에 돌립니다.
2	**자동 조합 풀** — 조합에 넣을 캐릭터를 고릅니다. 데이터 테이블의 캐릭터가 모두 나옵니다.
3	**슬롯 수 · 레벨 · 상위 N · 조합 수 상한 · 조합당 횟수** — 몇 명 파티인지, 모든 캐릭터의 레벨, 표에 보일 상위 개수, 계산할 최대 조합 수(기본 60), 조합마다 판 수(기본 80).
4	**가능한 조합 수 · 자동 생성해서 순위 보기** — 풀에서 슬롯 수만큼 고르는 조합 수(7명 중 4명 = 35개)와 총 전투 판 수. 상한을 넘으면 앞에서부터 상한까지만 계산한다고 알려 줍니다.

5	**순위 표** — 승률이 높은 순(같으면 평균 클리어가 짧은 순). 생존 인원·파티 턴당 피해·받은 피해를 함께 봅니다.
6	**열기** — 그 조합을 파티 전투로 열어 자세히 봅니다.
7	**결과 CSV** — 계산한 모든 조합의 결과를 CSV로 저장합니다.

적은 파티 전투 탭의 적 구성을 쓰고, 모든 조합에 같은 시드를 써서 운 차이 없이 비교합니다. 기본 데이터에서 화염 골렘을 잡을 수 있는 4인 조합은 아렌·브리지드·달리아·엘라(승률 28.7%, 평균 66.7턴) 하나뿐이고, 나머지 34개는 0%입니다. 탱커·딜러 둘·힐러가 아니면 이 보스를 넘을 수 없다는 뜻이라, 4인 콘텐츠로 쓰려면 보스 수치를 낮출지, 다른 역할 조합에도 길을 열어 줄지 판단하는 근거가 됩니다.

6.11 전투 로그

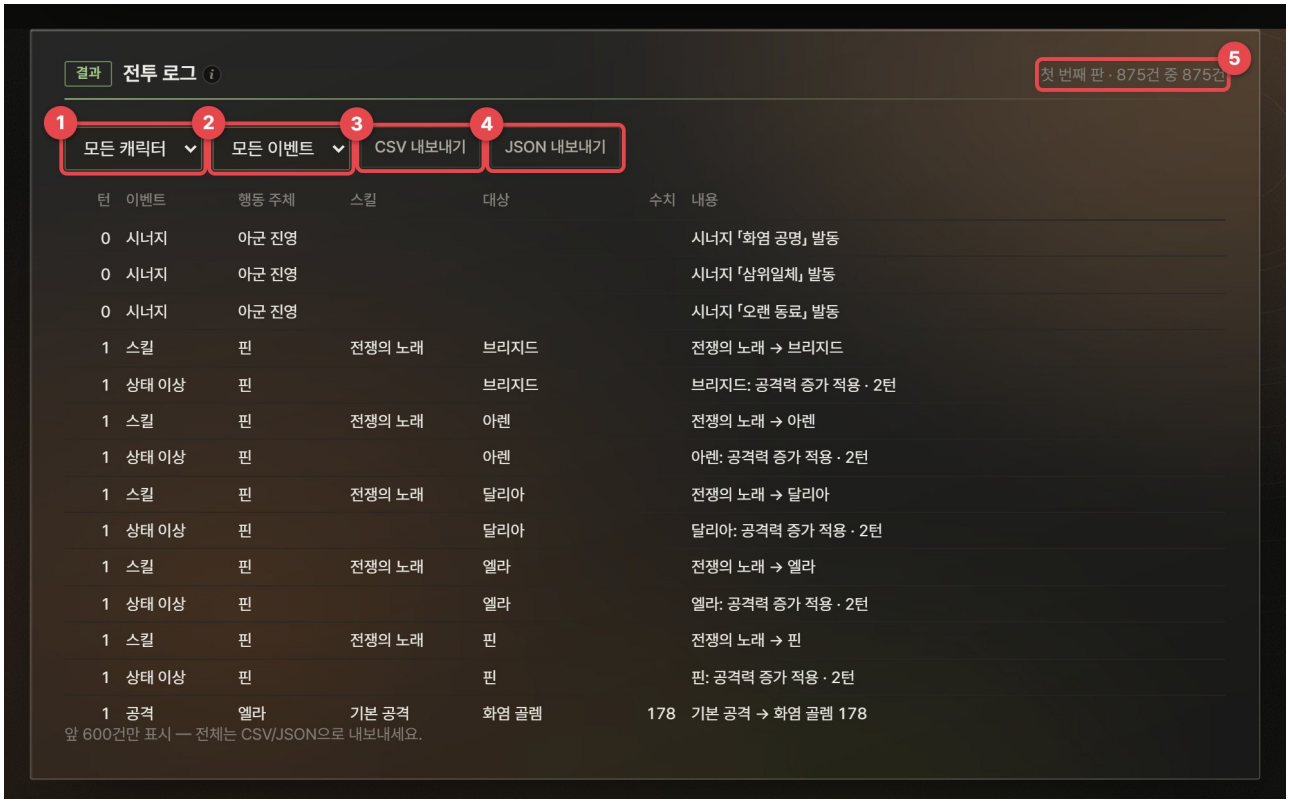


그림 55. 전투 로그 — 파티 전투 첫 번째 판 (875건)

1	**캐릭터 필터** — 그 캐릭터가 행동 주체이거나 대상인 줄만 봅니다.

2	**이벤트 필터** — 공격·빗나감·회복·스킬·상태 이상·해제·쓰러짐·지속 피해·페이즈·흐트러짐·부활·행동 불가·시너지 중 하나만 봅니다.
3	**CSV 내보내기** — 필터가 걸린 줄을 CSV(턴/초, 이벤트, 행동 주체, 스킬, 대상, 수치, 치명타, 내용)로 저장합니다.
4	**JSON 내보내기** — 같은 내용을 JSON으로 저장합니다.
5	**건수** — 첫 번째 판의 전체 건수와 필터 뒤 건수. 화면에는 앞 600건까지 나오고, 전체는 내보내기로 받습니다.

로그는 시뮬레이션 첫 번째 판 하나를 기록합니다. 숫자가 이상할 때 "어느 턴에 누가 무엇으로 얼마를 넣었나"를 그대로 확인하는 용도입니다. 이벤트 필터에서 "페이즈"나 "흐트러짐"을 고르면 보스 패턴의 전환 시점만 모아 볼 수 있습니다.

6.12 기획서 — 전투 규칙



그림 56. 기획서 — 전투 규칙 (앞부분)

1	**지금 설정으로 만든 설명** — 규칙이나 전투 종류를 바꾸면 문장도 함께 바뀝니다.

기획서 영역은 지금 고른 규칙을 기획서 문장으로 정리합니다: 1. 최종 스탯 계산 순서, 2. 데미지 공식(인자와 상수 값, 언제 쓰나, 다른 공식), 3. 한 대의 데미지 계산 순서, 4. 속성 상성(1이 아닌 배율 목록), 5. 전투 방식(진행 순서와 제한, 스킬·자원 규칙), 6. 효과, 7. 그로기, 파티 전투에서는 8. 타겟팅, 9. 시너지(테이블의 시너지 목록), 10. 기여도 계산 방식까지 이어집니다.

주의 시뮬레이터의 결과는 "이 규칙에서"의 결과입니다

스킬 선택(앞쪽부터, 쓸 수 있으면 씀), 타겟팅, 회복 기준 같은 AI 규칙은 단순화한 것입니다. 실제 게임의 AI가 스킬을 아끼거나 대상을 다르게 고르면 승률이 달라질 수 있습니다. 결과는 "같은 AI 규칙에서 수치·구성을 바꿨을 때의 차이"를 비교하는 데 가장 믿을 만합니다. 300판 승률의 95% 범위는 ±5%p 안팎이므로 작은 차이는 판 수를 늘려 다시 확인하세요.

7. 패턴 프레임 검증기

보스 연계의 선딜·판정·후딜과 플레이어의 회피 무적 프레임을 넣으면, 타격마다 회피 윈도우가 몇 프레임인지, 보고 반응해서 피할 수 있는지(아니면 외워야 하는지), 반격할 틈이 있는지를 판정하는 도구입니다.

7.1 무엇을 하는 도구인가

액션 전투 기획서의 "이 연계는 회피 윈도우가 3프레임" 같은 판단을 숫자로 검증합니다. 단독으로는 넉넉한 타격도 연계 중에는 직전 회피의 후딜 때문에 윈도우가 줄어드는데, 이 도구는 그 줄어든 윈도우까지 계산합니다. 결과는 전투 기획서의 기술 일람 양식 그대로 PowerPoint로 내보냅니다.

이런 질문에 답합니다

- 이 타격은 보고 반응해서 피할 수 있나, 외워야 하나? 몇 프레임 늦나?
- 연계 중 직전 회피의 후딜 때문에 다음 타격을 못 피하는 곳은?
- 판정이 무적보다 길어 구르기로는 피할 수 없는 공격이 있나?
- 반격은 어디서 몇 번 넣을 수 있나?
- 선딜을 몇 프레임 늘리면 반응 가능한 패턴이 되나?

활용 예

- **보스 패턴 설계 리뷰** — 애니메이션 작업 전에 프레임 수치만으로 난이도 등급(여유·빠듯·가혹·불가)을 확인합니다.
- **난이도 조정** — "입문 보스"는 모든 타격이 반응 가능하도록, "고난도 보스"는 일부 타격만 암기 패턴이 되도록 맞춥니다.
- **기획서 작성** — PPT 내보내기로 공격별 프레임 표와 타임라인을 바로 문서에 넣습니다.

7.2 화면 구성

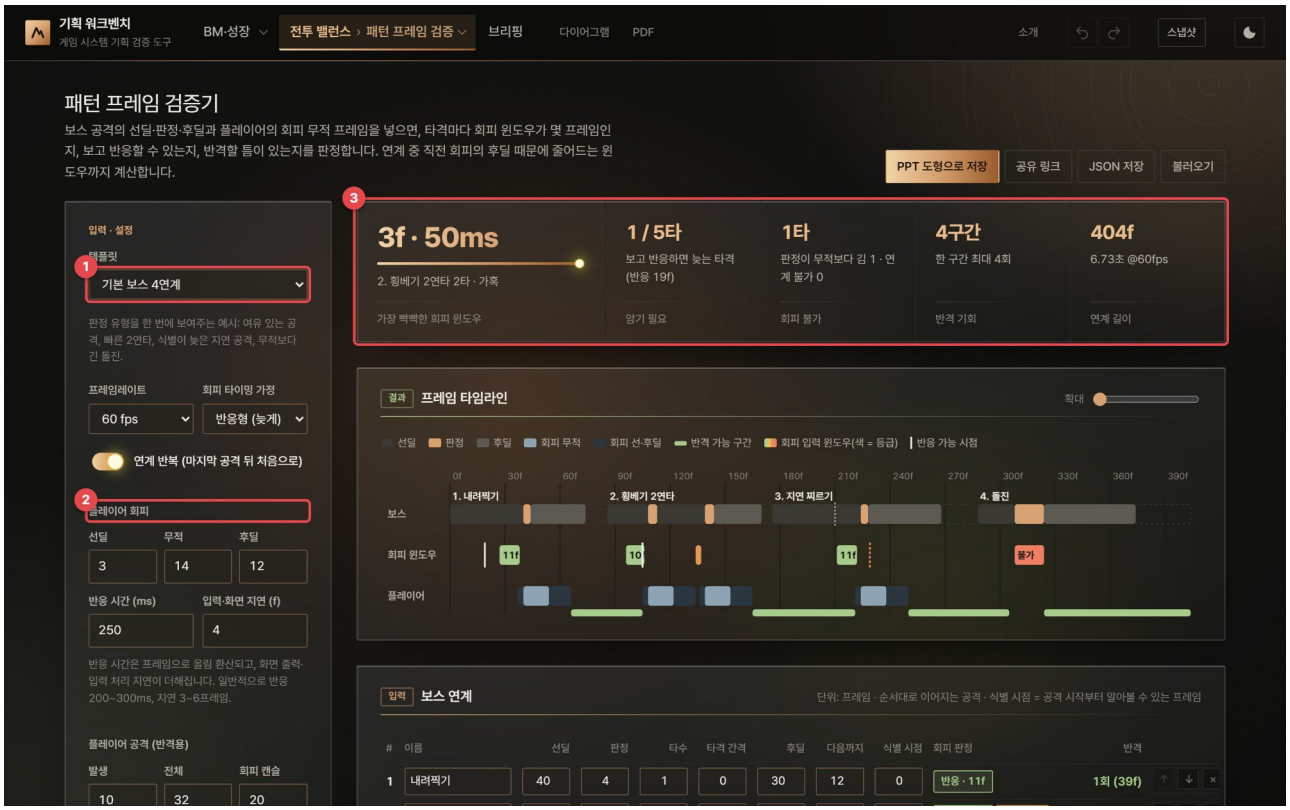


그림 57. 패턴 프레임 검증기 — 처음 화면

1	**템플릿** — 기본 보스 4연계, 빠른 연계(선입력 강제), 입문 보스(여유), 빈 패턴. 아래에서 프레임레이트(30/60/120fps), 회피 타이밍 가정(반응형 / 최적), 연계 반복 여부를 정합니다.
2	**플레이어 회피** — 회피 선딜·무적·후딜(프레임), 반응 시간(ms), 입력·화면 지연(f). 아래에 **플레이어 공격(반격용)**(발생·전체·회피 캔슬)과 **판정 기준**(여유 ≥ f, 빠듯 ≥ f)이 이어집니다.
3	**핵심 지표** — 가장 빡빡한 회피 윈도우(f·ms), 암기 필요 타격 수, 회피 불가 타격 수, 반격 기회, 연계 길이.

7.3 빠르게 시작하기

1. 템플릿을 고르고 프레임레이트를 맞춥니다.
2. **플레이어 회피**의 선딜·무적·후딜을 실제 캐릭터 값으로 넣습니다. 반응 시간은 보통 200~300ms, 입력·화면 지연은 3~6프레임입니다.
3. **보스 연계** 표에 공격을 순서대로 넣습니다 (선딜·판정·타수·타격 간격·후딜·다음까지·식별 시점).

4. **프레임 타임라인**과 표의 **회피 판정** 열, **검증 결과**에서 문제가 되는 타격과 수정 제안을 봅니다.
5. 필요하면 **PPT 도형으로 저장**으로 기술 일람 문서를 만듭니다.

7.4 프레임 타임라인



그림 58. 프레임 타임라인 — 위: 보스, 가운데: 회피 윈도우, 아래: 플레이어

1	**확대** — 타임라인을 가로로 늘려 프레임 단위로 봅니다.

보스 줄에는 선딜·판정·후딜(색은 그래프 위 범례 참고), 회피 윈도우 줄에는 각 타격을 피하려면 회피를 눌러야 하는 입력 구간(색 = 등급)과 반응 가능 시점, 플레이어 줄에는 회피의 선·후딜, 무적, 반격 가능 구간이 표시됩니다. 막대 위 숫자(예: 11f)는 그 타격의 회피 윈도우 길이입니다.

7.5 보스 연계 표와 판정



그림 59. 보스 연계 — 순서대로 이어지는 공격

1	**선딜** — 공격 시작부터 판정이 나오기까지의 프레임.
2	**판정** — 한 타격의 판정 지속 프레임. 다만 공격은 **타수**와 **타격 간격**으로 같은 판정을 반복합니다.
3	**식별 시점** — 공격 시작부터 플레이어가 "이 공격이구나" 알아볼 수 있는 프레임 (모션이 구분되는 시점).
4	**회피 판정** — 타격마다 "반응 · 11f"(보고 피할 수 있음, 윈도우 11f), "암기 · 11f"(윈도우는 있지만 보고 반응하면 늦음), "불가 · 판정>무적" 등.
5	**반격** — 이 공격이 끝난 뒤 다음 공격을 피하기 전까지 넣을 수 있는 반격 횟수와 여유 프레임.

등급	뜻
여유	회피 윈도우 ≥ 여유 기준(f)
빠듯	빠듯 기준 ≤ 윈도우 < 여유 기준
가혹	0 < 윈도우 < 빠듯 기준
불가	윈도우 ≤ 0 — 판정이 무적보다 길어 구르기로 회피할 수 없음 (이동 회피·방어 전제인지 확인)
암기	윈도우는 있지만 식별 시점 + 반응 시간 + 지연이 마지막 유효 입력보다 늦음 — 패턴을 외워야 함

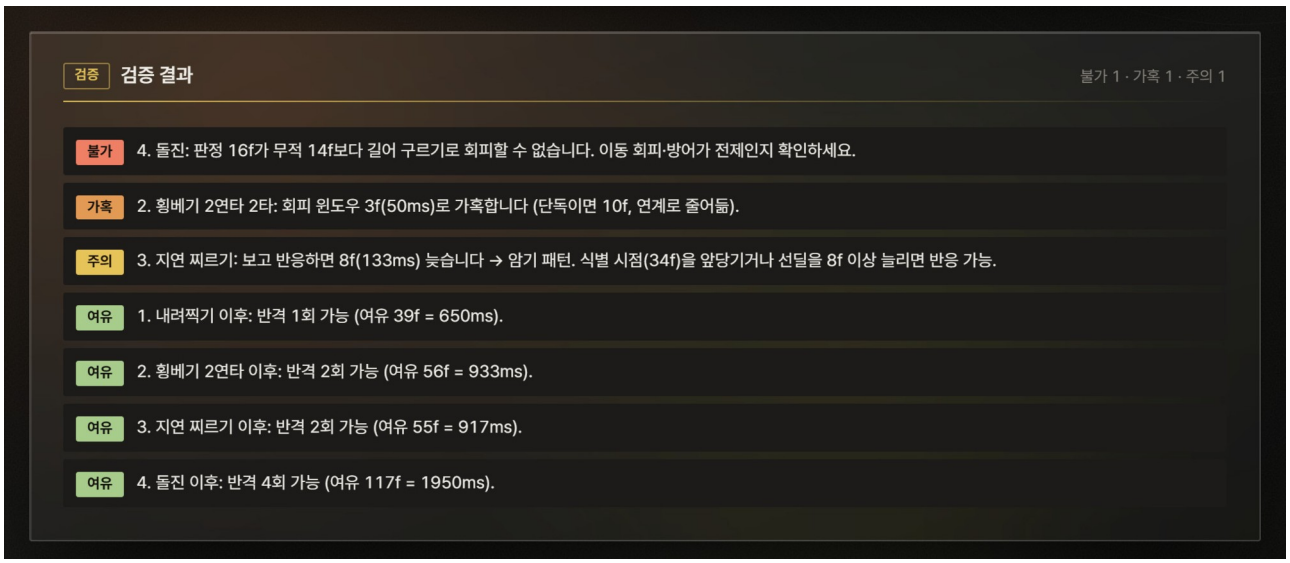


그림 60. 검증 결과 — 심각한 것부터, 수정 제안 포함

검증 문장은 "3. 지연 찌르기: 보고 반응하면 8f(133ms) 늦습니다 → 암기 패턴. 식별 시점(34f)을 앞당기거나 선딜을 8f 이상 늘리면 반응 가능."처럼 **몇 프레임 늦는지와 고치는 방법**까지 알려 줍니다.

7.6 계산 원리

알고리즘 회피 윈도우와 반응 판정

```
회피를 프레임 p에 입력: 선딜 p … p+dS−1, 무적 p+dS … p+dS+dI−1, 이후 후딜
타격 [h0, h1]을 무적으로 완전히 덮는 입력 p ∈ [ h1 − dI + 1 − dS ,  h0 − dS ]
  → 회피 윈도우 길이 = dI − A + 1    (A = 판정 프레임 수, 0 이하면 구르기로 회피 불가)
연계 중 윈도우 = [ max(lo, 직전 회피가 끝나 다시 행동 가능한 프레임), hi ]
반응 가능 ⇔ 식별 시점 + 반응 시간(프레임 올림) + 입력·화면 지연 ≤ 마지막 유효 입력 hi
```

반응 시간은 프레임레이트에 맞춰 프레임으로 올림 환산합니다(60fps에서 300ms = 18프레임). 회피 윈도우 공식은 모든 입력 프레임을 직접 대입해 본 결과(조합 1,200가지)와 같은지 테스트로 확인했습니다.

알고리즘 최적 입력 계획 — 동적 계획법

어디에 쓰였나: 회피 타이밍 가정 "최적 (선입력)", 선입력 강제 감지, 한 번에 여러 타격 회피.

방법: 타격을 순서대로 무리 [i..j]로 나눠 회피 한 번이 무리 전체를 덮게 합니다. f[i] = "타격 0..i−1을 모두 피한 뒤 다시 행동 가능한 가장 이른 프레임"으로 두면, 작을수록 이후에 유리하므로 이 값 하나만 기억하면 됩니다. 무리 [i..j]를 덮는 입력 p는 max(f[i], lo_j) ≤ p ≤ hi_i 를 만족해야 하고, 가장 이른 p를 고르면 f[j+1] = p + 회피 전체 프레임입니다. 모두 피할 수 없으면 도달 가능한 마지막 타격에서 "연계 회피 불가"로 표시하고 다음부터 다시 계획합니다.

반응형(보고 반응)으로는 못 피하지만 최적 입력으로는 피하는 타격을 "선입력 강제"로 표시합니다. 처음에는 가장 이른 입력만 고르는 탐욕법을 썼는데, 무작위 연계 1,000개를 모든 입력 조합(브루트포스)과 비교하는 테스트가 "한 번의 회피로 두 타격을 덮는 경우"를 탐욕법이 놓친다는 것을 찾아내 동적 계획법으로 바꿨습니다. 구현 노트 I.7.

7.7 PPT 내보내기

PPT 도형으로 저장은 전투 기획서의 기술 일람 양식으로 다음 슬라이드를 만듭니다: 패턴 타임라인 → 공격별 1장(공격 구간 확대, 정보 표) → 패턴별 판정 표 → 검증 결과. 모두 편집 가능한 도형이며, 애니메이션 캡처를 붙여 넣을 자리는 비워 둡니다.

8. 성장 표 템플릿 (엑셀)

레벨 곡선(base·구간별 증가율·점프·상한)과 콘텐츠 공급원, 유저 세그먼트를 정하면 세그먼트마다 일차별 레벨 진행과 레벨이 멈추는 기간을 미리 보여 주고, 수식이 살아 있는 엑셀(.xlsx)로 내보내는 도구입니다.

8.1 무엇을 하는 도구인가

레벨·경험치·성장 재료 수급은 게임마다 콘텐츠 규칙이 달라 도구 하나에 모두 담기 어렵습니다. 그래서 이 도구는 **뼈대(시트 구조·수식·서식·초기값)만 만들고**, 게임마다 다른 콘텐츠 규칙은 내보낸 엑셀의 ② 공급원 시트에 행을 더해 늘리게 합니다. 도구에서는 마일스톤(예: 코어 유저 Lv30을 14일차에)에 맞는 레벨 곡선을 자동으로 풀고, 같은 표에서 라이트·코어·헤비 유저가 각각 언제 몇 레벨인지, 레벨이 오르지 않는 날이 며칠 이어지는지를 확인합니다.

이런 질문에 답합니다

- 코어 유저가 14일차에 Lv30, 60일차에 Lv50이 되려면 레벨 곡선의 base와 증가율은 얼마인가?
- 같은 표에서 주 4일·40분 플레이하는 라이트 유저는 120일에 몇 레벨이고, 레벨이 멈추는 날이 며칠 이어지나?
- 콘텐츠 경계(Lv30·Lv50)의 30% 점프는 세그먼트마다 얼마나 체감되나?
- 사냥·일일 퀘스트·던전·주간 보스 중 어느 공급원이 성장을 끌고 가나? 유저 유형마다 다른가?
- 새 콘텐츠를 넣으면 엑셀에서 어디만 고치면 되나?

활용 예

- **성장 곡선 초안** — 마일스톤 두 개만 정하고 증가율까지 맞추기로 base·증가율 초안을 얻은 뒤, 엑셀로 내보내 팀이 함께 쓰는 성장 표로 씁니다.
- **콘텐츠 추가 검토** — 내보낸 엑셀의 ② 빈 행에 새 콘텐츠(예: 이벤트 던전)를 적으면 ④ 일차별 진행과 ⑤ 검증이 바로 다시 계산됩니다.
- **세그먼트 체감 점검** — 라이트 유저의 최장 무성장 기간이 길면 고정 보상과 현재 필요량에 비례하는 보상(% 방식) 중 무엇을 늘릴지 정합니다.

8.2 화면 구성



그림 61. 성장 표 템플릿 — 처음 화면 (MMO형 프리셋)

1	**템플릿** — MMO형(사냥 + 일일·주간 콘텐츠), 방치형(시간당 누적 + 주간 보상), 빈 설계. 아래 회색 글은 그 템플릿의 규칙과 맞춰 둔 마일스톤 설명입니다.
2	**기간 · 최대 레벨** — 일차별로 계산할 기간(일)과 최대 레벨. 최대 레벨은 엑셀 ① 시트의 행 수가 됩니다.
3	**① 레벨 곡선** — base(Lv1→2 필요량), 점프 배율, 필요량 상한(0 = 없음), 증가율 구간(시작 레벨 · 증가율 %), 점프 레벨. 아래로 레벨별 기준 획득량(표 방식)과 마일스톤이 이어집니다.
4	**핵심 지표** — 기준 세그먼트의 마일스톤 도달일(목표 일차와 비교), 세그먼트별 최장 무성장 기간, 기간 마지막 날의 끝 레벨.
5	**엑셀(.xlsx) 내보내기** — 지금 설정을 ① ~ ⑤ 시트가 수식으로 채워진 엑셀 파일로 저장합니다. 옆의 공유 링크 · JSON 저장 · 불러오기는 다른 탭과 같습니다.

오른쪽에는 위에서부터 레벨 진행 미리보기와 레벨당 일수 그래프, ② 공급원 · ③ 세그먼트 표, 검증 결과, ① 레벨별 필요량 표(접어 둠), 내보낸 엑셀의 구성 설명이 이어집니다. 표의 초록 칸(비중 · 끝 레벨 · 최장 무성장)은 계산 결과입니다.

8.3 빠르게 시작하기

1. **템플릿**에서 장르에 가까운 프리셋을 고르고 기간과 최대 레벨을 맞춥니다.
2. ② **공급원** 표에 게임의 주요 콘텐츠를 한 행씩 넣습니다 — 주기 · 해금 · 획득 방식을 고르고 획득값을 적습니다.
3. ③ **세그먼트** 표에 유저 유형별 하루 플레이 시간과 요일별 접속을 넣습니다.
4. **마일스톤**에 기준 세그먼트와 레벨 · 목표 일차 두 개를 넣고 **base 맞추기** 또는 **증가율까지 맞추기**를 누릅니다.
5. 레벨 진행 그래프와 검증 결과에서 세그먼트별 도달일과 최장 무성장 기간을 확인합니다.
6. **엑셀(.xlsx) 내보내기**로 파일을 받아 팀 성장 표로 쓰고, 콘텐츠가 늘면 ② 시트의 빈 행에 추가합니다(8.9절).

8.4 ① 레벨 곡선

필요량(L)은 레벨 L에서 L+1로 오르는 데 필요한 경험치(또는 성장 재화)입니다. 앞 레벨의 필요량에 그 레벨이 속한 구간의 증가율을 곱하고, 점프 레벨이면 점프 배율을 한 번 더 곱합니다.

수식 **레벨 곡선**

```
원래 값(1) = base
원래 값(L) = 원래 값(L-1) × (1 + 증가율(L) ÷ 100) × (L이 점프 레벨이면 J, 아니면 1)
증가율(L) = 시작 레벨 ≤ L 인 마지막 구간의 증가율
필요량(L) = round(원래 값(L)),  상한이 있으면 min(상한, 필요량(L))
누적(1) = 0,  누적(L) = 누적(L-1) + 필요량(L-1)     ← 레벨 L에 닿는 데 필요한 총량
```

예 — MMO형 프리셋(base 140, Lv1~40 증가율 14.96%, Lv41~ 8.55%, Lv30 · Lv50 점프 × 1.3): 필요량은 Lv1 140, Lv2 161, Lv10 491, Lv29 6,941이고, Lv30에서 10,374(× 1.1496 × 1.3)로 뜁니다. Lv50은 123,491, Lv60에 닿는 데 필요한 누적은 2,720,885입니다.

증가율 하나(단일 등비)로 길게 가면 후반 필요량이 폭주하기 쉽습니다. 구간을 나눠 후반 증가율을 낮추거나 상한을 두고, 콘텐츠 경계에서만 한 번 크게 오르는 **"완만한 경사 + 점프"**로 설계하는 것이 MMO에 잘 맞습니다. 검증 결과는 기준 세그먼트의 마지막 10레벨 레벨당 일수가 바로 앞 10레벨의 2배 이상이면 "후반 폭주"로 경고합니다.

레벨별 기준 획득량(표 방식)은 "표" 방식 공급원이 곱해 쓰는 레벨별 값입니다: 기준 획득량(L) = round(Lv1 값 × (1 + 레벨당 증가율 ÷ 100)^(L−1)). MMO형에서는 사냥 시간당 경험치로 쓰여 Lv1 300 → Lv30 1,626 → Lv60 9,336입니다.

8.5 ② 공급원 — 한 행 = 콘텐츠 하나



그림 62. ② 공급원 — MMO형 프리셋의 다섯 콘텐츠

1	**주기** — 매일(접속한 날마다), 매주(7 · 14 · 21…일차에, 그 주에 한 번이라도 접속했으면), 1회(해금 조건을 처음 만족한 날 한 번).
2	**해금 기준 · 해금 값** — 일차(그 일차부터) 또는 레벨(그날 시작 레벨이 값 이상일 때부터). 1회 + 레벨 해금은 시작 레벨이 처음으로 그 레벨 이상이 된 날 받습니다.
3	**획득 방식 · 획득값** — 고정: 값 그대로 / 표: 값 × 레벨별 기준 획득량 / %: 값% × 현재 레벨 필요량.
4	**시간 비례 · 보정 배율** — 시간 비례를 켜면 × 하루 플레이 분 ÷ 60. 사냥처럼 플레이 시간에 비례하는 콘텐츠에 씁니다. 보정 배율은 이벤트 · 버프 같은 배수입니다.
5	**세그먼트별 횟수** — 세그먼트마다 하루(또는 한 주) 몇 번 하는지. 비우면 1. 예: 일일 던전은 라이트 1회 · 코어 2회 · 헤비 3회.
6	**비중** — 기준 세그먼트의 기간 총공급 중 이 공급원이 차지하는 몫.
7	**+ 공급원 추가** — 행을 더합니다(100행까지). 내보낸 엑셀에서는 ② 시트의 빈 행에 적으면 됩니다.

수식 하루 공급

```
공급(d) = Σ 공급원 [그날 받나] × [획득량] × [시간] × 보정 배율 × 세그먼트 횟수
[그날 받나]  매일: 접속 × 해금  /  매주: (d가 7의 배수) × (이번 주 접속 > 0) × 해금
             1회: 해금 조건을 처음 만족한 날만 1
[획득량]     고정: 값  /  표: 값 × 기준 획득량(시작 레벨)  /  %: 값 ÷ 100 × 필요량(시작 레벨)
[시간]       시간 비례면 하루 플레이 분 ÷ 60, 아니면 1
```

예 — 코어 유저(하루 120분, 매일 접속)의 1일차 공급은 사냥 300 × 120 ÷ 60 = 600뿐이라 Lv4까지 오릅니다. 3일차(시작 Lv7)에는 일일 퀘스트가 해금되어 사냥 852 + 퀘스트 8% × 323 = 877.8, 8일차(시작 Lv16)에는 일일 던전 2,000 × 2회가 더해져 5,528.6이 됩니다. 15일차에는 Lv30 달성 보상 30,000이 한 번 들어옵니다.

같은 공급원이라도 세그먼트마다 비중이 다릅니다. 엑셀 ⑤ 시트에서 라이트 유저는 사냥 16.7% · 주간 보스 35.0%, 헤비 유저는 사냥 59.0% · 주간 보스 4.6%입니다. **고정량 콘텐츠 비중이 큰 라이트 유저일수록 필요량 점프를 더 크게 체감**하므로, 점프를 넣을 때는 % 방식 보상을 함께 늘리는 것을 검토합니다.

8.6 ③ 세그먼트와 마일스톤

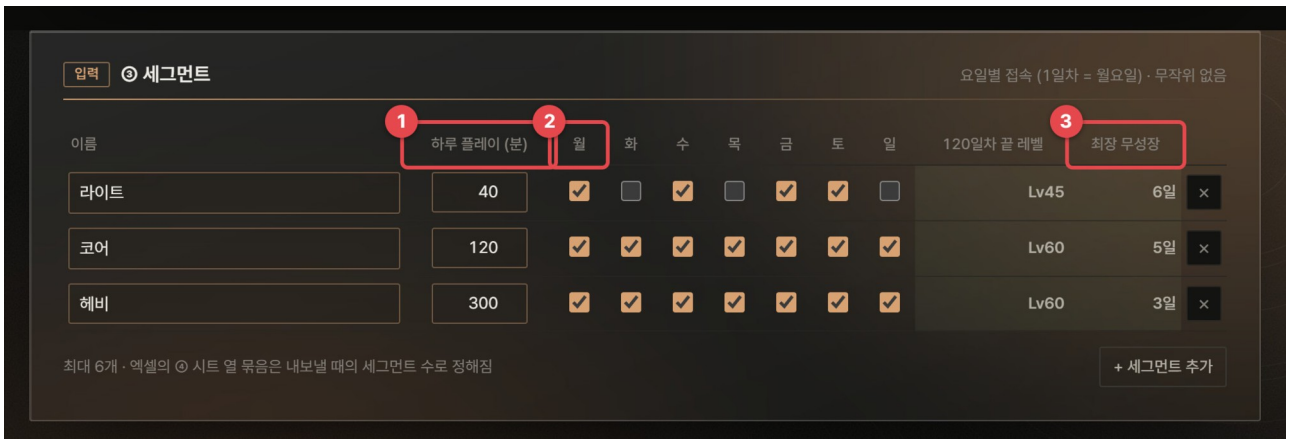


그림 63. ③ 세그먼트 — 하루 플레이 시간과 요일별 접속

1	**하루 플레이 (분)** — 시간 비례 공급원에 곱해집니다.

2	**요일별 접속** — 1일차를 월요일로 보고 요일마다 접속(체크) · 미접속을 정합니다. 무작위가 없어 같은 표면 언제나 같은 결과가 나옵니다.
3	**결과 열** — 기간 마지막 날의 끝 레벨과 최장 무성장(레벨이 오르지 않고 이어진 날의 최댓값).

MMO형 프리셋의 세그먼트는 라이트(40분, 월 · 수 · 금 · 토), 코어(120분, 매일), 헤비(300분, 매일) 입니다. 접속하지 않은 날도 무성장 연속에 들어갑니다 — 유저가 느끼는 "레벨이 안 오른 기간"이기 때문입니다.

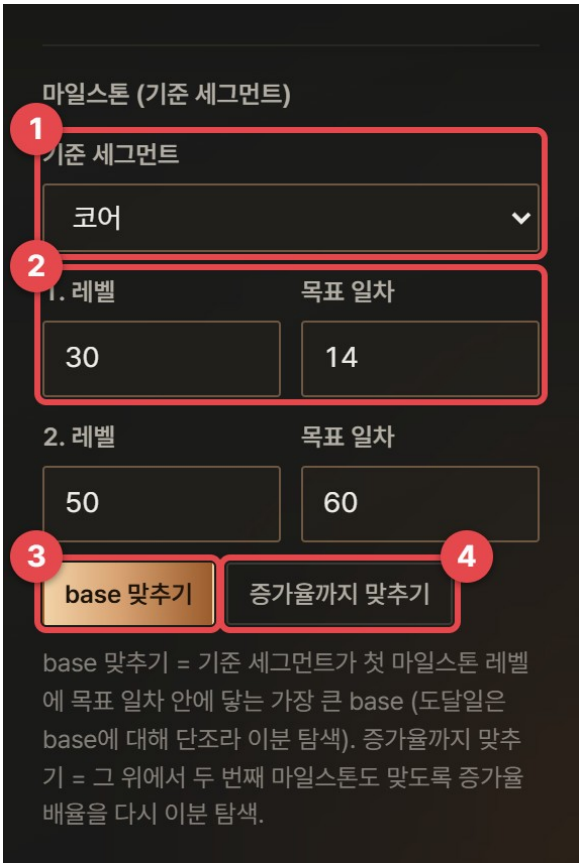


그림 64. 마일스톤과 자동 풀이

1	**기준 세그먼트** — 마일스톤과 자동 풀이의 기준이 되는 유저 유형. 브리핑 · 스냅샷의 도달일 지표도 이 세그먼트로 봅니다.
2	**마일스톤** — 레벨과 목표 일차. 첫 번째는 base 맞추기에, 두 번째는 증가율까지 맞추기에 씁니다.
3	**base 맞추기** — 증가율은 그대로 두고, 기준 세그먼트가 첫 마일스톤에 목표 일차 안에 닿는 가장 큰 base를 찾아 적용합니다.
4	**증가율까지 맞추기** — 모든 구간 증가율에 같은 배율을 곱해 두 번째 마일스톤까지 맞도록 다시 찾습니다 (base도 함께).

알고리즘 base · 증가율 자동 풀이 — 이분 탐색

어디에 쓰였나: base 맞추기, 증가율까지 맞추기.

방법: base가 커지면 모든 레벨의 필요량이 같은 비율로 커져, 기준 세그먼트의 마일스톤 도달일은 늦어지거나 그대로입니다(단조). 그래서 base 범위 1 ~ 10^{12}를 로그 가운데(√(lo × hi))로 반씩 좁히고, 정수 간격은 (lo + hi) ÷ 2로 마무리해 "목표 일차 안에 닿는 가장 큰 base"를 찾습니다. 증가율까지 맞추기는 모든 구간 증가율 × k(0 ~ 20)마다 base를 다시 풀고, 두 번째 마일스톤 도달일이 목표 이하인 가장 큰 k를 40번 이분 탐색합니다(중첩 이분 탐색).

MMO형 예: 1 ~ 1조 범위에서 13번 만에 base 140을 찾습니다(141이면 Lv30 도달이 15일차로 하루 늦음). 공급이 모두 % 방식이면 도달일이 base와 무관해 풀 수 없다고 알려 줍니다. 단계별 과정과 증가율 배율 표는 부록 A.8.1.

8.7 미리보기와 검증 결과

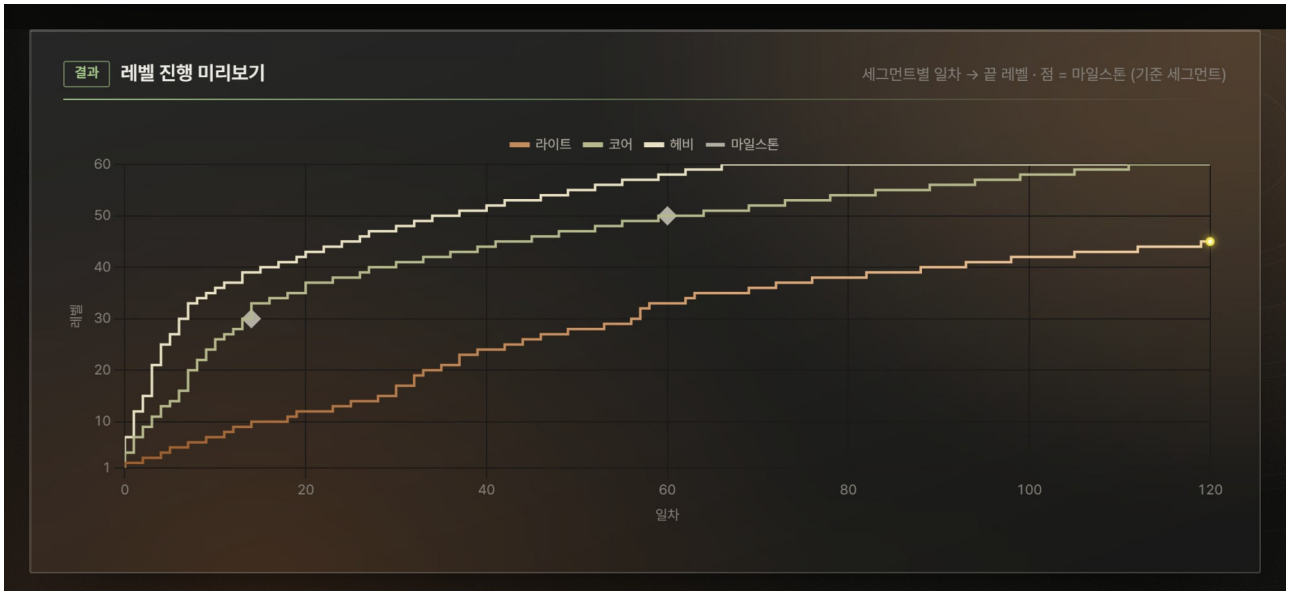


그림 65. 레벨 진행 미리보기 — 세그먼트별 일차 → 끝 레벨, 마름모 = 마일스톤

계단 모양 선은 세그먼트별로 그날 끝난 레벨입니다. MMO형에서 코어는 14일차 Lv30, 60일차 Lv50으로 두 마일스톤(마름모)을 정확히 지나고 112일차에 Lv60에 닿습니다. 헤비는 7일차 Lv30 · 35일차

Lv50 · 67일차 Lv60, 라이트는 57일차에 Lv30, 120일차에 Lv45입니다.



그림 66. 레벨당 일수 — 레벨 L에 닿는 데 걸린 날

레벨당 일수 = 도달일(L) − 도달일(L−1)입니다. 0은 하루에 여러 레벨이 오른 것이고, 값이 크게 튀는 곳이 체감이 멈추는 구간입니다. 라이트 유저는 Lv40 이후 레벨 하나에 5 ~ 7일이 걸립니다.

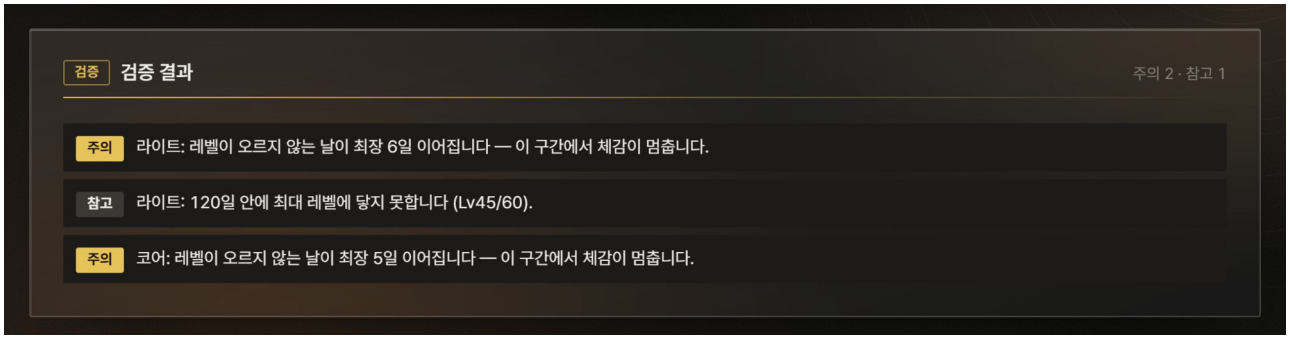


그림 67. 검증 결과 — MMO형 프리셋

검증 문장	등급	조건
마일스톤 미도달	심각	기준 세그먼트가 기간 안에 마일스톤 레벨에 닿지 못함
마일스톤 어긋남	주의	실제 도달일이 목표 일차와 max(2일, 목표의 10%) 이상 차이 — base 맞추기로 다시 풀 수 있음
무성장	주의	레벨이 오르지 않는 날이 5일 이상 이어짐
후반 폭주	주의	기준 세그먼트의 마지막 10레벨 평균 레벨당 일수가 바로 앞 10레벨의 2배 이상
최대 레벨 미도달	참고	기간 마지막 날의 끝 레벨 < 최대 레벨

MMO형에서는 라이트 유저가 Lv42에서 100 ~ 105일차, 코어 유저가 Lv55에서 85 ~ 89일차에 레벨이 멈춥니다(각각 최장 6일 · 5일). 라이트의 무성장을 줄이려면 Lv41 이후 증가율을 낮추거나, 접속한 날 받는 % 방식 보상을 늘립니다.

8.8 내보낸 엑셀의 구조

내보낸 파일은 다섯 시트로 되어 있고, **파란 칸이 입력, 나머지는 수식**입니다. 파일을 열면 엑셀이 모든 수식을 다시 계산합니다. 파일은 브라우저 안에서 만들어지며 서버로 보내지 않습니다.

시트	내용
① 레벨 곡선	입력: base, 점프 배율 J, 필요량 상한, 최대 레벨, 표 기준 획득량(Lv1 값 · 증가율), 증가율 구간(최대 10개), 점프 레벨(최대 10개). 수식: 레벨별 증가율 · 점프 · 원래 값 · 필요량 · 누적 · 표 기준 획득량.
② 공급원	한 행 = 콘텐츠 하나(100행). 주기 · 해금 기준 · 획득 방식 · 시간 비례는 드롭다운. 보정 배율 · 세그먼트 횟수는 비우면 1이고, 수식을 넣어도 됩니다.
③ 세그먼트	이름, 하루 플레이 분, 요일별 접속(1/0). 기준 세그먼트 이름은 B1.
④ 일차별 진행	세그먼트마다 열 묶음(시작 레벨 · 접속 · 이번 주 접속 · 공급 · 추가 공급 · 누적 · 끝 레벨 · 무성장 연속), 행 = 일차(0일차부터).
⑤ 검증	세그먼트 요약(끝 레벨 · 최장 무성장 · 최대 레벨 도달일 · 총공급), 마일스톤 목표일과 실제 도달일, 공급원별 총량 · 비중, 레벨별 도달일 · 레벨당 일수와 차트.

① 레벨 곡선 — 레벨별 필요량 (파란 칸 = 입력, 나머지 = 수식)

필요량(L) = L → L+1에 필요한 양. 원래 값(L) = 원래 값(L−1) × (1 + 증가율(L)/100) × (점프 레벨이면 J). 누적(L) = L에 도달하는 데 필요한 총량

기준 필요량 (base, Lv1→2)	140		구간 시작 레벨	증가율 %	점프 레벨
점프 배율 J	1.3		1	14.96	30
필요량 상한 (0 = 없음)	0		41	8.55	50
최대 레벨 (행 수는 내보낼 때 고정)	60				
표 기준 획득량 — Lv1 값	300				
표 기준 획득량 — 증가율 %	6				

증가율(L) = 시작 레벨이 L 이하인 마지막 구간 행. 점프 레벨 L에서 필요량이 J배

레벨	증가율 %	점프	원래 값	필요량 (L→L+1)	누적 (L 도달)	표 기준 획득량
1	14.96		140.0	140	0	300
2	14.96		160.9	161	140	318
3	14.96		185.0	185	301	337
4	14.96		212.7	213	486	357
5	14.96		244.5	245	699	379
6	14.96		281.1	281	944	401
7	14.96		323.2	323	1,225	426
8	14.96		371.5	371	1,548	451
9	14.96		427.1	427	1,919	478

그림 68. ① 레벨 곡선 시트 — 위는 입력(파란 칸), 아래는 레벨별 수식

② 공급원 — 한 행 = 콘텐츠 하나. 빈 행에 적으면 ④·⑤에 바로 반영됩니다 (100행까지)

주기: 매일(접속한 날) · 매주(7의 배수 일차, 그 주에 한 번이라도 접속) · 1회(해금 일차 또는 그 레벨에 처음 닿은 날). 획득 방식: 고정 = 획득값 · 표 = 획득값 × ① 표 기준 획득량(현재 레벨) · % = 획

이름	주기	해금 기준	해금 값	획득 방식	획득값	시간 비례	보정 배율	라이트 횟수	코어 횟수	헤비 횟수
사냥 (시간당)	매일	일차	1	표	1	예	1	1	1	1
일일 퀘스트	매일	레벨	5	%	8	아니오	1	1	1	1
일일 던전	매일	레벨	15	고정	2000	아니오	1	1	2	3
주간 보스	매주	레벨	30	고정	20000	아니오	1	1	1	1
Lv30 달성 보상	1회	레벨	30	고정	30000	아니오	1	1	1	1

그림 69. ② 공급원 시트 — 빈 행에 적으면 ④ · ⑤에 바로 반영

④ 일차별 진행 — 세그먼트마다 하루 한 행. 전날 행만 참조합니다 (순환 참조 없음)

시작 레벨 = 전날 끝 레벨 → 해금·획득량은 시작 레벨 기준 → 누적 → 끝 레벨 = 누적이 넘은 마지막 레벨(①의 누적 열 MATCH). 추가 공급 칸은 특이 규칙을 직접 수식으로 넣는 자리 (기본 0)

	라이트								코어							
일차	시작 레벨	접속	이번 주 접속	공급	추가 공급 (수식 가능)	누적	끝 레벨	무성장 연속	시작 레벨	접속	이번 주 접속	공급	추가 공급 (수식 가능)	누적	끝 레벨	무성장 연속
0	1	0	0	0	0	0	1	0	1	0	0	0	0	0	1	0
1	1	1	1	200.0	0	200	2	0	1	1	1	600.0	0	600	4	0
2	2	0	1	0.0	0	200	2	1	4	1	2	714.0	0	1,314	7	0
3	2	1	2	212.0	0	412	3	0	7	1	3	877.8	0	2,192	9	0
4	3	0	2	0.0	0	412	3	1	9	1	4	990.2	0	3,182	11	0
5	3	1	3	224.7	0	637	4	0	11	1	5	1,119.1	0	4,301	13	0
6	4	1	4	238.0	0	875	5	0	13	1	6	1,267.7	0	5,569	14	0
7	5	0	4	0.0	0	875	5	1	14	1	7	1,348.6	0	6,917	16	0
8	5	1	1	272.3	0	1,147	6	0	16	1	1	5,528.6	0	12,446	20	0
9	6	0	1	0.0	0	1,147	6	1	20	1	2	5,974.3	0	18,420	22	0
10	6	1	2	289.8	0	1,437	7	0	22	1	3	6,249.3	0	24,670	24	0
11	7	0	2	0.0	0	1,437	7	1	24	1	4	6,568.6	0	31,238	26	0
12	7	1	3	309.8	0	1,747	8	0	26	1	5	6,941.5	0	38,180	27	0
13	8	1	4	330.3	0	2,077	9	0	27	1	6	7,150.2	0	45,330	28	0
14	9	0	4	0.0	0	2,077	9	1	28	1	7	7,377.0	0	52,707	30	0
15	9	1	1	352.8	0	2,430	10	0	30	1	1	38,081.9	0	90,789	33	0
16	10	0	1	0.0	0	2,430	10	1	33	1	2	9,132.8	0	99,922	33	1
17	10	1	2	377.3	0	2,807	10	2	33	1	3	9,132.8	0	109,054	34	0

그림 70. ④ 일차별 진행 — 라이트 · 코어의 0 ~ 17일차

수식 ④ 일차별 진행의 한 행

```
시작 레벨(d) = 끝 레벨(d-1)                          ← 전날 행만 참조 (순환 참조 없음)
접속(d) = ③ 요일별 접속[(d-1) mod 7 + 1]
이번 주 접속(d) = 주 첫날이면 접속(d), 아니면 이번 주 접속(d-1) + 접속(d)
공급(d) = SUMPRODUCT(② 100행 전체의 [받나] × [획득량] × [시간] × 배율 × 횟수)
누적(d) = 누적(d-1) + 공급(d) + 추가 공급(d)
끝 레벨(d) = MIN(최대 레벨, MATCH(누적(d), ① 누적 열, 1))   ← 누적이 넘은 마지막 레벨
무성장 연속(d) = 레벨이 올랐거나 최대 레벨이면 0, 아니면 전날 + 1
도달일(L) = COUNTIF(끝 레벨 < L) + 1                 ← ⑤ 시트
```

하루 공급은 ② 시트 100행 전체를 한 SUMPRODUCT로 더하므로, 빈 행에 콘텐츠를 적으면 ④의 수식을 고치지 않아도 바로 반영됩니다(주기가 빈 행은 0). 추가 공급 칸은 표로 나타내기 어려운 특이 규칙(예: 이벤트 기간 2배)을 직접 수식으로 넣는 자리이고, 기본값은 0입니다.

⑤ 검증 — 도달일·레벨당 일수·최장 무성장 기간·공급 비중

레벨 L 도달일 = 끝 레벨이 L보다 낮았던 날 수 + 1 (120일 안에 못 닿으면 미도달). 최장 무성장 = 레벨이 오르지 않고 이어진 날의 최댓값

세그먼트	120일차 끝 레벨	최장 무성장 (일)	Lv60 도달일	총 공급
라이트	45	6	미도달	515,020
코어	60	5	112	3,138,897
헤비	60	3	67	6,924,024

마일스톤 — 기준 세그먼트(③ B1)의 실제 도달일

레벨	목표일	실제 도달일	차이 (일)
30	14	14	0
50	60	60	0

공급원별 120일 총량과 비중

공급원	라이트 총량	코어 총량	헤비 총량	라이트 비중	코어 비중	헤비 비중
사냥 (시간당)	85,849.3	1,208,336.0	4,088,305.0	16.7%	38.5%	59.0%
일일 퀘스트	115,170.3	1,148,561.4	1,783,718.7	22.4%	36.6%	25.8%
일일 던전	104,000.0	452,000.0	702,000.0	20.2%	14.4%	10.1%
주간 보스	180,000.0	300,000.0	320,000.0	35.0%	9.6%	4.6%
Lv30 달성 보상	30,000.0	30,000.0	30,000.0	5.8%	1.0%	0.4%

그림 71. ⑤ 검증 시트 — 세그먼트 요약, 마일스톤, 공급원별 총량 · 비중

8.9 엑셀에서 콘텐츠 늘리기

1. ② 공급원 시트의 첫 빈 행에 이름을 적고, 주기 · 해금 기준 · 획득 방식을 드롭다운에서 고릅니다.

2. 해금 값 · 획득값을 넣고, 필요하면 시간 비례(예/아니오), 보정 배율, 세그먼트별 횟수를 적습니다 (비우면 1).
3. ④ · ⑤ 시트가 바로 다시 계산됩니다 — ⑤의 마일스톤 실제 도달일과 최장 무성장을 확인합니다.
4. 규칙이 행 하나로 표현되지 않으면 ④ 시트의 추가 공급 칸에 직접 수식을 넣습니다.

> **팁 세그먼트 수와 최대 레벨은 내보낼 때 정해집니다**
>
> ④ 시트의 세그먼트 열 묶음과 ① 시트의 레벨 행 수는 내보낼 때의 설정으로 고정됩니다. 세그먼트를 더하거나 최대 레벨을 바꾸려면 도구에서 설정을 고쳐 다시 내보내고, 엑셀에서 추가한 ② 행은 새 파일로 옮겨 적으세요. 하루 플레이 분 · 요일별 접속 · 곡선 입력값은 엑셀에서 바로 고쳐도 됩니다.

8.10 계산 검증

화면의 숫자는 엑셀 수식과 같은 규칙으로 계산한 값입니다. 자동 테스트가 내보낸 파일을 LibreOffice로 다시 계산해 프리셋 3종의 일차별 값 · 도달일 · 공급원 총량이 화면과 같은지, ② 빈 행에 공급원을 적은 파일도 맞는지 확인합니다. Microsoft Excel에서도 같은 값, 수식 오류 0입니다.

9. 브리핑

강화·가챠·성장 표·스탯 전투·프레임 탭의 핵심 지표와 검증 문장을 한 장으로 모으는 탭입니다. 스냅샷 대비 변화, 지표별 목표 통과·미달, 결론·근거·리스크·결정 요청 메모를 함께 PDF로 저장해 회의 자료로 씁니다.

9.1 화면 구성

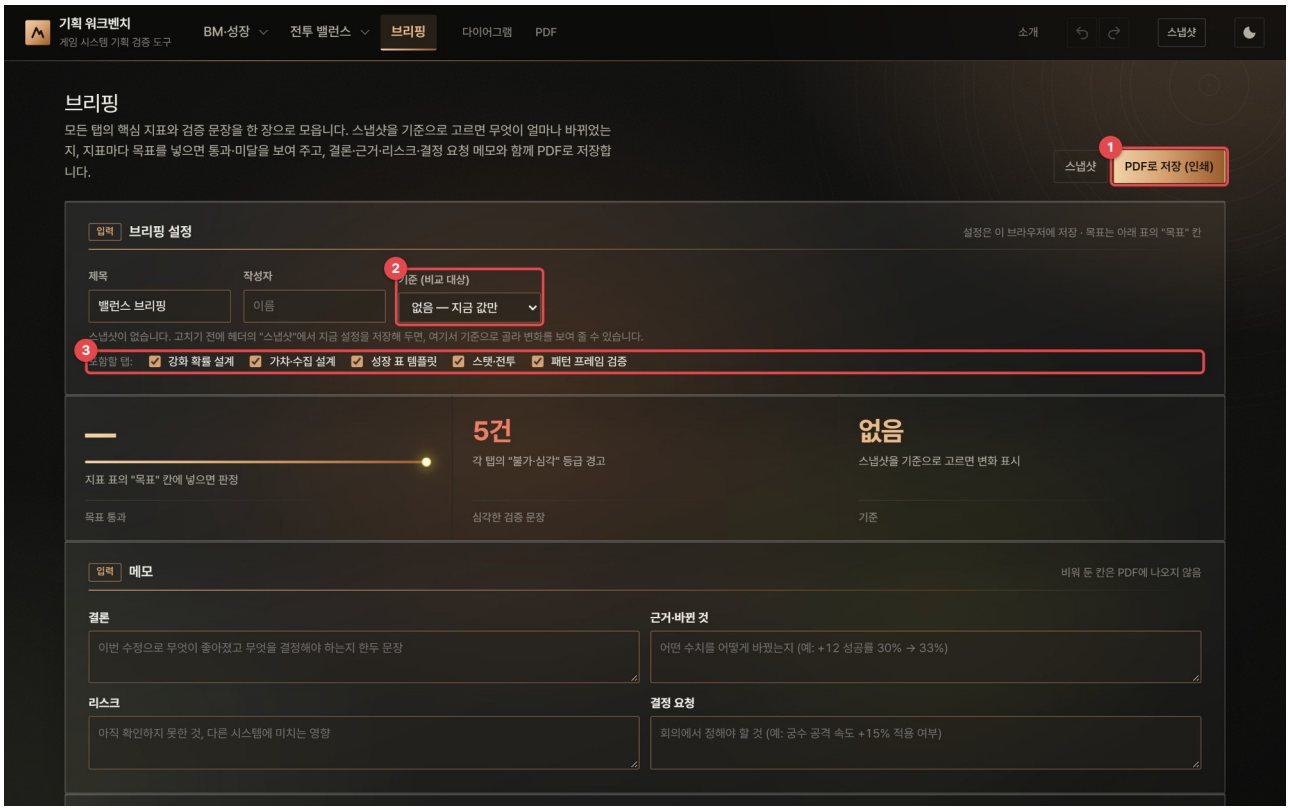


그림 72. 브리핑 — 설정과 요약

1	**PDF로 저장 (인쇄)** — 브라우저 인쇄 창을 엽니다. "PDF로 저장"을 고르면 입력칸·헤더를 숨기고 값만 남긴 문서가 됩니다.
2	**기준 (비교 대상)** — 스냅샷을 고르면 탭별 지표 옆에 기준 대비 변화가 나옵니다. 스냅샷이 없으면 헤더의 "스냅샷"에서 먼저 저장합니다.
3	**포함할 탭** — 브리핑에 넣을 탭을 고릅니다. 위의 제목·작성자도 PDF 머리에 들어갑니다.

요약 카드에는 **목표 통과**(목표를 넣은 지표 중 통과 수), **심각한 검증 문장**(각 탭의 "불가·심각" 등급 경고

수), **기준**(비교 대상)이 나옵니다. 그 아래 **메모**에 결론 · 근거·바뀐 것 · 리스크 · 결정 요청을 적습니다. 비워 둔 메모 칸은 PDF에 나오지 않습니다.

9.2 탭별 지표와 목표

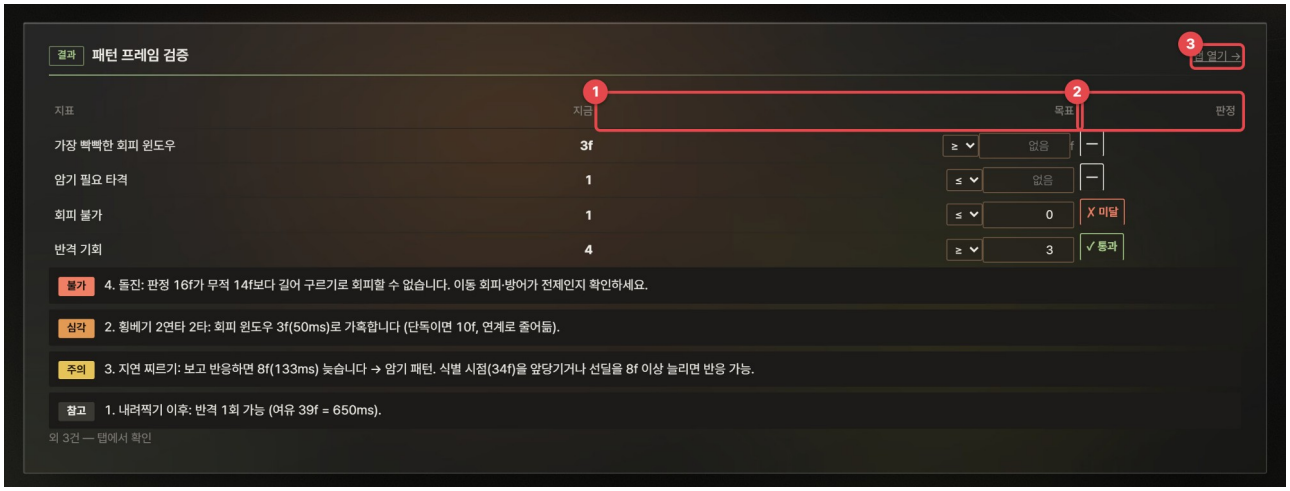


그림 73. 탭별 지표 표 (패턴 프레임 검증 예시 — 목표 두 개를 넣은 상태)

1	**목표** — ≤ 또는 ≥를 고르고 목표값을 넣습니다. 퍼센트 지표는 퍼센트 단위로 넣습니다 (예: 승률 ≤ 60).
2	**판정** — ✓ 통과 / ✗ 미달. 미달 목록은 요약에 모입니다.
3	**탭 열기 →** — 그 탭으로 이동해 수치를 고칩니다.

탭마다 핵심 지표 3~6개와, 심각한 것부터 최대 4개의 검증 문장이 나옵니다. 더 있으면 "외 n건 — 탭에서 확인"으로 줄입니다. 지표는 각 탭의 계산 모듈을 그대로 호출해 계산하므로 탭에서 보는 값과 같습니다.

탭	브리핑 지표
강화 확률 설계	기대 총비용, 기대 시도, 기대 파괴, 파괴 없이 도달
가챠·수집 설계	목표별 기대 비용
성장 표 템플릿	기준 세그먼트의 마일스톤 도달일, 세그먼트별 최장 무성장
스탯·전투	기준 레벨 엔티티별 평균 승률, CP와 실제 순위 상관, CP가 높은데 지는 쌍
패턴 프레임 검증	가장 빡빡한 회피 윈도우, 암기 필요 타격, 회피 불가, 반격 기회

데이터 테이블과 전투 시뮬레이터는 브리핑 지표가 없습니다. 전투 시뮬레이터의 결과는 프리셋 비교의 결과 CSV(1.8절)로 정리해 메모에 붙이세요.

9.3 회의 직전 5분 사용법

1. 수치를 고치기 **전에** 헤더의 **스냅샷**에서 지금 설정을 저장합니다 (예: "10/18 회의 전").
2. 각 탭에서 수치를 고칩니다.
3. 브리핑 탭에서 **기준**으로 그 스냅샷을 고르면 바뀐 탭과 지표별 변화가 한 장에 모입니다.
4. 중요한 지표에 **목표**를 넣어 통과·미달을 확인합니다 (예: 강화 기대 총비용 ≤ 25,000원).
5. **메모**에 결론과 결정 요청을 적고 **PDF로 저장**합니다.

> **팁 목표와 메모는 이 브라우저에만 저장됩니다**
>
> 브리핑 설정(제목·작성자·목표·메모·포함 탭)은 이 브라우저에 자동 저장되지만, 스냅샷·프로젝트 파일에는 다섯 개 도구의 설정과 데이터 테이블만 들어가고 브리핑 메모는 들어가지 않습니다. 회의 결과는 **PDF로 저장**해 남기세요.

10. 부가 도구 — 다이어그램과 PDF

밸런스 계산과 직접 관계는 없지만 기획 문서 작업에 자주 쓰는 도구 두 가지입니다.

10.1 자연어 다이어그램



그림 74. 자연어 다이어그램 — 설명을 쓰면 Mermaid 다이어그램으로

UI 흐름, 상태 머신, 퀘스트 분기, 서버 시퀀스를 문장으로 설명하면 Mermaid 다이어그램으로 만들어 줍니다. 오른쪽 아래 Mermaid 코드를 직접 고치면 미리보기에 바로 반영되고, PowerPoint에서 편집할 수 있는 도형(PPT), SVG, PNG로 저장합니다.

1. **템플릿 (예시 불러오기)** — UI 플로우·상태 머신·퀘스트 분기·시퀀스 예시로 결과와 PPT 내보내기를 먼저 체험할 수 있습니다 (API 키 없이 동작).
2. **설명** 칸에 흐름을 문장으로 씁니다. **다이어그램 형식**은 자동 선택 또는 흐름도·상태 머신·시퀀스 중에서 고릅니다.
3. **Claude API** 키와 모델을 넣고 **다이어그램 만들기**를 누릅니다. 키는 저장되지 않고, 이 기능만 외부 API를 부릅니다.

4. 고칠 점은 **수정 지시**에 문장으로 쓰고 **현재 다이어그램 수정**을 누르거나, Mermaid 코드를 직접 고칩니다.
5. **스타일 설정**에서 색상(블록 종류별·선명한 채움·흑백·기본), 방향, 선 모양, 간격, 글자 크기, "아니오·없음" 분기 빨간 선, "예·있음" 분기 초록 선, 범례(INDEX) 표시를 정합니다.
6. **PPT 도형으로 저장 / SVG 저장 / PNG 저장 / 공유 링크**로 내보냅니다.

블록 모양으로 종류를 나눕니다: 시작·종료 `id(["이름"])`, 처리 `id["이름"]`, 판단 `id{"이름"}`, 입출력 `id[/"이름"/]`, 화면 `id("이름")`, 데이터 `id[("이름")]`, 서브 프로세스 `id[["이름"]]` 등. 코드를 직접 고칠 때 모양만 바꾸면 색과 범례가 따라 바뀝니다.

10.2 PDF 도구

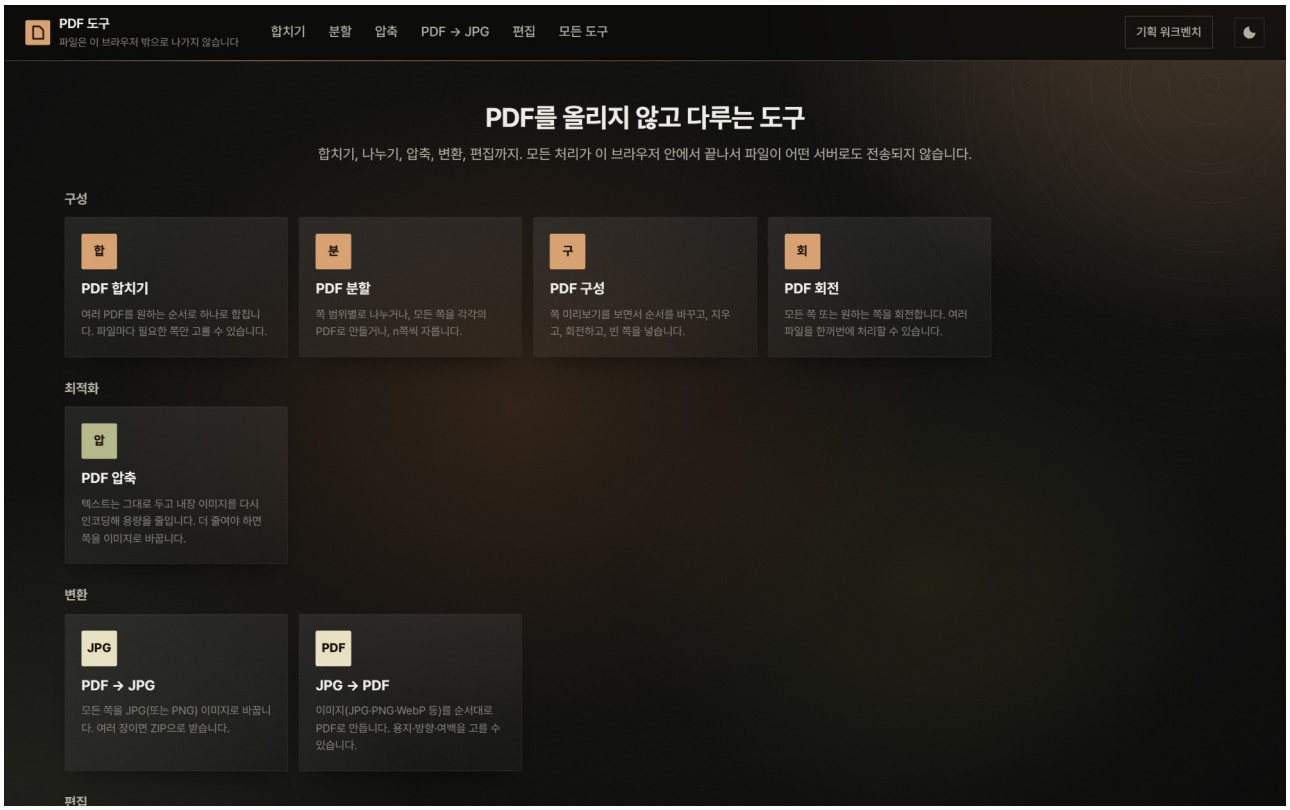


그림 75. PDF 도구 — 파일을 올리지 않고 브라우저 안에서 처리

헤더의 **PDF**를 누르면 별도 페이지로 열립니다. 모든 처리가 이 브라우저 안에서 끝나서 파일이 어떤 서버로도 전송되지 않습니다 (pdf-lib · pdf.js · JSZip 사용).

분류	기능	설명
구성	PDF 합치기	여러 PDF를 원하는 순서로 하나로. 파일마다 필요한 쪽만 고를 수 있음
구성	PDF 분할	쪽 범위별, 모든 쪽을 각각, 또는 n쪽씩
구성	PDF 구성	쪽 미리보기를 보며 순서 변경·삭제·회전·빈 쪽 넣기
구성	PDF 회전	모든 쪽 또는 원하는 쪽 회전, 여러 파일 한꺼번에
최적화	PDF 압축	텍스트는 그대로 두고 내장 이미지를 다시 인코딩. 더 줄여야 하면 쪽을 이미지로
변환	PDF → JPG	모든 쪽을 JPG(또는 PNG)로, 여러 장이면 ZIP
변환	JPG → PDF	이미지를 순서대로 PDF로. 용지·방향·여백 선택
편집	PDF 편집	문구와 이미지를 원하는 쪽·위치에. 한글 글꼴을 함께 넣어 깨지지 않음
편집	워터마크 · 페이지 번호 · 서명 · 자르기	반투명 문구, 쪽 번호 형식(1 / 10 등), 그린 서명·서명 이미지, 여백 자르기(mm)
정보	문서 정보	제목·작성자·주제·키워드 보기·바꾸기·모두 지우기

Word·Excel·PowerPoint 변환, OCR, 암호 걸기·풀기는 서버 처리나 별도 엔진이 필요해 제공하지 않습니다.

11. 도구를 엮어 쓰는 방법

도구 하나로 끝나는 질문도 있지만, 실제 기획에서는 여러 탭을 이어서 씁니다. 자주 쓰는 흐름을 정리했습니다.

11.1 강화 BM 가격 정하기

1. **강화** 탭에서 확률표와 비용을 넣고 **기준안 저장**.
2. 보조 아이템 가격을 임시로 넣고 **최적 정책 계산** — 절감률이 너무 크면(예: 50%) 아이템이 필수품이 됩니다.
3. **아이템 가격 곡선**에서 목표 절감률(예: 15%)을 넣고 계산 → "절감률이 목표 이하가 되는 가격"을 적용.
4. **브리핑**에서 기대 총비용·파괴 없이 도달 확률에 목표를 넣고 PDF로 정리.

11.2 가챠 공시 준비

1. **가챠** 탭에서 확률·소환 레벨·천장·픽업을 확정.
2. ② **목표**에 대표 목표(픽업 1장, 풀 초월 등)를 넣고 99% 비용과 도달 기간을 확인 → 필요하면 마일리지로 꼬리 상한을 둡니다.
3. **확률 공시표**에서 자릿수를 고르고 반올림 경고가 없는지 확인 → **공시표 CSV**.

11.3 성장 곡선 초안을 팀 엑셀로

1. 성장 표 탭에서 장르 프리셋을 고르고 ② 공급원 · ③ 세그먼트를 게임에 맞게 고칩니다.
2. 마일스톤 두 개(예: 코어 Lv30 14일차 · Lv50 60일차)를 넣고 증가율까지 맞추기 → 라이트 유저의 최장 무성장이 5일을 넘지 않는지 확인.
3. 스냅샷을 저장해 두고 브리핑에서 마일스톤 도달일 · 최장 무성장에 목표를 넣어 안(案)끼리 비교합니다.
4. 엑셀(.xlsx) 내보내기 → 팀 성장 표로 공유하고, 새 콘텐츠는 ② 빈 행에 추가합니다.

11.4 전투 데이터를 엑셀에서 고쳐 보스 난이도 맞추기

1. 데이터 테이블에서 몬스터·스킬 테이블을 CSV로 내보내 엑셀 기획 시트와 맞추고, 고친 CSV를 가져와 검증 오류를 0으로 만듭니다.
2. 전투 시뮬레이터의 파티 전투에서 대표 파티와 보스를 배치하고, 지금 상태를 프리셋("1차안")으로 저장합니다.
3. 보스의 그로기 최대치·페이즈 체력%·광폭화 수치나 규칙(그로기 배율 등)을 바꿔 "2차안"으로 저장하고 A/B 비교 — 승률과 클리어 P50·P90이 목표 안에 드는지 봅니다.
4. 조합 비교의 자동 생성으로 대표 파티 말고도 이길 수 있는 조합이 몇 개인지 확인합니다(한 조합만 이기면 사실상 정답 파티가 강제됩니다).
5. 교체 비교로 역할마다 빠졌을 때의 승률 변화를 보고, 프리셋 비교 결과 CSV와 기획서 영역의 문장으로 정리합니다. 테이블까지 묶어 남기려면 스냅샷을 저장합니다.

11.5 팀과 공유하기

방법	언제	담기는 것
공유 링크	메신저로 "이 설정 봐 주세요"	그 탭의 설정 전체 (주소 # 뒤, 서버로 가지 않음)
JSON 저장 / 불러오기	탭 하나의 설정을 파일로 버전 관리	그 탭의 설정
스냅샷	같은 브라우저에서 안(案)끼리 비교	다섯 개 도구의 설정 + 데이터 테이블
프로젝트 파일	다른 PC·팀원에게 전체 작업 전달	다섯 개 도구의 지금 설정 + 데이터 테이블 + 모든 스냅샷
데이터 테이블 JSON · CSV	전투 데이터를 팀·엑셀과 주고받기	일곱 개 테이블 전체(JSON) 또는 테이블 하나(CSV·JSON)
프리셋 (A/B)	같은 탭 안에서 안(案) 여러 벌 비교	그 탭의 설정 (이 브라우저, JSON으로 내보내기 가능)
브리핑 PDF	회의·보고	지표, 목표 판정, 검증 문장, 메모
CSV · PPT · 엔진 내보내기	엑셀·기획서·Unity/Unreal	탭마다 다름 (각 장 참고)

11.6 자주 묻는 질문

같은 설정인데 숫자가 매번 같나요?

네. 공식 계산은 당연히 같고, 시뮬레이션도 시드를 고정한 난수를 써서 같은 설정이면 같은 결과가 나옵니다. 설정을 바꾸면 결과가 바뀌는 이유는 설정 때문이지 "운" 때문이 아닙니다. 다른 난수로 다시 돌려 보고 싶으면 각 탭의 시뮬레이션 설정에서 시드를 바꾸고, 기본 시드로 되돌리면 원래 결과가 그대로 나옵니다(1.8절).

공식 계산과 시뮬레이션이 조금 다릅니다.

시뮬레이션에는 표본 오차가 있습니다. 계산 검증 카드에서 공식 계산값이 시뮬레이션 95% 범위 안에 있으면 정상입니다. 범위를 벗어나면 시뮬레이션 횟수를 늘려 다시 보고, 그래도 벗어나면 설정에 모델이 다루지 않는 규칙이 있는지 확인하세요.

분위표의 0.1%·99.9% 값이 설정을 조금만 바꿔도 크게 흔들립니다.

시뮬레이션 n회 중 바깥쪽 0.1%(2만 회면 20회, 3,000회면 3회)로 정해지는 값이라 그렇습니다. 꼬리가 중요한 판단이면 시뮬레이션 횟수를 늘리세요. 가챠의 픽업·특정 캐릭터 목표처럼 공식으로 분포를 구하는 목표는 흔들리지 않습니다.

평균(기대값)과 중앙값 중 무엇을 봐야 하나요?

강화·가챠처럼 운 나쁜 소수가 큰 비용을 쓰는 분포에서는 평균이 중앙값보다 큽니다. 매출·재화 소모 총량에는 평균이, "보통 유저의 체감"에는 중앙값이, "민원이 나올 수준"에는 90%·99% 값이 맞습니다. 세 가지를 함께 보세요.

탭을 바꿨더니 그래프가 비어 보입니다.

탭이 다시 보일 때 그래프 크기를 다시 맞춰 그리도록 되어 있습니다. 그래도 비어 있으면 Ctrl+F5로 새로고침하세요(이전 버전 파일이 브라우저에 남아 있을 수 있음).

되돌리기가 동작하지 않습니다.

입력칸 안에 커서가 있으면 Ctrl+Z가 그 칸의 글자 되돌리기로 동작합니다. 칸 밖을 누른 뒤 Ctrl+Z를

누르거나 헤더의 ↶ 버튼을 쓰세요. 되돌리기는 탭마다 따로 기록됩니다.

데이터가 서버에 남나요?

여덟 개 도구(브리핑·데이터 테이블 포함)와 PDF 도구는 서버로 아무것도 보내지 않습니다. 설정은 이 브라우저 저장소에만 있으므로, 브라우저 데이터를 지우면 사라집니다. 중요한 작업은 JSON·프로젝트 파일로 저장해 두세요. 다이어그램 도구의 "다이어그램 만들기"만 입력한 설명을 Claude API로 보냅니다.

전투 시뮬레이터 공유 링크를 보냈는데 결과가 다릅니다.

공유 링크·JSON·프리셋에는 전투 규칙과 구성만 들어가고 데이터 테이블은 들어가지 않습니다. 받는 사람의 테이블이 다르면 결과가 다릅니다. 데이터 테이블 탭의 JSON 저장 (전체)을 함께 보내거나, 스냅샷·프로젝트 파일로 묶어 보내세요.

부록 — 알고리즘·수학 사전

본문 보라색 상자에서 언급한 계산 방법을 도구별로 묶어 정리했습니다. A.1은 여러 도구가 함께 쓰는 방법이고, A.2~A.8은 본문 2~8장의 도구와 번호가 같습니다. 각 도구 절은 "그 도구가 무엇을 어떤 순서로 계산하는지"를 표로 먼저 보이고, 이어서 방법을 계산 순서대로 설명합니다. 방법마다 파란 줄의 "기획 관점으로 풀어 보기"에서 어떤 기획 문제를 푸는지 공통 예제로 보이고, "원리와 공식"에서 무엇인가 → 수식 → 쉬운 예 → 워크벤치에서 쓰인 곳 → 한계·주의 순서로 정리합니다. 계산 속도를 위한 행렬 풀이, 난수 생성, 수식 해석, 화면 배치, 자동 테스트처럼 구현에만 필요한 내용은 별도 문서 「기획 워크벤치 구현 노트」로 옮겼습니다.

번호	도구 / 방법
A.1	**여러 도구가 함께 쓰는 방법**
A.1.1	몬테카를로 시뮬레이션
A.1.2	기대값, 중앙값, 분위수
A.1.3	동적 계획법 — 상태 분포를 앞으로 전진시키기
A.1.4	이분 탐색, 로그 이분 탐색, 공통 난수
A.2	**강화 확률·비용 설계 (2장)**
A.2.1	마르코프 체인과 흡수 마르코프 체인
A.2.2	수반(adjoint) 방법을 이용한 민감도
A.2.3	마르코프 결정 과정과 정책 반복법
A.2.4	목표 비용 역산 — 곱셈 분배와 범위 넘김
A.3	**가챠·수집 설계 (3장)**
A.3.1	기하분포, 포아송화, 쿠폰 수집 문제
A.3.2	10연 보장의 실제 등급 비율
A.4	**스탯·전투 밸런스 (4장)**
A.4.1	독립 확률변수로 1:1 승률 계산
A.4.2	이항분포와 Wilson 점수 구간
A.4.3	스피어만 순위 상관
A.4.4	반대 가정과 토네이도 민감도
A.5	**데이터 테이블 (5장)**
A.5.1	참조 무결성 검사

번호	도구 / 방법
A.6	**전투 시뮬레이터 (6장)**
A.6.1	데미지 공식 네 가지 — 방어력이 데미지를 줄이는 모양
A.6.2	이산 사건 시뮬레이션 — 턴제와 실시간
A.6.3	반대 가정으로 보는 기여도 — 효과 빼기와 교체 비교
A.7	**패턴 프레임 검증기 (7장)**
A.8	**성장 표 템플릿 (8장)**
A.8.1	base와 증가율 자동 풀이 — 이분 탐색

A.1 여러 도구가 함께 쓰는 방법

강화·가챠·스탯·전투는 모두 확률이 섞인 결과를 다룹니다. 결과를 구하는 길은 두 가지입니다. 실제로 여러 번 굴려 보는 시뮬레이션(A.1.1)과, 같은 상태를 합쳐 정확한 분포를 계산하는 동적 계획법(A.1.3)입니다. 어느 쪽이든 결과는 평균·중앙값·분위수로 읽고(A.1.2), "목표에 맞추려면 얼마로 해야 하나"라는 역산과 두 설정의 비교에는 이분 탐색과 공통 난수를 씁니다(A.1.4). 도구별 절에서 이 네 가지를 계속 참조하므로 먼저 읽어 두세요.

기획 질문	방법	쓰는 도구	절
규칙대로 굴려 보면 결과가 어떻게 퍼지나?	몬테카를로 시뮬레이션	강화, 가챠, 스탯·전투, 전투 시뮬레이터	A.1.1
평균·보통 유저·운 나쁜 1%는 각각 얼마?	기대값·중앙값·분위수	강화, 가챠, 전투 시뮬레이터	A.1.2
천장·보장이 얽힌 결과의 정확한 분포는?	동적 계획법	가챠, 스탯·전투, 프레임	A.1.3
목표 값을 만들려면 수치를 몇 배로? 두 설정의 차이는?	이분 탐색·공통 난수	강화, 스탯·전투, 전투 시뮬레이터, 성장 표	A.1.4

A.1.1 몬테카를로 시뮬레이션

기획 관점으로 풀어 보기

풀려는 문제

강화 기대 비용의 평균은 A.2.1의 연립방정식으로 정확히 나옵니다. 하지만 기획자가 궁금한 것은

평균만이 아닙니다. "보통 유저는 얼마 쓰나(중앙값)", "운 나쁜 1%는 얼마 쓰나(99% 값)", "3만 원 넘게 쓰는 유저는 몇 %인가"는 비용이 유저들 사이에 어떻게 퍼져 있는지, 즉 **분포**를 알아야 답할 수 있습니다. 데미지 변동·특성이 섞인 전투처럼 규칙이 복잡하면 평균조차 공식으로 풀기 어렵습니다.

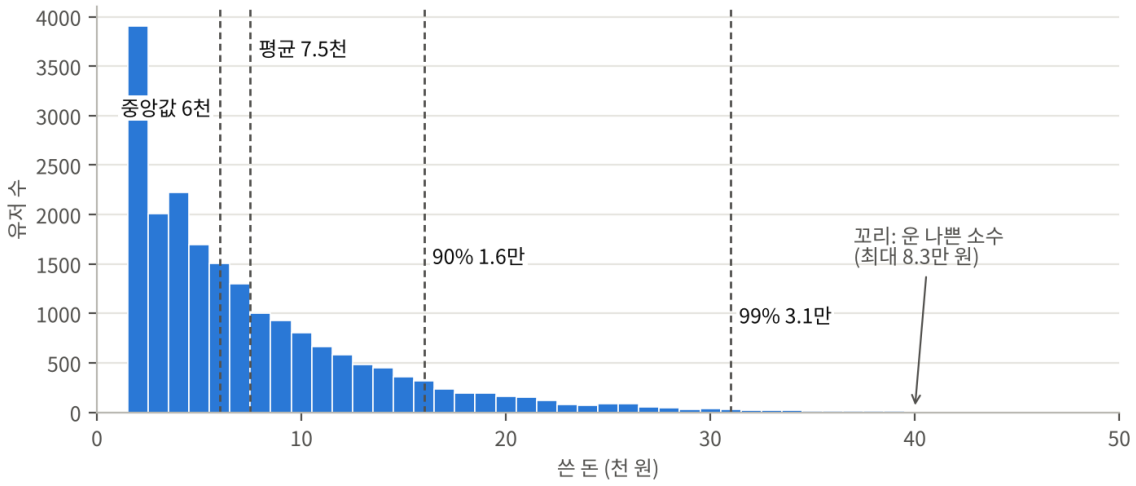
이 부록의 공통 예제

아래 작은 강화를 이 부록 곳곳(분위수, 이분 탐색, 강화의 세 항목, 토네이도)에서 계속 씁니다.

```
시도 1회 = 1,000원,  시작 +1 → 목표 +3
+1에서 시도: 성공 50% → +2,  실패 50% → +1 유지
+2에서 시도: 성공 40% → +3(목표),  실패 60% → +1로 하락
```

어떻게 푸나 — 실제로 2만 번 시켜 본다

이 예제를 가상 유저 2만 명에게 끝까지 시키고(시드 고정), 쓴 돈을 1,000원 구간마다 세어 막대로 그린 것이 아래 그림입니다.



예시 — 공통 예제 2만 회 시뮬레이션의 비용 분포

값	결과	뜻
평균	7,587원 (정답 7,500원)	2만 명 비용의 합 ÷ 2만
중앙값	6,000원	1만 번째 사람
90% 값	16,000원	18,000번째 사람 — 운 나쁜 10%의 시작
99% 값	31,000원	19,800번째 사람 — 운 나쁜 1%의 시작
최대	83,000원	가장 운 나쁜 1명

분포 모양과 "꼬리"

막대가 왼쪽(2천~5천 원)에 몰려 있고 오른쪽으로 가늘고 길게 이어집니다. 이 오른쪽의 가는 부분이 **꼬리**입니다. 이 예제는 꼬리가 길어서 가장 운 나쁜 유저(8.3만 원)가 중앙값(6천 원)의 약 14배를 씁니다. 꼬리가 길면 평균이 꼬리 쪽으로 끌려가 중앙값보다 커지고(7,500 > 6,000), "평균 7,500원"이라는 숫자만으로는 100명 중 1명이 31,000원 넘게 쓴다는 사실이 보이지 않습니다. 강화·가챠 민원은 이 꼬리에서 나옵니다.

정확도 — 표준오차

시뮬레이션 평균은 운에 따라 흔들립니다. 이 예제의 표준편차는 약 6,360원이므로 표준오차 = 6,360 ÷ √20,000 ≈ 45원, 95% 범위는 7,587 ± 88원(7,499~7,675원)입니다. 정답 7,500원이 범위 끝에 겨우 들어 있습니다. 2만 번을 돌려도 87원이 어긋날 수 있다는 것이 시뮬레이션의 한계이고, 오차를 절반으로 줄이려면 4배(8만 번)를 돌려야 합니다.

드문 사건에는 오히려 약합니다

몬테카를로는 드문 사건을 해결하는 도구가 아닙니다. 확률 0.01%짜리 사건은 2만 번 중 평균 2번 나오므로, 0번·2번·5번 중 몇 번 나오느냐에 따라 추정치가 크게 흔들립니다. 그래서 워크벤치는 평균·파괴 확률처럼 공식으로 풀 수 있는 값은 공식(A.2.1)으로 풀고, 몬테카를로는 ① **중앙값·상위 n% 같은 분포 확인**과 ② **공식 결과가 맞는지 대조**에 씁니다.

원리와 공식

무엇인가: 확률이 섞인 과정을 난수로 여러 번 실제로 돌려 보고, 나온 결과들의 평균·비율·분포로 답을 추정하는 방법입니다. 1940년대 핵물리 계산에서 쓰이기 시작했고, 이름은 카지노로 유명한 모나코의 몬테카를로에서 왔습니다. 수식으로 풀기 어려운 복잡한 규칙도 "규칙대로 한 번 굴려 보는 코드"만 있으면 계산할 수 있다는 것이 장점입니다.

```
평균 추정:   m = (x₁ + x₂ + … + xₙ) ÷ n
표준오차:     SE = s ÷ √n        (s = 표본 표준편차)
95% 범위:    m ± 1.96 · SE
비율(승률):  p̂ = (이긴 횟수) ÷ n,  95% 범위는 Wilson 구간 (A.4.2)
```

쉬운 예: 강화 +0→+15를 2만 명이 각자 한 번씩 끝까지 해 보게 하고, 각자 쓴 비용 2만 개의 평균을 냅니다. 2만 개를 작은 순으로 줄 세우면 1만 번째가 중앙값, 19,800번째가 99% 값입니다.

정확도: 표준오차가 1/√n로 줄어들기 때문에, 오차를 절반으로 줄이려면 횟수를 4배 늘려야 합니다. 이는 **중심극한정리**(독립적인 값을 많이 더한 평균은 원래 분포와 상관없이 정규분포에 가까워진다)에 근거합니다. 수렴 그래프의 95% 띠가 오른쪽으로 갈수록 좁아지는 것이 이 성질입니다.

시드 고정 난수: 워크벤치는 난수에 고정된 시드(씨앗값)를 넣어, 같은 설정이면 항상 같은 결과가 나옵니다. 그래서 설정을 조금 바꿔 비교할 때 차이가 운이 아니라 설정 때문임을 알 수 있습니다(공통 난수, A.1.4). 난수를 만드는 방식은 구현 노트 I.5.

워크벤치에서 쓰인 곳: 강화(2만 회), 가챠(목표당 3,000회 기본), 스탯·전투(데미지 변동·특성이 있는 대결과 검증 4,000회, 레벨 훑기는 6만 전투 예산, 배틀 시뮬레이터는 FIGHT 상대마다 지정 횟수·M.C 1만 회), 전투 시뮬레이터(설정당 300판, 조합 비교는 조합당 80판).

한계: 아주 드문 사건(예: 0.01%)은 횟수가 적으면 거의 나오지 않아 추정이 부정확합니다. 그래서 워크벤치는 공식으로 풀 수 있는 곳은 공식으로 풀고, 시뮬레이션은 검증과 분포 확인에 씁니다.

A.1.2 기대값, 중앙값, 분위수

기획 관점으로 풀어 보기

세 숫자는 질문이 다릅니다

숫자	답하는 질문	공통 예제
평균(기대값)	1,000명이 하면 총 얼마? — 매출·재화 총소모	7,500원
중앙값	가운데 사람은 얼마? — 보통 유저의 체감	6,000원
90% 값	운 나쁜 10%는 얼마부터?	16,000원
99% 값	운 나쁜 1%는 얼마부터? — 민원 구간	31,000원

매출 예측에는 7,500원을 곱하지만, 유저 대부분이 "7,500원쯤 든다"고 느끼는 것은 아닙니다. 절반은 6,000원 이하로 끝내고, 100명 중 1명은 31,000원 넘게 씁니다.

꼬리가 길면 평균 > 중앙값

소수가 아주 많이 쓰면 평균이 그쪽으로 끌려갑니다. 그래서 평균만 보고 밸런스를 맞추면 "평균은 맞는데 대부분은 싸게 끝나고, 일부는 지옥"인 상태가 됩니다. 천장·마일리지처럼 꼬리를 자르는 장치는 평균보다 99% 값을 훨씬 크게 줄이므로, 효과를 반드시 분위로 확인하세요.

분위 값을 읽을 때 주의

99% 값은 2만 개 중 바깥쪽 200개, 99.9%는 바깥쪽 20개로 정해집니다. 꼬리로 갈수록 근거가 되는 시행이 적어 흔들립니다. 워크벤치는 정렬한 결과에서 ⌈p·n⌉번째 값을 그대로 쓰므로(보간 없음), 표의 값은 실제 어떤 시행에서 나온 값입니다.

원리와 공식

기대값(평균): 모든 결과를 확률로 가중해 더한 값. 총량(매출, 재화 총소모) 계산에 맞습니다. **중앙값**: 줄 세웠을 때 가운데 값. "보통 유저"의 체감에 맞습니다. **p 분위수**: 유저의 p 비율이 그 값 이하인 값. 90% 분위 = 운 나쁜 10%의 경계.

꼬리가 긴 분포: 강화·가챠 비용은 오른쪽 꼬리가 깁니다(소수가 아주 많이 씀). 이때 평균 > 중앙값이고, 평균은 소수의 큰 값에 끌려 올라갑니다. 기본 강화 템플릿에서 평균 2만 7,967원, 중앙값 약 2.1만 원, 99% 11만 원.

```
경험적 분위수 (최근접 순위):  정렬된 표본 x₍₁₎ ≤ … ≤ x₍ₙ₎ 에서 Q(p) = x₍⌈p·n⌉₎
변동계수 = 표준편차 ÷ 평균   (1에 가까우면 편차가 평균만큼 큼)
```

워크벤치에서 쓰인 곳: 강화·가챠의 중앙값·상위 n%, 도달 예산, 달성 분위별 수치(0.1%~99.9%), 강화 KPI의 변동계수.

A.1.3 동적 계획법 — 상태 분포를 앞으로 전진시키기

기획 관점으로 풀어 보기

풀려는 문제

천장·반천장(픽업 실패 시 다음 최상위는 픽업 보장)이 있는 가챠는 결과 경로가 폭발합니다. 뽑기마다 "꽝 / 픽업 / 픽업 아닌 최상위" 세 갈래면 100번 뽑을 때 경로가 3^{100}개입니다. 시뮬레이션으로 하면 오차가 남고요.

핵심 아이디어 — 같은 상태끼리 합친다

경로가 달라도 **지금 상태가 같으면 앞으로의 확률은 같습니다.** 예를 들어 픽업 2장 목표에서 "10번째에 픽업 → 이후 꽝 15번"과 "3번째에 픽업 아닌 최상위 → 10번째에 보장 픽업 → 이후 꽝 15번"은 경로는 다르지만, 25번째 뽑기 후 상태가 둘 다 (1장, 천장 카운터 15, 보장 없음)입니다. 그러니 두 경로의

> 확률을 더해 한 칸에 담아 두면 됩니다. 매 뽑기마다 상태별 확률만 들고 다니므로 계산량은 상태 수 × 뽑기 수입니다.

예 — SSR 2%, 80번째 확정, 픽업 50%(실패 시 다음 SSR 픽업 보장), 픽업 1장

항목	값
상태	천장 카운터 0~79 × 보장 없음/있음 = 최대 160칸
평균	60.1회
중앙값	58회
90% 유저가 이 안에 달성	113회
최악	160회 (천장 두 번)

> 분포 전체가 오차 없이 나오므로 "예산 100회로 달성할 확률" 같은 값을 그대로 읽을 수 있습니다. 같은 방법이 전투의 "몇 번째 공격에 처치하나"(상태 = 일반 적중 수, 치명 수)와 프레임 회피 계획에도 쓰입니다.

원리와 공식

무엇인가: 큰 문제를 겹치는 작은 문제로 나누고, 작은 문제의 답을 저장해 다시 쓰는 방법입니다. 워크벤치에서는 주로 "확률 분포를 한 단계씩 앞으로 밀어내는" 형태로 씁니다. 모든 경로를 하나씩 따지면 경우의 수가 폭발하지만, 같은 상태에 도착한 경로들의 확률을 합쳐 두면 상태 수 × 단계 수만큼만 계산하면 됩니다.

가챠 (픽업·특정 캐릭터): 상태 = (모은 장수, 천장 카운터, 보장 여부). 뽑기마다 상태 확률을 다음 상태로 나눠 보내고, 목표 장수에 닿은 확률을 그 회차의 "처음 달성 확률"로 기록합니다. 결과는 분포 전체라 평균·분위수·예산별 달성 확률이 모두 오차 없이 나옵니다.

스탯·전투 (처치 공격 횟수): 상태 = (일반 적중 수, 치명 수). 공격마다 빗나감/일반/치명 확률로 전진하고, 누적 데미지가 상대 체력 이상이 되는 순간의 확률을 "n번째 공격에서 처치"로 기록합니다 (A.4.1).

프레임 (최적 회피): 상태 = 지금까지 처리한 타격 수, 값 = 다시 행동 가능한 가장 이른 프레임. 작을수록 이후에 유리하므로 이 값 하나만 기억하면 되는 구조입니다 (7.6절).

워크벤치에서 쓰인 곳: 가챠 정확해 ②, 스탯·전투 처치 횟수 분포, 프레임 최적 입력 계획.

A.1.4 이분 탐색, 로그 이분 탐색, 공통 난수

기획 관점으로 풀어 보기

풀려는 문제 — 목표에서 거꾸로 찾기

"평균 비용을 5,000원으로 맞추려면 성공률을 몇 배로 해야 하나?" 확률 → 비용은 계산할 수 있지만 비용 → 확률은 공식으로 뒤집기 어렵습니다. 대신 **성공률을 올리면 비용은 반드시 줄어든다(단조)**는 성질이 있습니다. 이 성질만 있으면 범위를 반씩 좁혀 답을 찾을 수 있습니다.

공통 예제 — 성공률 배율 α 찾기 (목표 평균 5,000원)

회차	범위	가운데 α	그때 평균	판단
1	0.5 ~ 2.0	1.250	5,200원	비쌈 → 위쪽 절반
2	1.25 ~ 2.0	1.625	3,432원	쌈 → 아래쪽 절반
3	1.25 ~ 1.625	1.438	4,159원	쌈
4	1.25 ~ 1.438	1.344	4,630원	쌈
5	1.25 ~ 1.344	1.297	4,901원	쌈
6	1.25 ~ 1.297	1.273	5,046원	비쌈
8	1.273 ~ 1.285	1.279	5,009원	…

매번 범위가 절반이 되므로 8번이면 1.28배 근처까지, 40번이면 처음 범위의 1조 분의 1까지 좁혀집니다.

로그 이분 탐색 — 배율은 곱셈으로 나눈다

1/100배 ~ 100배 범위를 산술 가운데((0.01 + 100) ÷ 2 ≈ 50)로 나누면 첫 질문이 "50배보다 큰가?"가 되어, 실제로 답이 있을 가능성이 높은 1배 근처를 거의 탐색하지 않습니다. 로그 가운데(√(0.01 × 100) = 1)로 나누면 ×2와 ×½을 같은 무게로 탐색합니다.

계단 모양 결과

공격 횟수처럼 정수가 끼면 배율을 조금 바꿔도 결과가 안 변하다가 한 번에 뜁니다. 목표에 정확히 닿지 않을 수 있으므로 "목표를 처음 넘는 값"과 바로 아래 값의 결과를 같이 보여 줍니다.

공통 난수 — 비교할 때 운을 지우기

시뮬레이션 두 개를 서로 다른 난수로 돌려 비교하면 "설정 차이 + 운 차이"가 섞입니다. 같은 난수열을

> 쓰면 두 설정이 같은 운을 겪으므로 차이에는 설정 효과만 남습니다. 같은 사람 2만 명에게 두 버전을 모두 시켜 보는 것과 같습니다.

원리와 공식

이분 탐색: 값을 키우면 결과가 한 방향으로만 변하는(단조) 관계에서, 목표를 만드는 값을 찾을 때 범위를 매번 절반으로 줄여 가는 방법입니다. 40번 반복하면 범위가 약 1조 분의 1로 줄어듭니다.

로그 이분 탐색: "배율"처럼 곱셈으로 의미가 있는 값은 산술 중간값((a+b)/2) 대신 로그 공간의 중간값(√(a·b))으로 나눕니다. 그래야 1/100배와 100배 사이를 고르게 탐색합니다.

계단 모양 결과: 공격 횟수처럼 정수가 끼면 결과가 계단 모양이라 목표값에 정확히 닿지 않을 수 있습니다. 이때는 "목표를 처음 넘는 값"과 그때의 결과, 바로 아래 값의 결과를 함께 보여 줍니다.

공통 난수: 시뮬레이션 결과를 비교할 때 두 설정에 같은 난수열을 쓰면, 차이에서 난수 잡음이 상당 부분 상쇄되어 작은 차이도 안정적으로 보입니다. 밸런스 도구의 시뮬레이션 쌍은 배율마다 같은 시드를 씁니다.

워크벤치에서 쓰인 곳: 강화 목표 비용 역산(최대 80회, A.2.4), 스탯·전투 최소 변경·목표 역산(1/64~64배, 공식 40회·시뮬레이션 22회), 성장 표 base·증가율 자동 풀이(A.8.1), 전투 시뮬레이터의 조합·교체·프리셋 비교(공통 난수).

A.2 강화 확률·비용 설계 (2장)

강화 탭은 단계별 확률표 하나로 세 가지 질문에 차례로 답합니다. 먼저 확률표를 "상태 이동"으로 모델링해 목표까지의 기대 비용과 파괴 위험을 정확히 구하고(A.2.1), 그 결과를 이용해 어느 단계 확률이 비용을 좌우하는지 한 번에 구한 뒤(A.2.2), 모델에 "아이템을 쓸지 말지"라는 선택을 더해 최적 사용 정책과 적정 가격을 찾습니다(A.2.3). 세 절은 같은 공통 예제로 이어집니다.

기획 질문	방법	절
목표까지 기대 비용·파괴 위험·시도 횟수는?	흡수 마르코프 체인 (천장·파괴 복구 포함)	A.2.1
어느 단계 확률을 손대야 비용이 가장 많이 바뀌나?	수반 방법 민감도	A.2.2

기획 질문	방법	절
보조 아이템을 언제 써야 이득이고, 가격은 적정한가?	마르코프 결정 과정·정책 반복	A.2.3
보통 유저·상위 1%는 얼마? 강화 경로 예시는?	시뮬레이션·분위수	A.1.1, A.1.2
목표 비용에 맞추려면 확률·비용을 몇 배로?	곱셈 분배·로그 이분 탐색	A.2.4, A.1.4

A.2.1 마르코프 체인과 흡수 마르코프 체인

기획 관점으로 풀어 보기

마르코프 체인은 "모델", 연립방정식은 "풀이"

마르코프 체인 자체는 연립방정식이 아닙니다. 강화를 "상태"와 "상태 사이를 옮겨 갈 확률표"로 표현하는 **모델링 방법**입니다. 이 모델에 "목표까지 기대 비용은 얼마인가?"를 물으면 그 답이 연립방정식으로 나옵니다.

왜 연립방정식이 되나 — 공통 예제

E_1 = +1에서 목표까지 앞으로 쓸 기대 비용, E_2 = +2에서의 기대 비용이라고 둡니다. 한 번 시도하면 1,000원을 쓰고 확률에 따라 다음 상태로 가는데, 다음 상태에서의 기대 비용은 다시 E_1이나 E_2입니다.

```
E₁ = 1,000 + 0.5·E₂ + 0.5·E₁      (+1: 성공하면 +2, 실패하면 +1)
E₂ = 1,000 + 0.4·0  + 0.6·E₁      (+2: 성공하면 끝, 실패하면 +1로 하락)
```

E_1을 알려면 E_2가 필요하고, E_2를 알려면 다시 E_1이 필요합니다. **하락 때문에 서로가 서로를 참조하는 순환**이 생기므로 위에서부터 차례로 더해서는 풀 수 없고, 동시에 풀어야 합니다. 첫 식에서 E_1 = 2,000 + E_2, 이를 둘째 식에 넣으면 0.4·E_2 = 2,200 → **E_2 = 5,500원, E_1 = 7,500원**. 실제 강화는 단계 × 천장 카운터만큼 식이 수백 개이고, 이를 한꺼번에 쓴 것이 아래의 (I − Q)E = r입니다.

시뮬레이션 대신 이것을 쓰는 이유

- **오차가 0입니다.** A.1.1에서 2만 번 돌린 평균은 7,587원으로 87원 어긋났지만, 연립방정식은 정확히 7,500원입니다.
- **빠릅니다.** 확률을 고칠 때마다 2만 명을 다시 돌리지 않고 식만 다시 풉니다.
- **작은 차이를 비교할 수 있습니다.** "성공률 1%p 올리면 몇 원 줄어드나(A.2.2)", "아이템을 쓰는 게

이득인가(A.2.3)"는 수십~수백 원 차이를 비교하는 문제라, 시뮬레이션 잡음이 그 차이보다 크면 판단할 수 없습니다.

● 대신 바로 나오는 것은 평균·분산까지입니다. 중앙값·상위 1%는 시뮬레이션(A.1.1)으로 봅니다.

천장 — 상태를 (단계, 카운터)로 넓히는 이유

천장은 연속 실패 횟수(카운터)를 세다가 정해진 횟수에 닿으면 다음 시도를 확정 성공으로 만드는 규칙입니다(2.4절 연속 보정). 이 카운터 때문에 같은 +14라도 "연속 실패 0회"와 "9회(천장 10회)"는 다음 성공 확률이 다릅니다. 마르코프 체인은 **지금 상태만으로 다음 확률이 정해져야** 하므로 두 경우를 다른 상태로 나눕니다. 그래서 상태 수 = 단계 수 × (천장 + 1)입니다.

설정	상태 수
15단계, 천장 없음	15
15단계, 천장 10회	15 × 11 = 165
30단계, 천장 50회	30 × 51 = 1,530

천장을 넣을 때마다 풀어야 할 식이 이만큼 늘어납니다. 그래도 즉시 풀리는 이유(띠 행렬)는 구현 노트 I.1에 정리했습니다.

파괴와 복구 — 기대 비용을 두 부분으로 나누기

파괴되면 정해진 복구 단계(예: +12)로 돌아가 다시 올라옵니다. 연속 카운터는 파괴 시 0으로 초기화되므로, 어디서 파괴되든 도착하는 상태는 **(복구 단계, 카운터 0) 하나**입니다. 그래서 기대 비용을 이렇게 나눠 볼 수 있습니다.

```
E_s = (파괴 없이 끝날 때까지 쓰는 비용) + (목표 전에 파괴될 확률 D_s) × (복구 단계부터 다시 쓰는 비용)
```

공통 예제에서 +2의 실패를 "하락 40%, 파괴 20%(→ +1로 복구, 복구비 3,000원)"로 바꿔 봅니다. 파괴를 "게임 끝"으로 치고 계산하면 +1에서 쓰는 비용은 6,000원, 목표 전에 파괴될 확률은 1/3입니다. 복구 단계(+1)부터 다시 쓰는 비용을 X라 하면 X 자신도 같은 식을 따릅니다.

```
X = 6,000 + 1/3 × X   →   X = 6,000 ÷ (1 − 1/3) = 9,000원
"파괴 없이 도달할 확률" = 1 − 1/3 ≈ 66.7%
```

파괴 위험 때문에 기대 비용이 6,000원에서 9,000원으로 1.5배가 됐습니다. 이 1.5가 1 ÷ (1 − D)

> 이고, "복구 단계에서 평균 몇 번 다시 올라가야 하나"를 뜻합니다. D가 1에 가까워지면 이 값이 폭발하므로 **고단계 파괴 확률이나 복구 단계를 조금만 바꿔도 비용이 크게 출렁입니다.** 이 분해가 Sherman–Morrison 공식의 실제 뜻입니다(구현 노트 I.1).

원리와 공식

무엇인가: 다음 상태로 갈 확률이 "지금 상태"에만 달려 있고 과거 경로에는 달려 있지 않은 확률 과정입니다(마르코프 성질). 상태 사이의 이동 확률을 표(전이 행렬 P)로 나타냅니다. **흡수 상태**는 한번 들어가면 나오지 않는 상태(강화의 "목표 도달")이고, 흡수 상태가 있는 체인을 흡수 마르코프 체인이라 합니다.

강화에 맞추는 방법: 강화는 "몇 강인가"만으로는 마르코프가 아닐 수 있습니다(연속 실패 횟수에 따라 보너스·천장이 있음). 그래서 상태를 (단계, 연속 카운터)로 넓혀 마르코프 성질을 만족하게 합니다. 천장이 10회면 단계마다 카운터 0~10의 상태가 생깁니다.

```
흡수되지 않는 상태끼리의 전이 행렬을 Q, 상태별 1회 비용을 r이라 하면
기대 총비용 E는   E = r + Q E    →    (I − Q) E = r
기본 행렬 N = (I − Q)⁻¹ :  N_ij = 상태 i에서 출발해 상태 j를 방문하는 기대 횟수
기대 시도 횟수 = N · 1,    기대 총비용 = N · r
2차 모멘트 S2:  S2_s = Σ_t Q_st [ w_st² + 2 w_st E_t ] + Σ_t Q_st S2_t,    분산 = S2 − E²
```

쉬운 예: "성공 50%면 +1, 실패면 제자리"인 1단계 강화의 기대 시도는 E = 1 + 0.5·E → E = 2회. 하락·파괴가 섞이면 손으로 풀기 어렵지만 같은 원리의 연립방정식입니다.

파괴 없이 도달할 확률: 파괴를 "실패로 끝나는 또 하나의 흡수 상태"로 바꾼 체인에서 목표에 흡수될 확률을 같은 방식으로 풉니다. **돌파 전 파괴**(그 단계에서 출발해 다음 단계에 닿기 전에 파괴될 확률)는 그 단계 이하 상태만의 체인에서 블록 밖으로 나가는 확률을 풉니다.

워크벤치에서 쓰인 곳: 강화 탭의 모든 공식 계산 — 기대 총비용, 표준편차, 기대 시도·파괴, 파괴 없이 도달, 단계별 기대 시도·돌파 전 파괴·비용 비중, 재화별 기대 소모.

한계: 상태 수가 많아지면(단계 × 카운터) 행렬이 커집니다. 구현 노트 I.1의 구조 활용으로 해결합니다. 확률이 시도 횟수에 따라 바뀌는 규칙(예: 100번째 시도마다 이벤트)은 상태에 넣지 않으면 표현할 수 없습니다.

A.2.2 수반(adjoint) 방법을 이용한 민감도

기획 관점으로 풀어 보기

풀려는 문제

"어느 단계 확률을 손대야 총비용이 가장 많이 바뀌나?" 단순한 방법은 단계마다 성공률을 1%p 올려 다시 계산하는 것인데, 15단계면 연립방정식을 15번 더 풀어야 하고 천장·아이템까지 있으면 무거워집니다.

핵심 아이디어 — 방문 횟수 × 한 번 성공의 가치

성공률을 δ만큼 올리면, 그 단계를 방문할 때마다 δ만큼 "실패 대신 성공"이 일어납니다. 그러니 **총비용 변화 ≈ (그 단계 방문 기대 횟수) × δ × (성공했을 때와 실패했을 때 남은 비용의 차이)**입니다. "남은 비용"은 A.2.1에서 이미 구했고, 방문 횟수는 수반 방정식 한 번으로 모든 단계에 대해 같이 나옵니다.

공통 예제로 계산

단계	방문 횟수	성공 vs 실패의 차이	+1%p 추정	실제 다시 풀면
+1	5회	5,500 − 7,500 = −2,000원	5 × 0.01 × (−2,000) = −100원	−98원
+2	2.5회	0 − 7,500 = −7,500원	2.5 × 0.01 × (−7,500) ≈ −188원	−183원

+1을 두 배 더 자주 방문하지만, +2에서의 성공은 "끝"과 "+1로 하락"의 차이(7,500원)라 한 번의 가치가 커서 +2가 더 민감합니다. 실제 강화에서 후반 단계 민감도가 큰 것도 같은 이유입니다. 방문도 많고, 실패의 대가도 큽니다. 방문 횟수의 합(5 + 2.5 = 7.5회) × 1,000원이 기대 총비용 7,500원과 맞는다는 점도 확인할 수 있습니다.

읽는 법·주의

작은 변화를 기준으로 한 근사입니다. 1%p에서는 거의 맞지만(−188 vs −183), 10%p처럼 크게 바꾸면 어긋납니다. 큰 조정은 실제로 표를 고쳐 다시 계산하세요.

원리와 공식

무엇인가: 많은 입력(단계별 성공률)에 대한 하나의 출력(시작 상태의 기대 비용)의 변화율을 한꺼번에

구하는 방법입니다. 입력마다 다시 풀면 단계 수만큼 연립방정식을 풀어야 하지만, 수반 방정식을 한 번 풀면 모든 입력에 대한 변화율이 나옵니다.

```
방문 횟수 벡터  N = (I − P)^−T e_시작        (시작에서 각 상태를 방문하는 기대 횟수)
단계 성공률을 δ(=1%p) 올렸을 때 기대 비용 변화
  dE_시작 ≈ Σ_s N_s · δ · [ E_성공 − (k·E_유지 + d·E_하락 + b·(복구비 + E_복구)) ÷ f ]
  (f = 실패 확률 합, 실패 결과에서 비율대로 δ를 뺀 효과)
```

쉬운 예: +14 단계를 1,000번 방문한다면(N이 큼) 그 단계 확률의 작은 변화가 총비용에 크게 반영됩니다. 그래서 후반 단계일수록 민감도 절댓값이 큽니다.

워크벤치에서 쓰인 곳: 강화 단계 표의 "민감도" 열 (성공률 +1%p 시 총 기대 비용 변화율).

한계: 1차 근사(작은 변화 기준)라 성공률을 10%p처럼 크게 바꿀 때의 변화는 정확히 맞지 않습니다. 큰 변화는 실제로 표를 바꿔 다시 계산해 보세요.

A.2.3 마르코프 결정 과정과 정책 반복법

기획 관점으로 풀어 보기

풀려는 문제

파괴방지권·하락방지권 같은 보조 아이템은 "어느 단계에서, 연속 실패 몇 회일 때 쓰나"에 따라 기대 비용이 달라집니다. 상태가 165개이고 상태마다 쓸지 말지만 골라도 경우의 수가 2^{165}개라 하나씩 비교할 수 없습니다.

핵심 아이디어 — "이번 한 번만 바꿔 보기"를 반복

- ① 지금 규칙(처음엔 "아무것도 안 씀")으로 상태별 기대 비용을 A.2.1처럼 구합니다.
- ② 상태마다 "이번 시도에만 아이템을 쓰고 이후는 지금 규칙대로"의 기대 비용(Q값)을 계산해, 더 싸면 그 상태의 규칙을 바꿉니다.
- ③ 바뀌는 상태가 없을 때까지 반복합니다. 매번 비용이 줄거나 같으므로 반드시 끝나고, 끝난 규칙이 최적입니다.

공통 예제 — 하락방지권의 적정 가격

+2에서 하락을 유지로 바꾸는 하락방지권(가격 X원)이 있다고 합시다. "안 씀" 규칙에서 E_1 = 7,500원,

> E_2 = 5,500원이므로 +2에서의 두 선택은
>
> ```
> 안 씀:  Q = 1,000     + 0.6 × E₁ = 1,000 + 4,500 = 5,500원  (실패하면 +1로)
> 씀:     Q = 1,000 + X + 0.6 × E₂ = 4,300 + X 원          (실패해도 +2에 남음)
> ```
>
> **X < 1,200원이면 +2에서 쓰는 쪽이 쌉니다.** 이 아이템을 1,200원보다 싸게 팔면 합리적인 유저에게는 "필수품"이 되고, 비싸게 팔면 아무도 안 씁니다. 워크벤치의 아이템 가격 곡선은 이 계산을 가격마다 반복해, 가격에 따라 사용 여부·절감률·BM 매출이 어떻게 바뀌는지 그립니다. 기본 템플릿의 "최적 정책으로 52.8% 절감"은 아이템이 너무 싸다는 신호로 읽습니다(2.10절).
>
> **주의**
>
> "유저는 기대 비용을 최소화한다"는 가정입니다. 실제 유저는 파괴가 무서워 손해를 보면서도 쓰거나, 아이템을 아끼기도 합니다. 결과는 "합리적 유저 기준의 기준선"으로 읽으세요.

원리와 공식

무엇인가: 마르코프 결정 과정(MDP)은 마르코프 체인에 "상태마다 고르는 행동"을 더한 모델입니다. 행동에 따라 다음 상태로 갈 확률과 비용이 바뀝니다. 상태 → 행동 규칙을 **정책**이라 하고, 목표는 기대 총비용을 최소로 하는 정책을 찾는 것입니다.

```
정책 평가:  지금 정책 π로 (I − P_π) E = r_π 를 풀어 상태별 기대 비용 E
정책 개선:  각 상태 s에서 Q(s, a) = r(s, a) + Σ_t P(t | s, a) · E_t  가 가장 작은 행동 a로 바꿈
반복:       바뀌는 행동이 없으면 그 정책이 최적 (유한 상태·행동이면 유한 번 안에 끝남)
```

쉬운 예: +14 단계, 연속 실패 0회 상태에서 "파괴방지권 사용"은 이번 시도 비용을 올리지만 파괴 후 복구라는 큰 비용을 없앱니다. Q값은 "이번에 이 행동을 하고 이후는 지금 정책대로" 했을 때의 기대 비용이고, 이것이 더 작으면 행동을 바꿉니다.

워크벤치에서 쓰인 곳: 강화의 보조 아이템 최적 사용 정책, 아이템 가격 곡선(가격 점마다 다시 풂). 계산량을 줄이는 방법은 구현 노트 I.2.

한계: "유저가 기대 비용을 최소화한다"는 가정입니다. 실제 유저는 위험을 싫어하거나(파괴가 무서워 과하게 씀) 아이템을 아끼기도 합니다.

A.2.4 목표 비용 역산 — 곱셈 분배와 범위 넘김

기획 관점으로 풀어 보기

풀려는 문제

"평균 비용을 7,500원에서 5,000원으로"는 숫자 하나지만 만드는 방법은 여럿입니다. 성공률을 올리면 유저가 느끼는 운이 좋아지고, 1회 비용을 내리면 재화 소모량이 줄어듭니다. 기획 의도에 따라 둘의 비중을 정하고, 그 비중대로 목표에 정확히 닿는 확률·비용을 찾아야 합니다.

공통 예제 — 7,500원을 5,000원으로 (F = 1.5)

가중치 w	실제 배율 s	+1 성공률	+2 성공률	1회 비용	기대 비용
0 (비용만)	1.5000	50%	40%	666.7원	5,000원
0.5	1.3581	58.27%	46.61%	858.1원	5,000원
1 (확률만)	1.2808	64.04%	51.23%	1,000원	5,000원

비용만 바꾸면 기대 비용이 비용에 정비례하므로 s = F = 1.5입니다. 확률을 올릴 때는 s가 F보다 작아도 됩니다(1.28). +2의 성공률이 오르면 "+1로 하락했다가 다시 올라오는" 경로까지 줄어, 같은 배율이라도 기대 비용이 더 많이 내려가기 때문입니다. 가운데(w = 0.5)는 확률과 비용에 같은 배율 √1.3581 = 1.1654씩을 줍니다.

범위에 걸리면 반대쪽으로 넘긴다

확률 최대를 55%로 두면 w = 0.5의 +1 성공률 58.27%가 범위를 넘습니다. 이때 +1은 55%(×1.1)에 고정하고, 모자란 몫을 비용으로 넘겨 1회 비용을 1,000원 ÷ (1.3581 ÷ 1.1) = 810원으로 더 내립니다. 단계마다 확률 배율 × 비용 배율 = s가 그대로라 목표 5,000원에 그대로 닿습니다. 확률과 비용이 둘 다 범위에 막히면 그 단계는 거기까지만 바꾸고, 만들 수 있는 기대 비용 범위를 알려 줍니다.

반올림 오차

계산된 비용 858.1원을 100원 단위로 반올림하면 900원이 되어 기대 비용이 5,244원(+4.9%), 내림하면 800원으로 4,662원(−6.8%)입니다. 도구는 반올림한 표로 기대 비용을 다시 계산해 오차를 함께 보여 주므로, 단위를 고를 때 오차가 허용 범위인지 바로 확인할 수 있습니다.

원리와 공식

곱셈 분배: 필요한 개선 배율 F = 지금 기대 비용 ÷ 목표를 확률 배율 Fp와 비용 배율 Fc의 곱으로 나눕니다. 가중치 w를 지수로 써서 Fp = s^w, Fc = s^(1−w)로 두면 w = 0이면 비용만, 1이면 확률만, 그 사이는 비율대로가 되고, 언제나 Fp × Fc = s입니다.

```
새 확률(단계마다) = 확률 × Fp      (성공·대성공에 곱하고, 실패 결과는 비율 유지)
새 비용(단계마다) = 비용 ÷ Fc
Fp를 [최소, 최대]로 자르면 Fc = s ÷ Fp,    Fc를 자르면 Fp = s ÷ Fc
```

s 찾기: 하락·파괴·연속 보정이 없는 구조라면 s = F입니다. 일반적으로는 구간의 모든 단계에 같은 s를 적용한 표의 기대 비용을 마르코프 체인(A.2.1)으로 계산하고, s가 커질수록 기대 비용이 줄어드는 단조성을 이용해 로그 이분 탐색(A.1.4)으로 s를 찾습니다. 시작 범위는 F ÷ 4 ~ F × 4이고, 목표를 감싸지 못하면 4배씩 넓힙니다.

워크벤치에서 쓰인 곳: 강화 목표 비용 역산(2.10절)과 강화 기획서 영역의 역산 설명.

한계: 구간 안의 모든 단계에 같은 배율을 줍니다. "고단계만 더 많이" 같은 모양 변경은 구간을 나눠 두 번 역산하세요. 반올림 단위는 모든 재화에 똑같이 적용됩니다.

A.3 가챠·수집 설계 (3장)

가챠 탭은 목표의 성격에 따라 계산 방법을 바꿉니다. 한 장이나 세트를 모으는 기본 목표는 기하분포·쿠폰 수집 공식으로 풀고(A.3.1), 천장·반천장·픽업이 얽힌 목표는 동적 계획법으로 정확한 분포를 구하며(A.1.3), 소환 레벨이 여러 개이거나 조각 교환이 있어 공식으로 풀 수 없는 목표만 시뮬레이션으로 계산합니다(A.1.1).

기획 질문	방법	절
한 장 뽑는 데 몇 번? 세트를 다 모으는 데 몇 번?	기하분포·쿠폰 수집·포아송화	A.3.1
천장·반천장·픽업이 있을 때 k장 목표의 분포는?	동적 계획법	A.1.3
공식으로 풀 수 없는 목표(소환 레벨, 조각 교환)는?	시뮬레이션	A.1.1
예산별 달성 확률, 운 나쁜 10%의 뽑기 수는?	분위수	A.1.2
10연 보장이 있으면 등급별 실제 비율은?	10번 묶음의 평균 비율	A.3.2

A.3.1 기하분포, 포아송화, 쿠폰 수집 문제

기획 관점으로 풀어 보기

풀려는 문제

가챠에서 기획자가 묻는 질문은 크게 둘입니다. "이 등급 하나 뽑는 데 평균 몇 번?"(기하분포)과 "이 세트를 다 모으는 데 몇 번?"(쿠폰 수집). 둘째가 첫째보다 훨씬 크고, 그 차이를 직감보다 정확히 알아야 합니다.

기하분포 — 하나 뽑기

확률 p인 뽑기를 당첨될 때까지 하면 평균 1/p번입니다. 0.02%면 5,000번. 하지만 평균이 "보통"은 아닙니다. 기하분포도 꼬리가 길어서

확률 0.02%	뽑기 수
중앙값 (절반이 이 안에 뽑음)	3,466번
평균	5,000번
90% (10명 중 1명은 이보다 더)	11,512번

"평균 5,000번"이라고 공시해도 10명 중 1명은 11,500번 넘게 뽑습니다. 천장은 이 꼬리를 자르는 장치입니다.

쿠폰 수집 — 다 모으기

10종이 같은 확률로 나올 때 모두 1장씩 모으려면 평균 $10 \times (1 + 1/2 + \dots + 1/10) \approx$ **29.3번**입니다. 10번이 아닌 이유는 중복입니다. 처음엔 뭘 뽑아도 새것이지만, 9종을 모은 뒤엔 마지막 1종이 나올 확률이 1/10이라 그것 하나에 평균 10번이 듭니다.

몇 번째 새 종	새것이 나올 확률	평균 뽑기
1번째	10/10	1번
5번째	6/10	약 1.7번
9번째	2/10	5번
10번째(마지막)	1/10	10번

마지막 몇 종이 비용의 대부분을 차지합니다. 등급 확률이 p면 $n \cdot H_n \div p$로, SSR 3% 안에 10종이면

> 약 976번입니다. 도감·세트 보상을 "종 수 × 1장 기대 뽑기"(10 × 33.3 = 333번)로 계산하면 실제의 3분의 1로 과소평가합니다.

포아송화 — "종마다 k장씩"을 공식으로 푸는 요령

"10종을 각각 3장씩"처럼 장수가 여러 개면 종별 개수가 서로 얽혀(합이 뽑기 횟수와 같아야 함) 직접 계산이 어렵습니다. 그래서 "뽑기가 정해진 횟수가 아니라 무작위 시각에 평균 1초에 1번 일어난다"고 바꿔 생각합니다. 그러면 종마다 나오는 횟수가 서로 영향을 주지 않는 독립 과정이 되어, **모든 종이 k장 이상일 확률 = (한 종이 k장 이상일 확률)^종 수**로 간단해집니다. 이렇게 바꿔도 "완료까지 기대 시간 = 완료까지 기대 뽑기 수"라서 답은 같습니다. 남은 적분을 수치로 계산하는 방법은 구현 노트 I.4에 있습니다.

원리와 공식

기하분포: 성공 확률 p인 시도를 성공할 때까지 반복할 때 걸리는 횟수 G의 분포. P(G = n) = (1 − p)^(n−1)·p, 평균 1/p. 레전드 확률 0.02%면 평균 5,000회.

쿠폰 수집 문제: n종이 같은 확률로 나올 때 모두 1장 이상 모으는 기대 횟수는 $n \cdot H_n$ ($H_n = 1 + 1/2 + \dots + 1/n$). 이미 모은 종이 또 나오는 중복 때문에 n보다 훨씬 큽니다(n = 10이면 약 29.3회). 등급 확률 p 안에서 n종이면 $n \cdot H_n / p$.

포아송화: "뽑기 횟수 n이 정해진" 문제는 종별 개수가 서로 얽혀(합이 n) 계산이 어렵습니다. 대신 뽑기가 "단위 시간당 1회 비율로 무작위 시각에" 일어난다고 바꾸면, 각 종이 나오는 횟수가 서로 독립인 포아송 과정이 됩니다. 이렇게 바꿔도 "완료까지 기대 시간 = 완료까지 기대 뽑기 수"가 성립합니다.

```
E[N] = (1/λ) ∫₀^∞ [ 1 − P(k, u)^K ] du
  λ = 등급 확률 ÷ 종 수,  K = 모을 종 수,  k = 종당 필요 장수
  P(k, u) = 평균 u인 포아송 값이 k 이상일 확률 = 정칙화 하부 불완전 감마 함수
```

워크벤치에서 쓰인 곳: 수집 목표의 공식 계산(포아송화 적분). 기하분포는 공식으로 풀 수 없는 가챠 목표의 시뮬레이션에서 "다음 당첨까지 몇 번"을 뽑는 데도 쓰입니다(구현 노트 I.3). 적분 계산과 검증은 구현 노트 I.4.

A.3.2 10연 보장의 실제 등급 비율

기획 관점으로 풀어 보기

풀려는 문제

"10연에 SR 이상 1장 보장"을 넣으면 공시 확률(SR 8%, SSR 0.8%)과 유저가 실제로 받는 비율이 달라집니다. 확률 공시와 목표 비용 계산에는 보장까지 반영한 실제 비율이 필요합니다.

공통 예제 — R 91.2% · SR 8% · SSR 0.8%, SR 이상 보장

등급	표기 확률	실제 1회당	계산
R	91.200%	87.219%	(9 × 0.912 + 0.563 × 0.912) ÷ 10
SR	8.000%	11.619%	(9 × 0.08 + 0.563 × 0.08 + 0.437 × 0.08 ÷ 0.088) ÷ 10
SSR	0.800%	1.162%	(9 × 0.008 + 0.563 × 0.008 + 0.437 × 0.008 ÷ 0.088) ÷ 10

보장 등급 이상 확률의 합 q = 8.8%, 앞 9번이 모두 R일 확률 a = 0.912^9 = 43.7%입니다. 10번째 뽑기는 43.7%의 경우 R이 나올 수 없고 SR·SSR이 원래 비율(8 : 0.8)대로 나옵니다. 열 번을 평균하면 SR·SSR이 각각 원래의 1.45배가 됩니다.

보장은 목표 비용도 바꿉니다. "SR 도감(20종)" 기대 뽑기는 899회에서 620회(−31%), "SSR 도감(12종)"은 1,361회에서 1,312회(−4%)가 됩니다. SR 목표는 크게, SSR 목표는 조금 줄어드는 것은 보장이 SSR에 더해 주는 몫이 0.36%p뿐이기 때문입니다.

원리와 공식

무엇인가: 10번째 뽑기의 분포가 앞 9번의 결과에 따라 달라지는 규칙을, 10번을 한 묶음으로 보고 묶음 안의 평균 비율로 바꾼 것입니다(기대값의 선형성).

```
q = Σ (보장 등급 이상 확률),   a = (1 − q)⁹
보장 등급 이상:  r = (9p + (1 − a)·p + a·p ÷ q) ÷ 10
그 아래:         r = (9p + (1 − a)·p) ÷ 10
```

워크벤치에서 쓰인 곳: 가챠의 고정 확률 결과·소환 레벨 결과의 실제 비율, 레벨 구간별 등급 분포, 기획서 영역(3.5절). 목표 비용은 이 평균 비율이 아니라, 10번째마다 보장을 실제로 적용하는 한 번씩 뽑기 시뮬레이션으로 계산합니다(보장 등급 이상의 목표일 때).

한계: 평균 비율은 천장을 빼고 계산합니다. 최상위 등급의 실효 확률은 천장까지 포함한 확률 공시표(3.10절)의 값을 보세요.

A.4 스탯·전투 밸런스 (4장)

스탯·전투 탭의 계산은 한 줄로 이어집니다. 먼저 "몇 번째 공격에 처치하나"의 분포를 동적 계획법으로 구하고(A.1.3), 두 분포를 결합해 1:1 승률을 공식으로 냅니다(A.4.1). 데미지 변동·특성이 있어 시뮬레이션으로 구한 승률에는 오차 범위를 붙이고(A.4.2), CP 순서가 실제 강함 순서와 맞는지 확인합니다(A.4.3). 마지막으로 밸런스 도구가 어느 요소를 고칠지 보여 줍니다. 특성·아이템을 하나씩 꺼 보는 효과 분해와 스탯별 민감도는 반대 가정·토네이도(A.4.4), 목표 승률에 맞추는 최소 변경은 이분 탐색·공통 난수(A.1.4)입니다.

계산 순서	방법	절
① 상대를 몇 번째 공격에 처치하나	동적 계획법	A.1.3
② 1:1 승률 (변동·특성 없음)	독립 확률변수 결합	A.4.1
③ 시뮬레이션 승률의 오차 범위	이항분포·Wilson 구간	A.4.2
④ CP가 실제 강함 순서와 맞나	스피어만 순위 상관	A.4.3
⑤ 무엇이 승패를 좌우하나, 무엇부터 고칠까	반대 가정·토네이도 민감도	A.4.4
⑥ 목표 승률에 맞추는 최소 변경	이분 탐색·공통 난수	A.1.4
⑦ 실제로 싸우면 선공·남은 체력·오버킬은 (배틀 시뮬레이터)	몬테카를로 시뮬레이션	A.1.1

A.4.1 독립 확률변수로 1:1 승률 계산

기획 관점으로 풀어 보기

풀려는 문제

승률 행렬의 모든 칸을 시뮬레이션으로 채우면 칸마다 수천 번 전투가 필요하고 오차가 남습니다. 데미지 변동·특성이 없는 대결은 공식으로 정확히 풀 수 있습니다.

핵심 아이디어 — 각자 몇 대 만에 잡나

A가 B를 쓰러뜨리는 데 필요한 공격 수 N_A는 A의 공격·치명·명중과 B의 방어·체력으로만 정해지고,

B가 A를 몇 대 만에 잡는지와는 상관없습니다(독립). 그래서 둘을 따로 구한 뒤, **"A가 n번째 공격으로 잡는 시각 전에 B가 아직 못 잡았을 확률"**을 모든 n에 대해 더하면 승률입니다. N_A의 분포는 A.1.3의 동적 계획법으로 구합니다.

예

```
A: 1초에 1회 (0, 1, 2, 3초 …). 3대(50%) 또는 4대(50%, 치명 여부)에 B 처치
B: 1초에 0.8회 (0, 1.25, 2.5초 …). 항상 3대에 A 처치

A가 3대(2초)에 잡는 경우: 그 전 B의 공격은 2번 → B는 아직 못 잡음 → A 승
A가 4대(3초)에 잡는 경우: 2.5초에 B의 3번째 공격 → B 승
P(A 승) = 0.5 × 1 + 0.5 × 0 = 50%
```

"공격 속도가 빠른 A가 치명이 터지느냐 하나로 승패가 갈린다"는 사실이 계산 과정에 그대로 보입니다. 이런 분해는 "왜 이 매치업이 50%인가"를 설명할 때 유용합니다.

원리와 공식

아이디어: A가 B를 쓰러뜨리는 데 필요한 공격 횟수 N_A와, B가 A를 쓰러뜨리는 데 필요한 횟수 N_B는 데미지 변동·특성이 없으면 서로 영향을 주지 않는(독립) 확률변수입니다. 그래서 "A가 n번째 공격으로 쓰러뜨리고, 그 전에 B가 못 쓰러뜨릴 확률"을 곱해 모든 n에 대해 더하면 승률이 됩니다.

```
시간 기반:  P(A 승) = Σₙ P(N_A = n) · P(N_B > m_B(n))
            m_B(n) = A의 n번째 공격 시각 전에 B가 하는 공격 수,  n번째 공격 시각 = (n − 1) ÷ 공격 속도
           (같은 시각이면 "동시 결정타" 설정: 반반 또는 무승부)
턴 교대:    P(A 승) = Σₙ P(N_A = n) · [ 선공 · S_B(n − 1) + (1 − 선공) · S_B(n) ],  S_B(m) = P(N_B > m)
무승부:     최대 시간·턴 안에 둘 다 못 쓰러뜨릴 확률
```

검증: Kalivra v1.0.1 엔진을 그대로 돌린 결과(턴 교대, 40만~140만 회)와 비교해 세 경우 모두 3σ 안에서 일치합니다 — K1 0.8(정확히), K2 0.72643 대 0.72676, K3 0.02640 대 0.02659. 시간 기반 공식은 무작위 설정 12개 × 두 방식에서 자체 시뮬레이션과 일치합니다.

워크벤치에서 쓰인 곳: 스탯·전투의 승률 행렬, 1:1 상세, 레벨별 승률, CP 대 승률, 밸런스 도구 (변동·특성이 없는 대결).

A.4.2 이항분포와 Wilson 점수 구간

기획 관점으로 풀어 보기

풀려는 문제

시뮬레이션 승률 52%가 진짜 우위인지 운인지 판단하려면 승률에 오차 범위가 필요합니다.

같은 52%라도 횟수에 따라 결론이 다릅니다

전적	95% 범위 (Wilson)	판단
52승 / 100전	42.3% ~ 61.5%	50% 포함 — 우위라 말할 수 없음
520승 / 1,000전	48.9% ~ 55.1%	아직 50% 포함
5,200승 / 10,000전	51.0% ~ 53.0%	50% 밖 — 우위로 판단

범위 폭은 횟수의 제곱근에 반비례하므로, 폭을 절반으로 줄이려면 4배를 더 돌려야 합니다. 범위가 50%를 포함하면 "아직 차이가 있다고 말할 수 없음"으로 읽으세요.

왜 Wilson 구간인가

흔한 공식 $\hat{p} \pm 1.96\cdot\sqrt{\hat{p}(1-\hat{p})/n}$은 승률이 0%·100%에 가까우면 망가집니다. 20전 20승이면 범위 폭이 0이 되어 "승률 100%가 확실하다"고 말하는 셈입니다. Wilson 구간은 같은 경우 **83.9% ~ 100%**로, "20번으로는 승률이 84% 이상이라는 것까지만 안다"고 정직하게 답합니다. 밸런스 작업에서는 압도적 상성 같은 극단적 매치업을 자주 보므로 이 차이가 중요합니다.

원리와 공식

이항분포: 성공 확률 p인 독립 시행 n번에서 성공 횟수의 분포. 시뮬레이션 승률 $\hat{p}$ = 이긴 횟수 ÷ n의 표준오차는 $\sqrt{p(1-p)/n}$.

Wilson 점수 구간: 정규근사 구간 $\hat{p} \pm 1.96\cdot\sqrt{\hat{p}(1-\hat{p})/n}$은 $\hat{p}$가 0이나 1에 가까울 때 0~1을 벗어나거나 너무 좁아집니다($\hat{p}$ = 0이면 폭 0). Wilson 구간은 "참값 p에서 $\hat{p}$가 나올 만한 범위"를 거꾸로 풀어 얻은 구간이라 항상 0~1 안에 있고 실제 포함 확률이 95%에 가깝습니다.

$$\text{중심} = (\hat{p} + z^2/2n) \div (1 + z^2/n)$$

$$\text{반폭} = z \cdot \sqrt{\hat{p}(1-\hat{p})/n + z^2/4n^2} \div (1 + z^2/n), \quad z = 1.96\ (95\%)$$

워크벤치에서 쓰인 곳: 스탯·전투 시뮬레이션 승률의 95% 범위, 승률 행렬 칸의 ±오차, 1:1 상세의

시뮬레이션 열, 승률 수렴 그래프의 띠.

A.4.3 스피어만 순위 상관

기획 관점으로 풀어 보기

풀려는 문제

전투력(CP)은 "높을수록 강하다"는 약속입니다. 그 약속이 실제 전투 결과와 맞는지 확인하려는데, CP 값의 크기(1만 vs 1.2만)가 아니라 **순서**가 맞는지가 핵심입니다.

방법 — 순위로 바꿔서 비교

캐릭터	CP 순위	실제 강함 순위	차이 d
전사	1	1	0
궁수	2	2	0
법사	3	4	−1
도적	4	3	+1

```
ρ = 1 − 6·Σd² ÷ (n(n² − 1)) = 1 − 6 × 2 ÷ (4 × 15) = 0.8
```

1이면 순서가 완벽히 같고, 0이면 무관, −1이면 정반대입니다.

읽는 법

캐릭터가 4명이면 한 쌍만 뒤집혀도 0.8로 떨어지고, 40명이면 한 쌍이 뒤집혀도 0.9998입니다. 값 자체보다 "CP가 높은데 지는 쌍" 목록을 함께 보세요. 그 쌍이 CP 공식이 놓치는 요소(공격 속도, 치명, 상성)를 가리킵니다.

원리와 공식

무엇인가: 두 값의 **순위**가 얼마나 같은 순서인지 재는 상관계수입니다. 값을 순위(1등, 2등 …)로 바꾼 뒤 순위끼리의 피어슨 상관을 구합니다. 값의 크기 차이가 아니라 순서만 보므로, "CP 순서가 실제 강함 순서와 맞나"를 묻기에 알맞습니다.

```
ρ = 순위끼리의 피어슨 상관 = Σ(rx − r̄x)(ry − r̄y) ÷ √( Σ(rx − r̄x)² · Σ(ry − r̄y)² )
동점은 평균 순위 (2등·3등 동점이면 둘 다 2.5등)
동점이 없으면 ρ = 1 − 6 Σ d² ÷ ( n(n² − 1) )
```

읽는 법: 1 = 순서 완전히 같음, 0 = 관계 없음, −1 = 정반대. 엔티티가 4개면 한 쌍만 뒤바뀌어도 0.8이 됩니다. 엔티티가 적을수록 한 쌍의 영향이 크므로, 값보다 "CP가 높은데 지는 쌍" 목록을 함께 보세요.

워크벤치에서 쓰인 곳: 스탯·전투 KPI "CP와 실제 강함의 순위 상관", 브리핑 지표.

A.4.4 반대 가정과 토네이도 민감도

기획 관점으로 풀어 보기

반대 가정 — 원인을 "바꿔 보고" 판정

"이 캐릭터가 이기는 게 특성 때문인가, 스탯 때문인가?"는 승률 숫자를 나란히 놓아서는 알기 어렵습니다. 특성과 스탯이 함께 작용하기 때문입니다. 반대 가정은 의심 요소를 실제로 꺼서(또는 바꿔서) 다시 계산하고, 결과가 얼마나 달라지는지로 그 요소의 몫을 판정합니다. 스탯·전투 탭의 아이템·특성 효과 분해가 이 방식입니다. 상관관계가 아니라 **"이것을 빼면 이만큼 달라진다"**는 답입니다.

토네이도 — 무엇부터 손댈지

변수를 하나씩 ±10% 바꾸고 나머지는 그대로 둔 채 결과 변화를 막대로 그립니다. 공통 예제(평균 7,500원)에 적용하면

변수 (±10%)	결과 범위	막대 길이
+2 성공률 (36% ~ 44%)	8,333 ~ 6,818원	1,515원
시도 비용 (900 ~ 1,100원)	6,750 ~ 8,250원	1,500원
+1 성공률 (45% ~ 55%)	8,056 ~ 7,045원	1,010원

같은 10%라도 +2 성공률이 +1 성공률보다 1.5배 영향이 큽니다(A.2.2의 민감도와 같은 결론). 막대가 긴 변수부터 위에 쌓으면 회오리 모양이 되고, 맨 위가 가장 효율적인 조정 손잡이입니다. 스탯·전투 밸런스 도구의 민감도는 같은 방식으로 "어느 스탯을 바꾸면 승률이 가장 많이 움직이나"를 보여 줍니다.

한계

한 번에 하나씩만 바꾸므로 두 변수를 함께 바꿀 때의 상호작용은 보이지 않습니다. ±10%는 작은 변화라 크게 바꾸면 순서가 달라질 수 있습니다.

원리와 공식

반대 가정(counterfactual): "만약 이 부분이 달랐다면 결과가 어땠을까"를 실제로 바꿔 다시 계산해 보는 방법입니다. 상관관계나 숫자 비교만으로 원인을 추정하는 것보다 직접적입니다. 스탯·전투 탭은 아이템·특성을 하나씩 끄고 다시 계산해, 각 요소가 승률·처치 시간에 얼마나 기여하는지 보여 줍니다.

토네이도(일변수) 민감도: 변수를 하나씩 ±p% 바꾸고 나머지는 그대로 둔 채 결과 변화를 막대로 그립니다. 영향이 큰 변수부터 위에 쌓아 회오리 모양이 됩니다.

한계: 한 번에 하나만 바꾸므로 두 변수를 함께 바꿀 때의 상호작용은 보이지 않습니다. 또 ±p%는 국소적 변화라, 크게 바꾸면 순위가 달라질 수 있습니다.

워크벤치에서 쓰인 곳: 스탯·전투 밸런스 도구의 민감도, 아이템·특성 효과 분해(하나씩 끄기).

A.5 데이터 테이블 (5장)

데이터 테이블은 확률 계산이 없습니다. 테이블끼리의 ID 참조가 끊기지 않았는지 확인하고(A.5.1), 직업·레벨·보정을 정해진 순서로 합쳐 전투 시작 스탯을 만듭니다(5.5절).

A.5.1 참조 무결성 검사

기획 관점으로 풀어 보기

풀려는 문제

기획 엑셀은 시트 여러 장이 ID로 이어진 작은 데이터베이스입니다. 스킬 시트에서 SK_SLAM을 지우거나 이름을 바꾸면, 그 스킬을 쓰던 몬스터와 보스 페이즈가 조용히 깨집니다. 깨진 참조는 전투를 돌려 봐야 "보스가 스킬을 안 쓴다"처럼 엉뚱한 증상으로 드러납니다.

방법 — ID를 먼저 모두 모으고, 참조를 하나씩 대조

검사는 두 번 훑습니다. 첫 번째로 일곱 개 테이블의 ID를 테이블별 목록에 모읍니다(같은 테이블에서 두 번 나오면 중복 ID 오류). 두 번째로 모든 행의 참조 칸(ClassID, 속성, 스킬·효과 목록, 페이즈 안의 스킬, 시너지 조건 값)을 하나씩 그 목록에서 찾습니다. 5.6절의 예에서는 이렇게 CL_WARIOR(오타), EF_STUNN(오타), 중복 CH_BRIGID가 잡혔습니다.

ID 바꾸기 — 참조를 함께 고치기

ID를 바꿀 때는 그 테이블을 가리키는 열을 테이블 정의에서 찾아(예: 스킬 → 직업·캐릭터·몬스터의 스킬 열, 몬스터 페이즈) 같은 ID를 모두 새 ID로 바꿉니다. 페이즈 문자열은 "=", "|", ";" 기준으로 나눠 ID 조각만 바꾸므로, SK_SLAM이 SK_SLAM2의 앞부분이어도 SK_SLAM2가 잘못 바뀌지 않습니다.

원리와 공식

참조 무결성(referential integrity): 다른 표를 가리키는 값이 실제로 그 표에 있어야 한다는 데이터베이스의 규칙입니다. ID 목록을 한 번 만들어 두면 참조 하나를 찾는 데 행 수와 상관없이 일정한 시간이 걸려, 행이 수천 개여도 검사가 즉시 끝납니다.

워크벤치에서 쓰인 곳: 데이터 테이블 검증 결과, 빨간 칸·배지, ID 이름 바꾸기, 전투 시뮬레이터의 "테이블에 없는 ID" 알림.

한계: 값이 "있는지"만 봅니다. 힐러에게 공격 스킬만 있다거나 보스 페이즈에 광역기가 없다는 식의 설계 의도는 검사하지 않습니다.

A.6 전투 시뮬레이터 (6장)

전투 시뮬레이터는 데이터 테이블의 규칙대로 전투를 한 판씩 진행하는 이산 사건 시뮬레이션(A.6.2)을 수백 번 반복하고(A.1.1), 결과를 분위로 읽습니다(A.1.2). 한 대의 데미지는 고른 데미지 공식(A.6.1)으로 계산하고, 누가 얼마나 기여했는지는 효과를 빼거나 사람을 바꿔 다시 계산하는 반대 가정(A.6.3, A.4.4)으로 봅니다.

기획 질문	방법	절
방어력이 오르면 데미지가 어떤 모양으로 줄어드나?	데미지 공식 네 가지	A.6.1
턴제와 실시간에서 결과가 왜 다른가?	이산 사건 시뮬레이션	A.6.2
승률·클리어 시간은 어떻게 퍼져 있나?	몬테카를로·분위수	A.1.1, A.1.2
누가 승패를 가르나? 버퍼의 몫은?	반대 가정(효과 빼기·교체)	A.6.3
조합·설정 비교가 운에 흔들리지 않으려면?	공통 난수	A.1.4

A.6.1 데미지 공식 네 가지 — 방어력이 데미지를 줄이는 모양

기획 관점으로 풀어 보기

풀려는 문제

데미지 공식은 "방어력 1의 가치"를 정합니다. 같은 스탯 테이블이라도 공식에 따라 방어력을 2배로 올렸을 때 받는 피해가 27% 줄 수도, 75% 줄 수도 있습니다. 성장 곡선과 인플레이션을 정하기 전에 공식의 모양을 알아야 합니다.

공통 예제 — 아렌(공격력 279)이 오크 전사(방어력 60)를 때릴 때

공식	지금 (공 279, 방 60)	방어만 2배 (방 120)	둘 다 2배 (공 558, 방 120)
상수 감소형 (K = 100)	174.4	126.8 (×0.73)	253.6 (×1.45)
단순 차감형 (최소 10%)	219.0	159.0 (×0.73)	438.0 (×2.00)
비율 감소형	229.6	195.1 (×0.85)	459.2 (×2.00)
곱연산형 (상한 90%)	111.6	27.9 (×0.25)	55.8 (×0.50)

방어만 2배로 올리면 상수 감소형·차감형은 데미지가 27% 줄고, 비율 감소형은 15%만 줄며, 곱연산형은 방어 120이 상한 90%에 걸려 75%나 줄어듭니다. 공격과 방어를 함께 2배로 올리는 인플레이션에서는 비율 감소형과 차감형만 데미지가 정확히 2배가 되고, 상수 감소형은 1.45배, 곱연산형은 오히려 절반이 됩니다. 성장으로 숫자가 커지는 게임에서 K를 고정한 상수 감소형을 쓰면, 후반으로 갈수록 방어력 1의 가치가 떨어지고 같은 비율로 자란 공격이 상대적으로 강해진다는 뜻입니다.

전투 결과로 보면

같은 1:1(아렌 Lv30 대 오크 전사, 턴제 300판)의 평균 클리어는 상수 감소형 23.4턴, 차감형 19.0턴, 비율 감소형 18.2턴, 곱연산형 34.3턴입니다. 승률은 모두 100%라 승패만 보면 차이가 없지만, 콘텐츠 시간과 보스 체력 설계는 공식에 따라 두 배 가까이 달라집니다.

원리와 공식

```
상수 감소형:  공 × K ÷ (K + 방)              방 = K이면 절반, 방이 아무리 커도 0이 되지 않음
단순 차감형:  max(공 − 방, 공 × 최소%)
비율 감소형:  공 × 공 ÷ (공 + 방) = 공 × 1 ÷ (1 + 방 ÷ 공)
```

```
곱연산형:     공 × (1 − min(방, 상한) ÷ 100)
```

동차성: 비율 감소형은 공과 방을 같은 배율 c로 키우면 데미지도 정확히 c배가 됩니다(1차 동차 함수 — 방 ÷ 공 비율만의 함수에 공을 곱한 꼴). 그래서 공·방이 같은 비율로 자라는 성장형 게임에서 "레벨이 올라도 몇 대 만에 잡는지"가 유지됩니다. 상수 감소형은 K가 고정이라 방어가 커질수록 감소율 방 ÷ (K + 방)이 1에 다가가고, 차감형은 방이 공에 가까워지면 최소 데미지 하한에 붙습니다.

워크벤치에서 쓰인 곳: 전투 시뮬레이터의 데미지 공식, 데미지 계산 순서 예시, 기획서 영역. 스탯·전투 탭(4.5절)은 같은 모양의 식을 규칙 수식으로 직접 씁니다.

한계: 데미지 변동(±%)과 방어 관통은 없습니다. 필요하면 스탯·전투 탭의 규칙 수식으로 따로 검토하세요.

A.6.2 이산 사건 시뮬레이션 — 턴제와 실시간

기획 관점으로 풀어 보기

풀려는 문제

스킬 쿨타임, 1초마다 들어가는 화상, 2턴 동안의 스턴, 체력 70%에서 바뀌는 보스 패턴처럼 "언제" 일어나는지가 중요한 규칙은 평균 데미지만으로 계산할 수 없습니다. 사건이 일어나는 순서대로 전투를 실제로 진행해야 합니다.

공통 예제 — 같은 파티, 두 방식

항목	턴제	실시간
승률 (300판)	31.7%	88.3%
평균 클리어	61.0턴	44.9초
핀(속도 145)의 한 판 행동 수	47.2회	57.6회
화염 골렘(속도 95)의 한 판 행동 수	52.0회	36.4회

턴제는 라운드마다 모두가 한 번씩 움직이고 속도는 순서만 정합니다. 그래서 핀과 골렘이 비슷한 횟수로 행동합니다. 실시간은 초당 행동 = 속도 ÷ 100이라 핀은 1초에 1.45번, 골렘은 0.95번 움직이고, 흐트러짐(그로기) 동안 골렘이 쉬면서 차이가 더 벌어집니다. 같은 테이블에서 승률이 세 배 가까이 차이 나는 이유입니다.

원리와 공식

이산 사건 시뮬레이션: 시간을 잘게 쪼개 매 순간을 계산하는 대신, 상태가 바뀌는 "사건"(행동, 1초 틱, 효과 만료)만 시간 순서대로 처리하는 방법입니다.

```
턴제:   라운드마다 살아 있는 유닛을 속도 내림차순 정렬 → 각 유닛:
        지속 피해 → 자원 회복 → 쿨타임 −1 → 행동(스턴·흐트러짐이면 건너뜀) → 내 효과 지속 −1
실시간: 다음 사건 = min(가장 이른 유닛 행동 시각, 다음 1초 틱)
        행동:   다음 행동 시각 = 지금 + 1 ÷ (속도 ÷ 기준 속도)
        1초 틱: 지속 피해 · 자원 회복 · 만료된 효과 정리
```

실시간의 첫 행동 시각에는 ±10%의 무작위 어긋남을 주어 모든 유닛이 0초에 동시에 움직이지 않게 합니다. 명중·치명타·효과 발동·랜덤 타겟팅도 같은 시드의 난수를 쓰므로 판마다 다르지만, 시드가 같으면 그대로 재현됩니다.

워크벤치에서 쓰인 곳: 전투 시뮬레이터의 1:1·파티·조합 비교·교체 비교·프리셋 비교 전부, 전투 로그.

한계: 이동·거리·투사체 시간은 없습니다. 실시간도 "행동 간격" 모델이라, 액션 게임의 프레임 단위 판정은 패턴 프레임 검증기(7장)로 보세요.

A.6.3 반대 가정으로 보는 기여도 — 효과 빼기와 교체 비교

기획 관점으로 풀어 보기

풀려는 문제

"버퍼가 정말 필요한가?"는 가한 피해만 보면 답이 틀립니다. 핀은 직접 피해가 파티에서 가장 적지만(3,495), 다른 사람의 피해를 늘리고 전투를 짧게 만듭니다. 한 사람의 몫을 보려면 그 사람이 없었다면 어땠을지를 계산해야 합니다(A.4.4의 반대 가정).

공통 예제 — 기본 파티 대 화염 골렘, 두 가지 반대 가정

방법	무엇을 빼나	결과	장단점
효과 빼기 (기여도 분석)	공격 한 번마다 남이 건 버프·디버프만 빼고 다시 계산	핀의 버프·디버프 기여: 판당 약 2,000	같은 판 안에서 계산해 잡음이 없지만, 전투가 짧아진 효과는 빠짐

방법	무엇을 빼나	결과	장단점
교체 비교	한 사람을 빼거나 바꾼 파티로 전투 전체를 다시	핀 빼기 −8.0%p (35.3% → 27.3%), 클리어 +7.0턴 / 다른 넷은 빼면 0%	모든 효과가 들어가지만 시뮬레이션을 다시 돌려야 함(같은 시드)

> 두 결과를 함께 읽으면 핀은 "없어도 이길 수는 있지만 이길 확률과 속도를 올리는" 역할이고, 나머지 넷은 한 명만 빠져도 클리어가 불가능한 핵심 역할입니다. 효과 빼기는 판마다 공정하게 몫을 나누고, 교체 비교는 "정말 없어도 되는가"에 답합니다.

원리와 공식

반대 가정 기여: 기여 = (실제 결과) − (그 요소가 없었다면의 결과). 효과 빼기는 데미지 한 번 단위로, 교체 비교는 전투 전체 단위로 이 차이를 구합니다.

```
효과 빼기:  기여 = 데미지(모든 효과) − 데미지(다른 아군의 버프·우리 진영의 디버프 제외)
        → 그 효과를 건 사람 수로 똑같이 나눠 각자에게 더함
교체 비교:  Δ승률 = 승률(바꾼 파티) − 승률(지금 파티)   (같은 시드, 판 수의 절반·최소 50판)
```

효과를 뺀 계산에서 명중·치명타·상성·그로기는 실제 판의 값을 그대로 쓰므로 버프·디버프의 몫만 남습니다. 교체 비교는 두 파티에 같은 난수열을 써서(공통 난수, A.1.4) 작은 차이도 운에 묻히지 않게 합니다.

워크벤치에서 쓰인 곳: 전투 시뮬레이터 기여도 분석의 "버프·디버프 기여", 교체 비교, 조합 비교.

한계: 버프를 여럿이 함께 걸면 똑같이 나누므로 누가 더 중요한지는 구분하지 않습니다. 교체 비교의 기본 150판에서 승률의 95% 범위는 ±8%p 안팎이라, 작은 차이는 판 수를 늘려 확인하세요.

A.7 패턴 프레임 검증기 (7장)

프레임 탭의 최적 회피 입력 계획과 "선입력 강제" 판정은 동적 계획법(A.1.3)으로 계산합니다. 상태는 "지금까지 처리한 타격 수", 값은 "다시 행동 가능한 가장 이른 프레임"입니다(7.6절). 프레임 탭만 쓰는 별도의 수학 방법은 없습니다.

A.8 성장 표 템플릿 (8장)

성장 표 탭은 확률을 쓰지 않습니다. 레벨 곡선과 하루 공급을 엑셀 수식과 같은 규칙으로 하루씩 앞으로 계산하고(8.8절), 마일스톤에 맞는 base와 증가율은 이분 탐색(A.1.4)으로 거꾸로 찾습니다.

기획 질문	방법	절
세그먼트마다 언제 몇 레벨이 되고, 어디서 멈추나?	일차별 전진 계산 (전날 행만 참조)	6.8
첫 마일스톤에 맞는 base는?	로그 이분 탐색 + 정수 이분 탐색	A.8.1
두 마일스톤을 함께 맞추는 증가율은?	중첩 이분 탐색	A.8.1

A.8.1 base와 증가율 자동 풀이 — 이분 탐색

기획 관점으로 풀어 보기

풀려는 문제 — 마일스톤에서 곡선을 거꾸로 찾기

"코어 유저가 14일차에 Lv30이 되려면 Lv1→2 필요량(base)을 얼마로 해야 하나?" base → 도달일은 하루씩 계산하면 나오지만, 도달일 → base는 공식으로 뒤집을 수 없습니다. 공급이 레벨에 따라 바뀌고(표 · % 방식), 해금 · 주간 · 1회 보상이 계단처럼 끼기 때문입니다. 대신 **base를 키우면 모든 레벨의 필요량이 같은 비율로 커지므로 도달일은 늦어지거나 그대로다(단조)**라는 성질이 있어, 범위를 반씩 좁혀 답을 찾을 수 있습니다.

공통 예제 — MMO형 프리셋, 코어 유저 Lv30을 14일차에

회차	범위	가운데 base	Lv30 도달	판단
1	1 ~ 1조	100만	미도달	늦음 → 아래 절반
2	1 ~ 100만	1,000	75일차	늦음 → 아래
3	1 ~ 1,000	31	4일차	14일 안 → 위 절반
4	31 ~ 1,000	176	17일차	늦음 → 아래
5	31 ~ 176	73	8일차	14일 안 → 위
6	73 ~ 176	113	11일차	14일 안 → 위
7	113 ~ 176	141	15일차	늦음 → 아래
8	113 ~ 141	126	13일차	14일 안 → 위

회차	범위	가운데 base	Lv30 도달	판단
9	126 ~ 141	133	13일차	14일 안 → 위
10	133 ~ 141	136	13일차	14일 안 → 위
11	136 ~ 141	138	14일차	14일 안 → 위
12	138 ~ 141	139	14일차	14일 안 → 위
13	139 ~ 141	140	14일차	14일 안 → base = 140

13번 만에 1 ~ 1조 범위에서 140을 찾았습니다. 처음 범위가 1조까지 넓어도 로그 가운데(√(lo × hi))로 나누면 자릿수가 반씩 줄어, 6번째에 이미 답 근처(113 ~ 176)로 옵니다. 정수 몇 개 사이만 남으면 보통 가운데((lo + hi) ÷ 2)로 마무리합니다.

왜 "가장 큰" base인가

도달일이 정수라 같은 14일차가 되는 base가 137 ~ 140으로 여러 개입니다(141부터 15일차). 가장 큰 값은 "목표 일차를 넘지 않는 범위에서 가장 느린 곡선"이라, 첫 마일스톤을 맞추면서도 이후 성장이 필요 이상 빨라지지 않습니다.

중첩 이분 탐색 — 두 번째 마일스톤까지

base 하나로는 마일스톤 하나만 맞출 수 있습니다. Lv50 60일차까지 맞추려면 증가율도 정해야 합니다. 모든 구간 증가율에 같은 배율 k를 곱하고, k마다 위 방법으로 base를 다시 맞춘 뒤 Lv50 도달일을 봅니다. 모든 행에서 Lv30 도달은 14일차입니다.

배율 k	증가율 (Lv1~40 / Lv41~)	맞춘 base	Lv50 도달
0.8	11.97% / 6.84%	214	42일차
0.9	13.46% / 7.70%	181	51일차
0.95	14.21% / 8.12%	160	56일차
1.0	14.96% / 8.55%	140	60일차
1.05	15.71% / 8.98%	130	67일차
1.1	16.46% / 9.41%	118	74일차

k가 커지면 초반 필요량(base)은 작아지지만 후반 필요량이 더 빨리 커져 Lv50 도달이 늦어집니다(단조). 그래서 바깥에서 k를 0 ~ 20 범위로 40번 이분 탐색하고, 안쪽에서 매번 base를 다시 풉니다. MMO형 프리셋의 증가율 14.96% / 8.55%와 base 140이 이렇게 푼 값입니다(k = 1).

원리와 공식

단조성: base가 커지면 필요량(L) = base × (증가율 · 점프로 정해지는 배수)가 모든 L에서 같은 비율로 커지므로(상한이 있으면 상한에서 멈춤), 공급이 같으면 도달일은 늦어지거나 같습니다. 공급이 모두 % 방식(현재 필요량에 비례)이면 공급도 같은 비율로 커져 도달일이 base와 무관해지므로, 이때는 풀 수 없다고 알려 줍니다.

로그 + 정수 이분 탐색: base 범위 1 ~ 10^{12}를 √(lo × hi)로 나누다가(A.1.4의 로그 이분 탐색), 정수 간격이 좁아지면 (lo + hi) ÷ 2로 마무리합니다. 도달일 ≤ 목표면 lo를, 아니면 hi를 옮겨 "목표 안에 닿는 가장 큰 정수 base"를 남깁니다.

중첩 이분 탐색: 바깥의 증가율 배율 k(0 ~ 20, 40회)마다 안쪽 base 풀이를 한 번씩 돌립니다. k는 소수 셋째 자리에서 내림해 두 번째 마일스톤의 목표 일차를 넘지 않게 합니다.

워크벤치에서 쓰인 곳: 성장 표의 base 맞추기, 증가율까지 맞추기 (8.6절).